

**COMSATS University Islamabad,
Sahiwal Campus, Pakistan**

SmartID Ecosystem

**A project presented to
COMSATS University Islamabad, Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Software Engineering (2022-2026)

By

Asmar Ahmad	CIIT/FA22-BSE-069/SWL
Iqra Suleman	CIIT/FA22-BSE-073/SWL
Abdul Ghaffar	CIIT/FA22-BSE-113/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Asmar Ahmad

Iqra Suleman

Abdulghaffar

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) **SmartID Ecosystem** was developed by **Asmar Ahmad (CIIT/FA22-BSE-069/SWL)**, **IQRA SULEMAN (CIIT/FA22-BSE-073/SWL)** and **ABDUL GHAFAR (CIIT/FA22-BSE-113/SWL)** under the supervision of **Ms. KANWAL FATIMA** and co supervisor **Mr. AHMAD SHA** and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

Supervisor

Ms. Kanwal Fatima

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr Ghulam Ali

Assistant Professor

Department of Computer Science

University of Okara

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

The increasing complexity of academic institutions and the growing need for secure, efficient campus management have necessitated intelligent automation solutions. Traditional attendance systems, reliant on manual processes or isolated semi-automated tools, are insufficient to meet the operational and security demands of modern universities. This project, the **SmartID Ecosystem**, addresses these challenges through a comprehensive, integrated campus identity and attendance management platform developed as a Final Year Project at COMSATS University Islamabad, Sahiwal Campus.

The **SmartID Ecosystem** is a hardware and web-based system that automates attendance recording, enforces secure campus access control, and provides real-time administrative oversight via a centralized web dashboard. It integrates three authentication modalities; facial recognition, fingerprint scanning, and RFID, to ensure accurate, tamper-proof identity verification, eliminating proxy attendance and manual errors.

A distinguishing feature is its IoT integration through Raspberry Pi-based smart locks, linking physical campus access to digital records in real time. A parental notification module automatically alerts guardians upon student absenteeism, while a machine learning-powered predictive analytics module provides administrators with insights into attendance trends and anomalous access patterns.

The system is built using Python and Flask for the backend, ReactJS for the frontend, and FaceNet with MTCNN for facial recognition. Biometric data is stored exclusively as AES-256 encrypted embeddings, ensuring raw data has never persisted. A digital consent module ensures users explicitly authorize biometric data usage prior to enrollment.

Development follows the Agile Scrum methodology across iterative sprints, with a modular architecture supporting scalability across campus facilities. The three-member team; **Asmar Ahmad**, **Iqra Suleman** and **Abdul Ghaffar**, works under the supervision of **Ms. Kanwal Fatima** across nine defined modules.

The SmartID Ecosystem aims to modernize campus operations, reduce administrative burden, enhance institutional security, and promote data-driven decision-making, establishing a foundation for intelligent campus management aligned with contemporary software engineering standards.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Ms. KANWAL FATIMA**” and our co-Supervisor “**Mr. AHMAD SHAF**”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Asmar Ahmad

Iqra Suleman

Abdul Ghaffar

Abbreviations

Abbreviation	Full Form
AES	Advanced Encryption Standard
API	Application Programming Interface
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ER	Entity-Relationship
FR	Functional Requirement
GPIO	General Purpose Input/Output
HRMS	Human Resource Management System
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
KPI	Key Performance Indicator
MQTT	Message Queuing Telemetry Transport
MTCNN	Multi-task Cascaded Convolutional Networks
NFR	Non-Functional Requirement
OOD	Object-Oriented Design
QR	Quick Response
RDBMS	Relational Database Management System
REST	Representational State Transfer
RFID	Radio Frequency Identification
SDLC	Software Development Life Cycle
SE	Software Engineering
SMTP	Simple Mail Transfer Protocol
SPI	Serial Peripheral Interface
SQL	Structured Query Language
SRS	Software Requirements Specification
UART	Universal Asynchronous Receiver-Transmitter
UI	User Interface
UML	Unified Modeling Language

Table of Contents

1	Introduction.....	1
1.1	Brief.....	3
1.1.1	Purpose of the Project	3
1.1.2	Scope of the system	3
1.1.3	Technologies Used.....	4
1.1.4	System Overview	4
1.1.5	Outcome of the Project	4
1.2	Relevance to Course Modules.....	5
1.2.1	Software Engineering Concepts.....	5
1.2.2	Software Requirement Engineering	5
1.2.3	Software Design and Architecture	5
1.2.4	Database Systems	6
1.2.5	Web Technologies	6
1.2.6	Human Computer Interaction	6
1.2.7	Object Oriented Programming	6
1.2.8	Machine Learning and Artificial Intelligence	6
1.2.9	Information Security	7
1.2.10	Computer Networks	7
1.2.11	Software Quality Engineering.....	7
1.2.12	Software Project Management	7
1.3	Project Background	8
1.4	Literature Review	9
1.5	Analysis from Literature Review.....	10
1.5.1	Alignment with Digital Campus Management Trends	11
1.5.2	Comparison with Biometric and IoT-Based Research.....	11
1.5.3	Relevance to Existing Attendance and Access Management Systems	11
1.5.4	Integration of Centralized Student Records and Smart ID.....	12
1.5.5	Application of Security and Access Control Standards	12
1.5.6	Involvement of Mobile and Web Application Concepts	12
1.5.7	Addressing Limitations of Existing Campus Technologies.....	12
1.6	Methodology and Software Lifecycle.....	13
1.6.1	Methodology	13
1.6.2	Agile Practices Applied	14
1.6.3	Software Development Life Cycle (SDLC) Model	14
1.6.4	Product Backlog Management	14
1.6.5	Incremental Delivery	14
1.6.6	Rationale Behind Selected Methodology.....	15
1.6.7	Development Tools and Environment	15

1.6.8	Testing and Quality Assurance	15
1.6.9	Advantages of the Selected Methodology and SDLC	16
2	Problem Definition.....	17
2.1	Problem Statement	18
2.2	Deliverables and Development Requirements	18
2.2.1	Deliverables	18
2.2.2	Development Requirements	19
2.3	Current System	20
2.3.1	Manual Attendance Recording	21
2.3.2	Inadequate Physical Access Control	21
2.3.3	Fragmented Student Identity Management	21
2.3.4	Lack of Real-Time Administrative Visibility	22
2.3.5	Security and Privacy Concerns	22
2.3.6	Poor Integration Between Administrative Modules.....	22
2.3.7	Summary of Limitations	23
2.4	Summary	23
3	Requirement Analysis.....	25
3.1	Introduction to Requirement Analysis	25
3.2	System Overview	25
3.3	Actors in the System	26
3.3.1	Administrator	26
3.3.2	Professor	27
3.3.3	Student	27
3.3.4	Parent	27
3.4	Use Cases Diagram	27
3.5	Use Case Descriptions.....	29
3.5.1	Authenticate User.....	29
3.5.2	Record Attendance.....	30
3.5.3	Authorize Access	30
3.5.4	Grant Physical Access.....	31
3.5.5	Enroll User	32
3.5.6	Manage Users	33
3.5.7	Manage Access Permissions	33
3.5.8	Generate Reports.....	34
3.5.9	Monitor System Activity.....	35
3.5.10	View Attendance Records.....	35
3.5.11	Login to Parent Portal	36
3.5.12	View Attendance Reports	37
3.5.13	Provide Consent.....	37
3.5.14	Receive Attendance Notifications.....	38

3.5.15	View Student Attendance	39
3.6	Functional Requirements.....	40
3.6.1	User Management	40
3.6.2	Biometric Enrollment and Identity Verification	41
3.6.3	Attendance Management	41
3.6.4	IoT-Based Access Control	41
3.6.5	Smart ID Card Management	42
3.6.6	Leave Request Management	42
3.6.7	Parent Portal.....	42
3.6.8	Notifications and Alerts	43
3.6.9	Report Generation and Analytics.....	43
3.6.10	System and Admin Management	43
3.7	Non-Functional Requirements	44
3.7.1	Performance	44
3.7.2	Scalability	44
3.7.3	Security	44
3.7.4	Reliability and Availability.....	45
3.7.5	Usability.....	45
3.7.6	Maintainability.....	45
3.7.7	Compliance	46
3.7.8	Backup and Recovery	46
3.7.9	Performance Monitoring and Logging.....	46
4	Design and Architecture	47
4.1	System Architecture	47
4.1.1	Architectural Style	47
4.1.2	Components	47
4.1.3	Interactions.....	49
4.1.4	System Architecture Diagram.....	50
4.2	Data Representation.....	50
4.2.1	Database Design	51
4.2.2	Data Flow.....	52
4.3	Process Flow/Representation	53
4.4	Design Models.....	55
4.4.1	Class Diagram.....	55
4.4.2	Sequence Diagrams.....	56
4.4.3	State Transition diagrams	61
4.4.4	Timing Diagram.....	67
4.4.5	Schematic Diagram.....	67
5	Implementation	68
5.1	Algorithm	68

5.1.1	Multi-Modal Authentication Algorithm	68
5.1.2	Automated Attendance Logging Algorithm	69
5.1.3	IoT-Based Smart Access Control Algorithm	70
5.1.4	Parental Notification Algorithm	71
5.1.5	Predictive Analytics Algorithm	71
5.1.6	User Enrollment Algorithm	72
5.1.7	Privacy and Consent Management Algorithm	73
5.2	External APIs.....	74
5.3	User Interface	76
6	Testing and Evaluation.....	83
6.1	Manual Testing	83
6.1.1	System testing	83
6.1.2	Unit Testing	83
6.1.3	Functional Testing.....	86
6.1.4	Integration Testing	89
6.2	Automated Testing:.....	90
7	Conclusion and Future Work.....	91
7.1	Conclusion.....	91
7.2	Future Work.....	91
7.2.1	Mobile Application Integration.....	91
7.2.2	Multi-Campus Deployment	92
7.2.3	QR Code and Voice Authentication	92
7.2.4	Learning Management System Integration.....	92
7.2.5	Advanced Predictive Analytics and AI Insights	92
7.2.6	Enhanced Security and Privacy Features	92

List of Figures

Figure 3.1: Use Case Diagram.....	28
Figure 4.1: Architecture Diagram of SmartID Ecosystem	50
Figure 4.2: ER Diagram of SmartID Ecosystem	51
Figure 4.3: DFD Level 0.....	52
Figure 4.4: DFD Level 1.....	53
Figure 4.5: Activity Flow Diagram	54
Figure 4.6: Class diagram of SmartId Ecosystem	55
Figure 4.7: Sequence Diagram of User Enrollment Module	56
Figure 4.8: Sequence Diagram of Multi-Modal Authentication.....	57
Figure 4.9: Sequence Diagram of Real Time Attendance	57
Figure 4.10: Sequence Diagram of IOT Based Smart Access Control	58
Figure 4.11: Sequence Diagram of Admin Dashboard.....	58
Figure 4.12: Sequence Diagram of Notification Module	59
Figure 4.13: Sequence Diagram of Consent Management	60
Figure 4.14: Sequence Diagram of Predictive Analytics	60
Figure 4.15: system Settings and configuration Module	61
Figure 4.16: State Transition Diagram of User Authentication.....	62
Figure 4.17: State Transition Diagram of Attendance Record Lifecycle	63
Figure 4.18: State Transition Diagram of Smart access	64
Figure 4.19: State Transition Diagram of Notification.....	65
Figure 4.20: State Transition Diagram of Consent Managemnet.....	66
Figure 4.21: Timing Diagram of SmartID Ecosystem	67
Figure 4.22: Schematic Diagram of SmartID Ecosystem.....	67
Figure 5.1: Login page.....	76
Figure 5.2: Admin Dashboard	77
Figure 5.3: User Enrollment Page	77
Figure 5.4: Statistics of Attendance.....	78
Figure 5.5: Leave Request	79
Figure 5.6: Teaching Analytics	79
Figure 5.7: Notifications.....	80
Figure 5.8: Attendance Session	80
Figure 5.9: Attendance reports	81
Figure 5.10: Parent Dashboard	82

List of Tables

Table 1.1: Comparison of Existing Systems with the SmartID Ecosystem.....	10
Table 3.1: Authenticate user use case	29
Table 3.2: Record Attendance Use Case.....	30
Table 3.3: Authorize Access Use case	31
Table 3.4: Physical Access Use case	31
Table 3.5: Enrollment use case	32
Table 3.6: User Management Use case.....	33
Table 3.7: Permission access Management use case	34
Table 3.8: Report Generation use case	34
Table 3.9: Monitoring Activities use case	35
Table 3.10: Attendance record use case.....	36
Table 3.11: Parental Login use case	36
Table 3.12: Attendance Reports use case	37
Table 3.13: Consent use case	38
Table 3.14: Notification use case.....	39
Table 3.15: Attendance parental view use case	39
Table 5.1: External API's used in our SmartID Ecosystem	74
Table 6.1: User Enrollment Unit Testing.....	84
Table 6.2: Facial Recognition unit testing	84
Table 6.3: RFID unit testing	85
Table 6.4: Real Time Attendance Unit Testing	85
Table 6.5: IOT Based Access Unit Testing	85
Table 6.6: Noification Unit Testing	86
Table 6.7: Privacy & consent Unit Testing.....	86
Table 6.8: Role Based Login Functional Testing	87
Table 6.9: Repeated Login Attempts Functional Testing	87
Table 6.10: Attendance Report Generation Functional Testing	88
Table 6.11: Analysis risk Functional Testing	88
Table 6.12: IOT Access Denial Functional Testing.....	89
Table 6.13: Integration Testing.....	89
Table 6.14: Automated Testing.....	90

Chapter 01

INTRODUCTION

1 Introduction

Educational institutions across Pakistan continue to operate under attendance and identity management systems that are largely manual, fragmented, and ill-equipped to meet the demands of a growing student population. Universities and colleges rely on faculty-driven roll calls, paper registers, and basic identification cards to track student presence and regulate campus access. These conventional approaches are not only time-consuming and error-prone, but also vulnerable to manipulation through proxy attendance and unauthorized entry. As campuses expand in size and complexity, encompassing multiple departments, laboratories, libraries, and restricted facilities, the inadequacy of these methods become increasingly apparent. The need for a reliable, automated, and integrated identity management solution within Pakistani educational institutions has never been more pressing.

In recent years, Pakistan has witnessed significant digital transformation across sectors such as banking, e-commerce, and telecommunications. However, educational institutions have largely been left behind in this shift. While some universities have introduced online portals to partially automate attendance marking, these systems still depend on manual input from faculty members, lack integration with other campus systems, and offer no mechanism for identity verification or physical access control. The absence of a unified digital ecosystem means that attendance records, access logs, and student data remain scattered across disconnected platforms, making real-time monitoring, accountability, and data-driven decision-making difficult to achieve. A comprehensive, automated campus management solution is needed to bridge this gap.

The SmartID Ecosystem is developed as a direct response to these challenges within the context of Pakistan's educational environment. It is an integrated hardware and web-based platform designed to automate attendance management, enforce identity verification, and control campus access through a single, cohesive system. The purpose of the SmartID Ecosystem is to eliminate manual processes, reduce administrative burden, improve campus security, and provide institutions with the tools necessary for real-time monitoring and informed decision-making. Whether deployed at a large university in Islamabad or a regional campus in Sahiwal, the system is designed to be scalable, inclusive, and adaptable to varying institutional environments.

The system is built on a modern and robust technology stack to ensure reliability, performance, and extensibility. The frontend is developed using ReactJS, providing a responsive and intuitive web-based dashboard for administrators and faculty. The backend is implemented in Python with the Flask framework, offering lightweight and modular architecture suited to the system's diverse

functional requirements. For artificial intelligence components, the system leverages FaceNet and MTCNN for facial recognition and embedding generation, while TensorFlow and Keras support machine learning-based predictive analytics. IoT integration is achieved through Raspberry Pi and Arduino, enabling the system to interface directly with smart locks, biometric sensors, and RFID readers. Together, these technologies form a cohesive foundation capable of serving a large and technically diverse academic environment.

The SmartID Ecosystem enables students and staff to enroll through a secure web portal by registering their biometric data, including facial embeddings, fingerprint scans, or RFID credentials, in encrypted form. Once enrolled, users are authenticated through a multi-modal verification process that supports facial recognition, fingerprint scanning, and RFID-based identification. Upon successful authentication, attendance is automatically recorded and synchronized with the cloud in real time, removing the need for any manual input from faculty. The system also empowers parents and guardians by sending automated notifications whenever a student misses a class, addressing a critical accountability gap that existing systems have long overlooked.

From an administrative perspective, the system provides a centralized dashboard through which administrators and faculty can access attendance records, monitor access logs, generate analytical reports, and identify absenteeism trends. IoT-enabled smart locks integrated with Raspberry Pi sensors allow physical access to classrooms, laboratories, and restricted areas to be controlled and logged digitally, linking campus infrastructure directly to the attendance management platform. Privacy and data security are upheld through AES-256 encryption, HTTPS protocols, and a dedicated consent management module that gives users control over their biometric data and authentication preferences.

The administrative layer of the SmartID Ecosystem ensures institutional oversight, security compliance, and operational efficiency across all modules. System administrators can manage user roles, configure authentication methods, update hardware device settings, and monitor system activity through a centralized settings and configuration module. The predictive analytics module further enhances decision-making by using machine learning to forecast absenteeism patterns and flag unusual access behaviors, giving administrators actionable intelligence rather than raw data alone. Together, these capabilities reduce dependency on human oversight while ensuring that campus operations remain transparent, secure, and accountable.

Overall, the SmartID Ecosystem aims to modernize campus identity and attendance management in Pakistan by delivering an intelligent, unified platform that connects students, faculty, administrators, and parents within a single digital ecosystem. The system reduces reliance on

manual and fragmented processes, minimizes the risks associated with proxy attendance and unauthorized access, and provides a scalable solution that can grow alongside the institution it serves. By addressing the specific operational and security challenges faced by Pakistani universities, the SmartID Ecosystem represents a practical, forward-looking advancement that has the potential to enhance academic integrity, administrative efficiency, and campus security across the country.

1.1 Brief

The SmartID Ecosystem is designed as an integrated campus identity and attendance management platform that connects students, faculty, administrators, and parents through a unified hardware and web-based solution. The aim of the project is to overcome the limitations of manual and semi-automated attendance systems, where student presence is recorded inconsistently, identity verification is absent, and campus access remains uncontrolled and unlinked to digital records. The SmartID Ecosystem consolidates authentication, attendance logging, access control, analytics, and parental communication into a single cohesive platform, ensuring accuracy, security, real-time monitoring, and administrative efficiency across the institution. The system leverages modern biometric technologies, IoT hardware integration, and intelligent web-based tools to deliver reliable, scalable, and inclusive campus management experience.

1.1.1 Purpose of the Project

The primary purpose of the SmartID Ecosystem is to automate and centralize campus attendance and access management by offering an all-in-one solution for biometric authentication, real-time attendance recording, IoT-based access control, and data-driven reporting. The project aims to eliminate the inefficiencies of faculty-dependent manual attendance marking by replacing it with a fully automated, multi-modal verification process that requires no human intervention. It also seeks to enhance institutional accountability by notifying parents when students are absent, providing administrators with real-time analytics, and securing all biometric and identity data through privacy-conscious design principles.

1.1.2 Scope of the system

The scope of the SmartID Ecosystem extends across both hardware and software components, including a web-based administrative dashboard, IoT-enabled smart access devices, biometric sensors, and RFID readers. The project handles various core functionalities such as user enrollment, multi-modal authentication through facial recognition, fingerprint scanning and RFID identification, automated attendance logging, smart lock control, absenteeism notifications, predictive analytics, and consent-based biometric data management. By covering

end-to-end campus identity and attendance operations, the system aims to serve as a complete digital and physical solution for universities and educational institutions. The scope is limited to attendance automation and campus access control; advanced features such as multi-campus integration or AI-based behavioral profiling are considered outside the current scope.

1.1.3 Technologies Used

The SmartID Ecosystem is developed using ReactJS for the web-based administrative frontend, providing a responsive and modular interface suitable for real-time data display and user management. The backend is built using Python with the Flask framework, offering a lightweight, modular, and easily extensible server architecture. For biometric intelligence, the system employs FaceNet and MTCNN for facial detection and embedding generation, while TensorFlow and Keras support the machine learning components used in predictive analytics. IoT integration is implemented through Raspberry Pi and Arduino, enabling direct communication with smart locks, fingerprint sensors, and RFID readers. Together, these technologies form a robust and scalable foundation for a campus management platform capable of handling diverse hardware and software requirements simultaneously.

1.1.4 System Overview

The SmartID Ecosystem operates as an interconnected system that serves users across multiple roles. Students and staff can enroll through a secure web portal by registering their biometric credentials, after which the system automatically verifies their identity and records attendance upon each authenticated entry. Administrators and faculty can access a centralized dashboard that displays real-time attendance data, access logs, absenteeism trends, and user activity reports, along with tools to export records and manage system settings. The IoT module interfaces with smart locks and sensors to grant or restrict physical access based on authentication outcomes, linking campus infrastructure directly to digital records. Parents and guardians receive automated notifications whenever a student is marked absent, ensuring continuous accountability beyond the classroom. The system's predictive analytics module further processes attendance history to surface patterns and anomalies, supporting proactive institutional decision-making.

1.1.5 Outcome of the Project

The project results in a complete campus identity ecosystem that improves accuracy, security, and efficiency of attendance management and access control. The automated and centralized structure eliminates proxy attendance, reduces faculty workload, prevents data inconsistencies, and strengthens communication between the institution and student families. The outcome is a secure, intelligent, and scalable digital-physical environment where identity verification, attendance recording, and campus access operate seamlessly as a unified system, enabling administrators to make data-driven decisions and allowing institutions to uphold academic integrity, operational transparency, and modern campus security standards.

1.2 Relevance to Course Modules

The SmartID Ecosystem is strongly connected with the core subjects studied throughout the Bachelor of Science in Software Engineering program at COMSATS University Islamabad. Every stage of the system, from requirement gathering and hardware integration to implementation, testing, and deployment, reflects concepts that were formally taught and assessed during the degree. The project demonstrates how knowledge accumulated across multiple semesters can be applied to develop a complete, automated campus management solution that integrates biometric authentication, IoT-based access control, machine learning, and a web-based administrative platform. The following subheadings explain the contribution of major course modules to the project.

1.2.1 Software Engineering Concepts

Software Engineering Concepts played a fundamental role in shaping the structure and development process of the SmartID Ecosystem. Concepts such as requirement analysis, system modeling, documentation standards, and software development lifecycle methodologies were applied directly throughout the project. The development follows structured practices including use case modeling, modular system design, and systematic testing, all of which are core elements covered in this course. The understanding of functional and non-functional requirements, combined with disciplined design techniques, allowed the system to be built in an organized, traceable, and maintainable manner.

1.2.2 Software Requirement Engineering

Software Requirement Engineering provided the formal foundation for identifying, documenting, and validating the needs of all stakeholders involved in the SmartID Ecosystem. Techniques for eliciting requirements from students, faculty, administrators, and parents were applied to define the system's functional scope, constraints, and acceptance criteria. This course ensured that the project began with a clear and unambiguous understanding of what the system must achieve, reducing the risk of scope creep and misaligned development throughout the lifecycle.

1.2.3 Software Design and Architecture

Software Design and Architecture directly informed the structural decisions behind the SmartID Ecosystem. Concepts such as layered architecture, modular decomposition, component design, and interface specification were applied to organize the system into clearly defined and independently functioning modules. The separation of the authentication layer, attendance logging engine, IoT control interface, and administrative dashboard reflects the architectural principles studied in this course, ensuring

that the system remains maintainable, extensible, and scalable.

1.2.4 Database Systems

Database Systems provided the knowledge required to design and implement a secure and structured database for managing student profiles, biometric embeddings, attendance records, access logs, and notification histories. The schema design reflects core concepts such as indexing, data retrieval, relational integrity, and query optimization. Topics such as secure data storage, access control, and efficient transaction management were directly applied to ensure that sensitive biometric and attendance data is stored safely and retrieved reliably under real-time operational conditions.

1.2.5 Web Technologies

The development of the administrative web dashboard is directly supported by the concepts learned in Web Technologies. The use of ReactJS, component-based development, responsive layout design, state management, and API integration aligns with the principles introduced in this course. It provided the foundation for building a dynamic, interactive, and real-time interface through which administrators and faculty can monitor attendance, manage users, configure system settings, and generate analytical reports efficiently.

1.2.6 Human Computer Interaction

Human Computer Interaction guided the design of all user-facing interfaces within the SmartID Ecosystem. The principles of simplicity, consistency, clarity, and task-oriented navigation were applied to ensure that administrators, faculty, students, and parents can interact with the system comfortably and without unnecessary complexity. This course helped in designing an enrollment portal and admin dashboard that are visually balanced, logically structured, and well-suited to the operational demands of a real campus environment.

1.2.7 Object Oriented Programming

Object Oriented Programming contributed significantly to organizing the logic and structure of the SmartID Ecosystem. Concepts such as classes, objects, inheritance, abstraction, and modularity were applied across both the backend Flask application and the frontend ReactJS components. The modular architecture of the system, where each functional unit such as authentication, attendance logging, and access control operates independently, directly reflects the object-oriented design principles studied and practiced in this course.

1.2.8 Machine Learning and Artificial Intelligence

The predictive analytics and biometric authentication components of the SmartID Ecosystem draw directly from the concepts introduced in Machine Learning and Artificial Intelligence. The

use of FaceNet, and MTCNN for facial recognition, biometric embedding generation, and absenteeism pattern forecasting reflects the application of deep learning, neural network architecture, and model training techniques studied in these courses. They equipped the team with the knowledge necessary to develop intelligent, data-driven features that extend the system beyond basic automation into proactive institutional decision-making.

1.2.9 Information Security

The SmartID Ecosystem handles highly sensitive biometric and identity data, making Information Security a critical contributor to the project. Concepts such as authentication, authorization, secure communication, and encryption were directly applied through AES-256 encryption for biometric embeddings, HTTPS protocols for all data transmission, and role-based access controls across the platform. This course provided a thorough understanding of security threats and countermeasures, enabling the team to design a system that is both functionally robust and ethically responsible in its handling of personal data.

1.2.10 Computer Networks

The system architecture of the SmartID Ecosystem relies on continuous client-server communication, cloud synchronization, IoT device networking, and real-time data flow, all of which were studied in Computer Networks. This course helped in understanding how hardware components such as Raspberry Pi sensors and smart locks interact with the web backend over a network, how API requests are processed efficiently, and how distributed data remains consistent under concurrent access. Concepts such as latency management, network reliability, and request-response handling directly influenced the overall system design and deployment strategy.

1.2.11 Software Quality Engineering

Software Quality Engineering ensured that the SmartID Ecosystem was developed with a consistent focus on correctness, reliability, and testability. Concepts such as test planning, unit testing, integration testing, defect tracking, and quality metrics were applied throughout the development lifecycle. This course helped in establishing verification and validation practices that confirmed each module, from biometric authentication to IoT access control, performs as intended under a variety of operational conditions.

1.2.12 Software Project Management

Software Project Management provided the framework for planning, tracking, and delivering the SmartID Ecosystem within defined scope, time, and resource constraints. Concepts such as project scheduling, risk management, task decomposition, and team coordination were applied through the Agile Scrum methodology adopted for the project. This course ensured that development activities were systematically organized across sprints, with clear milestones,

deliverables, and accountability for each team member.

1.3 Project Background

The idea behind the SmartID Ecosystem originates from the increasing need for a reliable, automated, and integrated identity and attendance management system within educational institutions. Traditional campus management practices rely heavily on manual processes, paper-based registers, and faculty-dependent attendance marking. In most universities, student presence is recorded by hand during each class session, identity verification is absent or limited to basic ID cards, and access to campus facilities remains largely uncontrolled. This creates room for proxy attendance, unauthorized entry, administrative inconsistencies, and a general lack of accountability that affects both academic integrity and campus security.

With the rapid growth of technology and the widespread availability of biometric sensors, IoT devices, and web-based platforms, educational environments now have the means to automate and integrate processes that were previously handled manually. The concept of a centralized campus management ecosystem focuses on connecting students, faculty, administrators, and parents through a single, unified digital and physical platform. This idea aims to eliminate the fragmentation between identity verification, attendance recording, and access control by allowing all stakeholders to interact with the system in a structured, automated, and secure way. The project idea developed further through an analysis of the specific challenges faced by institutions in managing student attendance accurately and fairly. Faculty members spend valuable class time calling registers, and even digitized portals still depend on manual input, leaving them vulnerable to human error and deliberate manipulation through proxy attendance. Parents and guardians also remain largely uninformed about their children's presence on campus unless contacted directly. By creating a centralized and automated system, the SmartID Ecosystem ensures that attendance is recorded instantly upon authenticated entry, that records remain accurate and tamper-proof, and that parents receive real-time notifications whenever a student is absent, closing a critical accountability gap that existing systems have consistently failed to address.

The idea was also shaped by the need for stronger and more reliable identity verification on campus. Basic ID cards are easily lost, borrowed, or misused, and single-factor systems such as standalone fingerprint scanners are prone to failure under variable environmental or hardware conditions. The SmartID Ecosystem responds to this by introducing multi-modal authentication, combining facial recognition, fingerprint scanning, and RFID-based identification, so that no single point of failure can compromise the verification process. This approach improves both the reliability of identity confirmation and the overall security of campus access.

Another important factor behind the project idea is the growing demand for intelligent, data-driven campus management. Traditional attendance systems generate raw records but offer no analytical insight, making it difficult for administrators to identify chronic absenteeism, detect unusual access patterns, or allocate campus resources effectively. The SmartID Ecosystem incorporates a predictive analytics module powered by machine learning, enabling the system to forecast absenteeism trends and surface anomalies proactively. This transforms attendance data from a passive record into an active tool for institutional decision-making.

Overall, the idea for the SmartID Ecosystem is based on creating a modern, integrated campus management platform that addresses the limitations of manual and semi-automated attendance systems. The project aims to bring accuracy, accountability, and intelligence to campus operations by unifying identity verification, attendance recording, access control, and analytical reporting into a single interconnected ecosystem. It reflects how technology, when thoughtfully applied to real institutional challenges, can enhance academic integrity, reduce administrative burden, and create a safer and more transparent campus environment for students, faculty, and administrators alike.

1.4 Literature Review

The reviewed literature shows that several campus attendance and access management systems offer features such as biometric authentication, automated attendance recording, and basic administrative reporting. However, each of these platforms also contains important weaknesses that limit their ability to provide a complete, integrated, and campus-specific management solution. Many of these systems were originally designed for general enterprises or corporate environments and have since been adapted for academic use, which often results in a poor fit with the unique operational demands of university campuses. As a result, institutions are frequently forced to rely on multiple disconnected tools to cover different aspects of campus management, leading to data fragmentation, administrative inefficiencies, and increased operational costs. Furthermore, the lack of real-time data synchronization across departments means that decision-makers often work with outdated or incomplete information, which undermines the reliability of the overall system. To clearly highlight these gaps, the following table compares well-known attendance and access management systems with the proposed SmartID Ecosystem. Table 1.1 presents three key aspects for each application, including the application name, the major weakness identified through literature and practical usage, and the specific solution provided by the SmartID Ecosystem to overcome that weakness. This comparison helps demonstrate why a unified, biometric-driven, and IoT-integrated campus management platform is necessary and how the SmartID Ecosystem effectively addresses the

limitations found in existing systems. By consolidating these functions into a single cohesive platform, the SmartID Ecosystem not only eliminates the redundancies present in current solutions but also establishes a scalable foundation for future enhancements in smart campus infrastructure.

Table 1.1: Comparison of Existing Systems with the SmartID Ecosystem

Application Name	Weakness	Proposed Project Solution
Jibble	Facial recognition may be less accurate under variable lighting and uses limited biometric depth; can still allow proxy inputs if not tightly configured. [1]	SmartID uses multi-modal authentication (facial, fingerprint, RFID) to improve reliability and accuracy across environments.
Buddy Punch	Relies on webcam/selfie capture without true biometric verification, requiring manual review and vulnerable to manipulation. [2]	SmartID incorporates true biometric verification hardware/software to eliminate reliance on manual review.
TimeClock Plus	Requires specialized hardware and is costly; not ideal for environments without existing infrastructure. [3]	SmartID uses low-cost IoT hardware (e.g., Raspberry Pi + sensors) to reduce cost while maintaining accuracy.
SumHR	Limited advanced analytics and real-time insight. [3]	SmartID provides real-time analytics dashboards for administrators and automated reporting.
Pocket HRMS	Not specifically designed for educational attendance; features oriented toward HR workflows. [4]	SmartID offers education-centric attendance and access features including class presence tracking and absentee alerts.
AttenFace	Not commercialized and scalability issues for real deployment. [5]	SmartID is a fully implemented, scalable system, not just prototype research.
eSSL Face Recognition	Heavy reliance on hardware terminals and upfront cost. [6]	SmartID offers low-cost IoT endpoints with web integration, reducing hardware dependency.
ZKTeco Attendance Devices	Requires physical devices and can lack remote reporting without middleware. [7]	SmartID integrates web dashboards and cloud connectivity for remote monitoring.

1.5 Analysis from Literature Review

The analysis of the reviewed literature demonstrates how the SmartID Ecosystem aligns with global trends in digital campus management and how it specifically addresses the existing gaps within educational institutions' attendance, access control, and administrative operations. The literature emphasizes the need for centralized student data management, secure biometric authentication, efficient attendance workflows, and integrated IoT-based access systems. These findings support the core features implemented in the SmartID Ecosystem and validate the relevance of developing such a system within an academic environment where manual and fragmented management practices remain prevalent. The comparison between existing research

and the functionalities of the SmartID Ecosystem shows that the project is not only aligned with modern campus management solutions but also designed to overcome limitations highlighted in previous studies.

1.5.1 Alignment with Digital Campus Management Trends

The literature identifies a major shift from paper-based and manual attendance systems to electronic, biometric, and IoT-integrated platforms across educational institutions worldwide. Research highlights that digital campus management solutions reduce administrative errors, improve real-time visibility of student presence, and enhance communication between students, faculty, and administrative staff. The SmartID Ecosystem aligns strongly with these trends by offering a centralized platform for managing attendance records, access control, and student identification. This alignment shows that the project is built upon well-established research and the proven benefits of digital transformation in academic environments. The system incorporates the same principles discussed in the literature, such as improved accuracy, real-time data availability, and reduced dependency on manual processes.

1.5.2 Comparison with Biometric and IoT-Based Research

Studies on biometric and IoT-based campus solutions highlight advantages such as automation, real-time monitoring, and reduced scope for proxy attendance. The literature also stresses the importance of secure and reliable identity verification mechanisms in institutional environments. The SmartID Ecosystem reflects these findings by incorporating fingerprint-based biometric authentication and IoT-integrated gate access control within a unified platform. The system allows administrators to monitor student and staff presence in real time, which is consistent with research on automated campus surveillance and access management. The analysis shows that the SmartID Ecosystem adopts the strengths of biometric and IoT technologies discussed in the literature and applies them to a practical, campus-specific solution. [8]

1.5.3 Relevance to Existing Attendance and Access Management Systems

Research on platforms such as BioTime, Suprema, and other institutional attendance tools emphasizes the importance of automating attendance recording and controlling physical access to restricted areas. These systems mainly focus on hardware-level identification and basic reporting, with limited integration into broader campus workflows. The literature also mentions limitations of these platforms, such as the absence of complete student profile integration and the lack of unified administrative dashboards. The SmartID Ecosystem extends beyond the functionality of these existing systems by integrating attendance tracking, gate access control, fee management, and student records within a single cohesive platform. Unlike many systems mentioned in the literature, the SmartID Ecosystem also provides role-based access for administrators, faculty, and students, ensuring that each stakeholder interacts with the system in a controlled and purposeful manner.

1.5.4 Integration of Centralized Student Records and Smart ID

Studies show that centralized student record systems improve institutional efficiency, reduce duplication of data entry, and ensure reliable long-term storage of academic and administrative information. The literature also identifies issues with physical ID cards and manual registers, such as loss, forgery, and inconsistent record-keeping. The SmartID Ecosystem directly addresses these concerns through digital smart ID issuance and centralized student profile management. The project incorporates every aspect mentioned in the literature, including structured attendance histories, access logs, and easy accessibility for both administrative staff and faculty. The analysis highlights that the SmartID Ecosystem builds upon established research while adding a layer of smart identification tailored specifically to the operational context of educational institutions.

1.5.5 Application of Security and Access Control Standards

The literature strongly focuses on security risks associated with campus management systems and the need for strict authentication, role-based authorization, and secure data handling mechanisms. Many studies mention that unauthorized physical access to restricted areas and manipulation of attendance records are major concerns in institutional environments. The SmartID Ecosystem implements security mechanisms that match these recommendations, such as biometric verification, role-based system access, IoT-controlled gate locking, and secure storage of student and staff data. The analysis shows that the project is developed in line with recognized campus security guidelines, ensuring that the system remains reliable, tamper-resistant, and trustworthy for institutional use. [9]

1.5.6 Involvement of Mobile and Web Application Concepts

Research on mobile and web-based institutional platforms suggests that accessible digital interfaces significantly increase user engagement, reduce administrative workload, and support better communication between students and institutional management. The SmartID Ecosystem incorporates these findings by offering a dedicated web-based administrative panel alongside a mobile application for students and faculty. The features included in these interfaces, such as attendance viewing, access history, leave requests, and notification alerts, reflect the benefits discussed in the literature on digital campus portals. The analysis shows that the SmartID Ecosystem effectively translates research findings into real-world, user-facing functionalities suited to an academic environment.

1.5.7 Addressing Limitations of Existing Campus Technologies

The literature identifies several shortcomings in current campus management solutions, such as decentralized attendance records, lack of real-time access monitoring, poor integration between administrative modules, and vulnerability to proxy attendance and unauthorized entry. The SmartID Ecosystem is designed specifically to overcome these issues by centralizing all campus

data, integrating biometric hardware with software workflows, ensuring secure identity verification, and providing a consistent interface for all users across different roles. The comparison indicates that the SmartID Ecosystem addresses each limitation highlighted in the literature and offers a more complete, unified, and intelligent campus management ecosystem. The analysis shows that the SmartID Ecosystem is built upon strong foundations supported by research and technological advancements discussed in literature. The project incorporates the strengths of existing digital campus systems while addressing their weaknesses and adapting them to the specific needs of educational institutions. Each reviewed topic, including biometric authentication, IoT integration, student record management, security practices, and mobile accessibility, directly influences the system design and validates the necessity of the SmartID Ecosystem as a modern solution. The comparison confirms that the project aligns with international standards for campus management while offering features tailored to the practical challenges faced by academic institutions today.

1.6 Methodology and Software Lifecycle

The development of the SmartID Ecosystem follows a structured approach that combines modern software engineering practices with a suitable Software Development Life Cycle (SDLC) model. Considering the complexity of the system, which integrates a web-based administrative panel, a mobile application for students and faculty, biometric hardware modules, and IoT-controlled access gates, a methodology that supports iterative development, continuous feedback, and adaptive planning was deemed essential. This approach ensures that all components of the system are thoroughly developed, rigorously tested, and effectively integrated into a cohesive smart campus management ecosystem. By following a clear methodology, the project team can manage risk, improve productivity, and deliver a robust system that meets the functional, non-functional, and security requirements of modern educational institutions.

1.6.1 Methodology

The SmartID Ecosystem adopts the Agile methodology, which emphasizes iterative development, incremental delivery, and close collaboration among team members. Agile supports continuous reassessment of requirements and frequent delivery of working software, making it ideal for campus management applications where requirements may evolve due to feedback from faculty, administrative staff, institutional regulations, or hardware integration challenges. The flexibility of Agile ensures that both software and hardware components of the SmartID Ecosystem are developed in coordination and refined progressively throughout the project lifecycle.

1.6.2 Agile Practices Applied

In the SmartID Ecosystem, Agile practices are implemented through short development cycles called sprints. Each sprint focuses on a specific set of modules, such as biometric enrollment, attendance recording, IoT gate access control, student profile management, and administrative reporting. Iterative development allows the team to implement, test, and refine features progressively. Feedback from supervisors, institutional staff, and end users is integrated after each sprint, ensuring that the system aligns with real-world campus workflows. Agile practices also include regular stand-up meetings, sprint planning sessions, and sprint reviews to monitor progress, address integration challenges between hardware and software components, and plan improvements for subsequent development cycles.

1.6.3 Software Development Life Cycle (SDLC) Model

The project adopts the Scrum model of Agile SDLC due to its suitability for multi-component software development involving parallel workstreams. Scrum divides the project into time-boxed sprints, typically lasting two to four weeks, during which specific tasks are planned, developed, tested, and reviewed. The Scrum framework emphasizes transparency, accountability, and collaboration, ensuring efficient management of tasks across the web administrative panel, mobile application, biometric hardware integration, and IoT access control modules of the SmartID Ecosystem.

1.6.4 Product Backlog Management

Scrum methodology relies heavily on a product backlog, which contains a prioritized list of features, enhancements, bug fixes, and system requirements. In the SmartID Ecosystem, the backlog includes modules such as biometric fingerprint enrollment, smart ID card generation, automated attendance recording, IoT-based gate access management, leave request handling, fee record integration, role-based user access, and real-time notification delivery. The backlog is updated continuously to reflect changing priorities, new requirements, or feedback from stakeholders, ensuring that the development remains aligned with institutional needs and overall project objectives.

1.6.5 Incremental Delivery

Incremental delivery is a key principle of Agile and Scrum. The SmartID Ecosystem is developed in stages where each sprint delivers a working module that can be tested and evaluated independently before being integrated into the broader system. For example, the first sprint may focus on user registration, role assignment, and student profile management, while subsequent sprints implement biometric enrollment, attendance automation, IoT gate control, and administrative dashboard development. Incremental delivery ensures stakeholder feedback and a secure, functional campus system.

1.6.6 Rationale Behind Selected Methodology

The Agile methodology and Scrum SDLC were chosen for the SmartID Ecosystem because of their adaptability to complex, multi-tier systems that involve both software development and hardware integration. Campus management applications require frequent iterations to accommodate feedback from diverse stakeholders including students, faculty, and administrators, as well as adjustments arising from IoT device testing and biometric module configuration. Agile provides the necessary flexibility to implement these changes efficiently, while Scrum ensures structured planning, task allocation, and progress tracking. This combination allows for continuous improvement, reduces the risk of late-stage failures, and maintains high-quality development standards throughout the project lifecycle.

1.6.7 Development Tools and Environment

The SmartID Ecosystem is developed using ReactJS for the frontend web-based administrative panel, with the backend implemented in Python using the Flask framework, exposed through secure RESTful APIs. For biometric intelligence, the system employs FaceNet and MTCNN for facial detection and embedding generation, while TensorFlow and Keras support the predictive analytics module. Data is stored in a relational database (SQLite during development, MySQL intended for production), ensuring structured, auditable management of user, attendance, and biometric records. Hardware components include fingerprint sensors, RFID readers, and Raspberry Pi/Arduino-based IoT gate access devices that communicate with the backend through secure APIs to control smart locks in real time. These tools support modular development, scalability, and secure handling of sensitive data. The development environment also includes version control, integrated testing frameworks, and API testing tools that facilitate Agile, sprint-based workflows.

1.6.8 Testing and Quality Assurance

Testing is integrated into every phase of the project to ensure system reliability, security, and functionality. Unit tests, integration tests, and end-to-end testing are performed iteratively during each sprint. Special attention is given to critical campus management functionalities such as biometric verification accuracy, real-time attendance synchronization, IoT gate response reliability, role-based access enforcement, and data integrity across all modules. Quality assurance practices in the SmartID Ecosystem also include code reviews, peer testing, hardware-software integration testing, and validation against both functional and non-functional requirements to ensure that the system meets institutional standards and user expectations.

1.6.9 Advantages of the Selected Methodology and SDLC

The Agile-Scrum approach provides flexibility, adaptability, and continuous improvement, which are crucial for a project as technically diverse as the SmartID Ecosystem. The methodology ensures early detection of errors across both software and hardware components, incremental delivery of functional modules, and ongoing feedback from institutional stakeholders. It promotes strong collaboration between the development team and campus management experts, enhances overall system usability, and ensures that every feature delivered is aligned with the practical needs of the educational environment. The iterative nature of Scrum also allows the team to manage the complexity of integrating biometric devices and IoT infrastructure alongside software development, reducing overall project risk and ensuring a well-tested, production-ready final system.

Chapter 2

PROBLEM DEFINITION

2 Problem Definition

This chapter discusses the precise problem to be solved by the SmartID Ecosystem project. The main issue is the lack of a unified digital platform for managing campus operations in educational institutions. Currently, student attendance is recorded through manual registers or isolated software tools, physical access to restricted areas is poorly monitored, and student identification relies on traditional ID cards that are vulnerable to loss and forgery. This fragmented system creates administrative inefficiencies, increases operational burdens, compromises institutional security, and hinders the ability of management to make informed, data-driven decisions.

Educational institutions face numerous challenges that affect students, faculty, and administrative staff alike. The absence of a centralized digital system results in inconsistent attendance records, unauthorized access to restricted campus areas, inefficient identity verification, and poor communication between departments. Many institutions continue to rely on manual roll calls, paper-based registers, and standalone access control devices that do not communicate with one another, often leading to inaccurate records, proxy attendance, and difficulty in maintaining a comprehensive view of campus activity. These challenges are particularly prominent in large universities where hundreds of students and staff members must be tracked across multiple buildings, departments, and time slots simultaneously.

In addition, the growing enrollment rates and increasing complexity of campus operations have intensified the need for an organized system that can handle student and staff data effectively. Students frequently lose their physical ID cards, and replacements are issued without proper digital tracking. Faculty members struggle to maintain accurate attendance sheets manually, which consumes valuable teaching time and introduces the risk of human error. Administrative staff lack real-time visibility into campus activity, making it difficult to enforce institutional policies, monitor gate access, or generate reliable reports for decision-making purposes.

Furthermore, the absence of biometric verification in most campus environments leaves institutions vulnerable to proxy attendance, where one student marks attendance on behalf of another, undermining academic integrity. There is also no secure mechanism for controlling and logging physical access to sensitive areas such as laboratories, server rooms, faculty offices, and examination halls. The lack of a centralized system prevents administrators from analyzing attendance trends, monitoring access patterns, or implementing proactive campus security measures in a timely and efficient manner.

Moreover, the lack of integration among existing systems often results in duplicated records, delayed information processing, and reduced operational transparency. Stakeholders are unable to access accurate, real-time data when required, affecting overall institutional efficiency.

2.1 Problem Statement

Educational institutions currently face significant challenges due to fragmented attendance management, manual record-keeping, unreliable physical access control, and the absence of a unified campus management platform. Students exploit the lack of biometric verification to engage in proxy attendance, while faculty members spend considerable time managing paper-based registers that are prone to errors and inconsistencies. Administrators lack a centralized dashboard to monitor campus activity, enforce access policies, and generate accurate reports in real time. Physical ID cards are easily lost, damaged, or misused, and there is no digital mechanism to track their usage or instantly deactivate them when required.

The SmartID Ecosystem addresses these issues by providing a centralized, biometric-driven, and IoT-integrated campus management platform. It enables secure student and staff identification through multi-modal biometric authentication, automates attendance recording, controls physical access to campus areas through IoT-enabled gates, and integrates all campus data into a single unified system accessible to administrators, faculty, and students. By bringing together identity management, attendance automation, access control, and administrative reporting into one cohesive ecosystem, the SmartID Ecosystem aims to improve institutional efficiency, eliminate proxy attendance, enhance campus security, and deliver a reliable and modern campus management experience.

2.2 Deliverables and Development Requirements

The SmartID Ecosystem is designed to create a comprehensive digital campus management platform. This section outlines the main deliverables of the project and the technical and functional requirements necessary for its successful development and deployment. Deliverables define the expected outputs of the project, while development requirements specify the tools, technologies, and system capabilities needed to achieve these outcomes.

2.2.1 Deliverables

The primary deliverables of the SmartID Ecosystem project include:

2.2.1.1 Web-Based Administrative Panel

A responsive web application designed for institutional administrators to manage the entire campus ecosystem. The panel provides functionalities including student and staff profile management, attendance monitoring, access log review, leave request handling, report generation, and system configuration. The dashboard offers real-time visibility into campus activity, enabling administrators to make informed decisions efficiently.

2.2.1.2 Mobile Application for Students and Faculty

A cross-platform mobile application that allows students to view their attendance records, check access history, submit leave requests, and receive real-time notifications regarding campus activity. Faculty members can use the application to monitor class attendance, review student profiles, approve leave requests, and access departmental reports on the go, ensuring continuous engagement with campus operations without dependency on desktop interfaces.

2.2.1.3 Biometric Enrollment and Authentication Module

A hardware-integrated biometric module that supports fingerprint-based student and staff enrollment. This module is responsible for capturing and securely storing biometric data, which is then used for identity verification during attendance marking and gate access events. The module ensures that authentication is accurate, tamper-resistant, and fast enough to handle high volumes of users during peak campus hours.

2.2.1.4 IoT-Based Gate Access Control System

An IoT-integrated physical access control system that manages entry and exit at campus gates and restricted areas such as laboratories, server rooms, and examination halls. The system grants or denies access based on verified biometric identity and predefined role-based permissions. All access events are logged in real time and made available to administrators through the web panel for review and auditing purposes.

2.2.1.5 Smart ID Card Generation Module

A digital smart ID card generation system that produces uniquely identifiable cards for students and staff. Each card is linked to the individual's biometric profile and campus role, enabling it to serve as a secure identification token across different campus touchpoints. The module supports instant deactivation of lost or expired cards and maintains a complete issuance history for administrative reference.

2.2.1.6 Centralized Database

A secure and scalable database to store all campus-related data including student and staff profiles, biometric records, attendance logs, access events, leave requests, and generated reports. The database supports efficient retrieval, role-based access control, and data integrity mechanisms to ensure that sensitive institutional information remains protected and consistently available across all system components.

2.2.2 Development Requirements

The development requirements define the technical needs for building the SmartID Ecosystem. These requirements ensure that the system is reliable, scalable, secure, and capable of supporting the integration of biometric authentication, attendance management, and access control

functionalities. The selected technologies must facilitate seamless communication between hardware and software components while providing an efficient user experience for students, faculty members, and administrators. The development environment should also support future enhancements, system maintenance, and integration with additional institutional services as organizational needs evolve.

2.2.2.1 Technical Requirements

The technical requirements define the tools and technologies needed for the SmartID Ecosystem. React is used for the responsive web administrative panel, Flutter enables cross-platform mobile application development, and Node.js with Express handles backend operations and API management. MongoDB provides scalable and secure data storage, while biometric sensors and IoT devices are integrated through secure hardware APIs.

- Frontend Development: React for a modular and responsive web-based administrative panel.
- Backend Development: Node.js with Express for server-side operations, RESTful API development, and business logic handling.
- Database: MongoDB for secure and scalable storage of student profiles, attendance records, biometric data, and access logs.
- Biometric Integration: Fingerprint sensor modules interfaced with the backend through secure APIs for enrollment and real-time authentication.
- IoT Integration: IoT-enabled gate controllers communicating with the central server to enforce access permissions and log entry and exit events.
- Cloud Hosting: Deployment on cloud infrastructure to ensure system accessibility, scalability, and reliable uptime across the institution.

2.3 Current System

Campus management in most educational institutions remains largely dependent on traditional, manual, and fragmented processes that limit efficiency, accuracy, and security. Although a few advanced private universities have begun adopting isolated digital tools for attendance or access control, these solutions remain disconnected and inconsistent across the sector. There is currently no unified platform that combines student identification, biometric attendance, IoT-based access control, and administrative reporting into a single integrated campus management system.

Due to the absence of such a centralized solution, students, faculty, and administrators face daily challenges in maintaining accurate records, verifying identities, and securing restricted campus areas. Most institutions continue to rely on manual roll calls, paper attendance sheets, and physical ID cards, which are not only time-consuming but also prone to manipulation, loss, and data inconsistency. Furthermore, digital systems that do exist in some institutions are restricted to a single function such as library management or exam scheduling and do not communicate with other administrative modules, resulting in data silos across the campus ecosystem.

2.3.1 Manual Attendance Recording

A significant portion of educational institutions, including schools, colleges, and universities, continue to rely heavily on paper-based attendance recording. Faculty members call out student names at the beginning of each class and manually mark presence or absence in printed registers. At the end of each month, this data must be compiled and submitted to the administrative office, a process that is both labour-intensive and highly susceptible to human error.

This manual system creates several operational and academic challenges. Paper registers are vulnerable to damage, misplacement, and unauthorized alteration. In many cases, students engage in proxy attendance, where a classmate marks attendance on their behalf, directly undermining academic integrity and institutional policies. Faculty members also spend several minutes of each class period on attendance, reducing the time available for actual instruction. The lack of a real-time digital record means that administrators cannot monitor attendance trends or intervene promptly when a student's attendance falls below the required threshold.

2.3.2 Inadequate Physical Access Control

Access control across most campus environments is either entirely absent or managed through basic security personnel stationed at entry points. There is no biometric or digital verification mechanism to confirm the identity of individuals entering or exiting the campus, laboratory facilities, server rooms, or examination halls. Physical ID cards, where they exist, are checked manually and can be easily borrowed, forged, or presented by unauthorized individuals.

This inadequate approach creates serious security vulnerabilities. Unauthorized individuals may enter restricted areas without detection, sensitive equipment and data may be exposed to theft or tampering, and the institution has no reliable log of who accessed which area and at what time. During examinations, the lack of verified identity checks increases the risk of impersonation. The absence of automated access logging also means that administrators cannot conduct post-incident investigations effectively, leaving the institution with no accountability trail for physical security breaches.

2.3.3 Fragmented Student Identity Management

Student identification in most institutions is managed through physical ID cards that are printed once during enrollment and rarely updated. These cards carry basic information such as the student's name, roll number, and department, but are not linked to any digital profile or verification system. When a card is lost or damaged, a replacement is issued manually with minimal tracking, creating opportunities for misuse of duplicate cards.

There is no centralized system to manage the complete lifecycle of a student's identity within the

institution. Profile updates, departmental transfers, graduation status changes, and card deactivations are handled through separate administrative processes that are often poorly coordinated. This fragmentation means that outdated or invalid ID cards may remain in circulation long after a student has left the institution, posing ongoing security risks and administrative complications.

2.3.4 Lack of Real-Time Administrative Visibility

In the existing campus environment, administrators have no real-time visibility into ongoing campus activity. Attendance data is available only after manual registers are collected and compiled, a process that may take days. Access logs, where they exist at all, are maintained on paper and stored locally at individual checkpoints without any central aggregation. This means that by the time administrators are aware of a problem, such as consistently poor attendance in a particular course or unauthorized access to a restricted area, significant time has already passed and the opportunity for timely intervention has been lost.

The absence of a centralized dashboard also makes report generation an extremely time-consuming task. Administrative staff must manually collect data from multiple sources, consolidate it into spreadsheets, and produce reports that are often outdated by the time they are reviewed. This inefficiency not only increases the administrative workload but also reduces the quality and reliability of institutional decision-making.

2.3.5 Security and Privacy Concerns

Current campus management practices do not adequately safeguard sensitive student and institutional data. Attendance registers, student profiles, and access records are stored in physical files or unprotected local computers that are accessible to a wide range of personnel without proper authorization controls. These systems are vulnerable to unauthorized modification, accidental loss, or deliberate tampering, with little to no mechanism for detecting or auditing such events.

Privacy concerns are particularly significant because student records contain personal, academic, and behavioral information that must be protected in accordance with institutional policies and ethical obligations. The lack of role-based access controls means that individuals with no legitimate need to view certain records may still be able to access them. Weak data security practices not only violate student privacy but also reduce institutional trust and expose the administration to legal and reputational risks.

2.3.6 Poor Integration Between Administrative Modules

Although some institutions have introduced isolated digital tools for specific functions such as library management, fee collection, or examination scheduling, these systems operate

independently and do not share data with one another. There is no unified platform interconnecting attendance, access control, identity management, and administrative reporting, resulting in a fragmented digital ecosystem where the same student data must be entered and maintained separately in multiple systems.

This lack of integration prevents the smooth flow of information across departments, leading to duplicated data entry, inconsistent records, and increased chances of error. For example, a student's enrollment status updated in the admissions system may not automatically reflect in the attendance or access control module, causing outdated permissions to remain active. Overall, the poor integration of existing digital tools significantly limits the potential of technology to enhance campus operations and highlights the urgent need for a centralized and interconnected campus management solution.

2.3.7 Summary of Limitations

In summary, the current campus management environment in educational institutions is hindered by multiple structural and operational limitations that collectively compromise efficiency, security, and institutional effectiveness. The heavy dependence on manual processes leads to inaccurate attendance records, proxy attendance, fragmented student profiles, and the absence of any reliable access control mechanism. Faculty members lose valuable teaching time to administrative tasks, while administrators lack the real-time data needed to monitor campus activity and make informed decisions.

These issues are amplified by the growing size of student populations and the increasing complexity of institutional operations, which make manual and disconnected approaches even less sustainable. Collectively, these limitations demonstrate the urgent need for a secure, centralized, biometric-driven, and IoT-integrated campus management platform. A system like the SmartID Ecosystem would address these challenges by automating attendance through biometric verification, controlling physical access through IoT-enabled gates, centralizing all campus data into a unified platform, and providing real-time visibility to administrators, faculty, and students through a dedicated web interface.

2.4 Summary

The second chapter of this report presents a detailed exploration of the challenges within the current campus management environment and establishes the necessity for a unified digital solution such as the SmartID Ecosystem. The chapter begins by defining the core problem: educational institutions lack a centralized, biometric-driven campus management platform capable of integrating student identity management, automated attendance recording, IoT-based access control, and administrative reporting into a single cohesive system. This fragmented

environment creates systemic inefficiencies that negatively impact students, faculty, and administrators alike.

The chapter highlights how the absence of digital integration results in proxy attendance, unauthorized campus access, inconsistent student records, poor real-time visibility, and an excessive administrative workload. Most institutions continue to rely on manual roll calls, paper registers, physical ID cards, and isolated digital tools that operate independently and fail to share data across departments. As a result, student information becomes difficult to track, attendance data is unreliable, and campus security remains vulnerable to exploitation.

These limitations are further intensified by growing student populations, increasing campus complexity, and the rising institutional demand for accurate, timely, and accessible management data. The chapter concludes that these challenges, manual processes, fragmented systems, inadequate access control, weak security practices, and poor administrative visibility, collectively demonstrate the urgent need for a centralized, integrated campus management system. The SmartID Ecosystem is presented as a comprehensive solution designed to digitize and unify campus operations through biometric authentication, IoT integration, and a centralized data platform. By offering real-time attendance automation, secure access control, unified student profiles, and reliable web-based accessibility, the SmartID Ecosystem aims to enhance institutional efficiency, eliminate proxy attendance, strengthen campus security, and ensure a modern and reliable campus management experience for all stakeholders.

Chapter 3

REQUIREMENT ANALYSIS

3 Requirement Analysis

The Requirement Analysis chapter provides a comprehensive and structured understanding of what the proposed system must accomplish to function effectively within the intended campus management environment. It serves as a foundation for translating user expectations into clear technical requirements by identifying system behavior, user interactions, and the specific tasks the system must perform. This chapter outlines how the SmartID Ecosystem is expected to operate, detailing both functional and non-functional requirements while defining the roles and responsibilities of each actor involved, such as students, parents, professors, and administrators. Using the use case diagram as a reference, this chapter organizes the requirements into well-defined sections that guide the design, development, and validation processes. Through this analysis, the development team gains a precise understanding of the system's objectives, constraints, and performance expectations, ensuring that the final implementation aligns with the needs of all stakeholders and effectively addresses the limitations of the current campus management environment.

3.1 Introduction to Requirement Analysis

Requirement Analysis is a critical phase in the software development lifecycle, serving as the foundation upon which the entire system is designed, implemented, and evaluated. For the SmartID Ecosystem, this phase ensures that all functional expectations, performance needs, and operational constraints are clearly understood before development begins. The purpose of conducting requirement analysis is to bridge the gap between user needs and technical implementation, ensuring the final system effectively resolves existing campus management challenges such as proxy attendance, unauthorized access, fragmented student records, and inefficient administrative workflows. This chapter outlines the detailed requirements of the SmartID Ecosystem platform, describing how each system component interacts with users, how data flows across modules, and what outcomes are expected from every interaction. The analysis draws directly from the Software Requirements Specification and use case definitions to ensure complete alignment between documented expectations and the system being developed.

3.2 System Overview

The SmartID Ecosystem is designed as an integrated digital platform that connects students, parents, professors, and administrators within a unified campus management environment. The system enables students to view their attendance records, check access history, submit leave requests, and receive real-time notifications regarding their academic standing. Professors can

monitor class-level attendance, review individual student profiles, approve or reject leave requests, and generate attendance reports for their respective courses. Parents are provided with controlled visibility into their child's attendance and campus activity, ensuring they remain informed without requiring direct communication with administrative staff. Administrators oversee the entire platform, managing user accounts, configuring biometric and IoT access permissions, monitoring real-time campus activity, and generating institutional reports.

The use case diagram illustrates the relationships between these actors and the system, showing how functionalities such as biometric enrollment, automated attendance recording, IoT gate access control, smart ID card management, leave handling, and report generation are interconnected. This overview reflects the system's role as a centralized hub that replaces manual and fragmented campus processes with a streamlined, biometric-driven, and IoT-integrated solution accessible from both web and mobile interfaces. By consolidating all these processes, the SmartID Ecosystem addresses the key issues of attendance manipulation, poor access security, data fragmentation, and lack of real-time administrative visibility.

3.3 Actors in the System

The SmartID Ecosystem interacts with multiple categories of users, each performing distinct roles that contribute to the overall functionality of the platform. These users, also known as actors, engage with the system through specific use cases that reflect their responsibilities, permissions, and operational needs. Understanding these actors and their interactions is crucial for defining accurate system requirements and ensuring that the platform delivers a seamless, secure, and role-appropriate experience. The following sections describe each actor in detail, outlining their characteristics, goals, and the actions they perform within the SmartID Ecosystem.

3.3.1 Administrator

The administrator is the most privileged actor within the SmartID Ecosystem and is responsible for overseeing the entire platform. Their responsibilities include managing student and staff accounts, configuring biometric enrollment settings, assigning role-based access permissions for IoT-controlled gates, monitoring real-time campus activity through the administrative dashboard, and ensuring that all data stored within the system remains accurate, consistent, and secure. Administrators also handle smart ID card generation and deactivation, review system-wide attendance and access logs, approve or escalate leave requests where required, and generate institutional reports for academic and operational decision-making. Their control panel serves as the central command interface of the SmartID Ecosystem, providing complete visibility and control over all system components.

3.3.2 Professor

Professors are key academic actors who interact with the SmartID Ecosystem primarily through the web interface to manage attendance and student engagement within their courses. The system automatically records attendance through biometric verification, eliminating the need for manual roll calls and allowing professors to focus entirely on instruction. Professors can view real-time and historical attendance data for their classes, review individual student attendance percentages, approve or reject leave requests submitted by students, and generate course-level attendance reports for submission to the academic office. The system ensures that professors have organized, easily accessible, and reliable attendance data at all times, reducing their administrative workload and improving the accuracy of academic records.

3.3.3 Student

The student is the primary end user of the SmartID Ecosystem and the central subject of the platform's core functions. Students interact with the system through biometric authentication points at campus entry gates and classroom terminals, where their attendance is automatically recorded upon successful identity verification. Through the web interface, students can view their attendance history, monitor their attendance percentage against the institutional threshold, submit leave requests with supporting justification, track the status of submitted requests, and receive real-time notifications regarding attendance warnings or access events.

3.3.4 Parent

The parent is a read-only stakeholder in the SmartID Ecosystem with secure access to their child's campus activity through a web portal. Parents can view attendance records, monitor gate entry and exit logs, and receive notifications when attendance falls below institutional requirements. This feature enhances transparency, promotes accountability, and strengthens communication between the institution and the student's family.

3.4 Use Cases Diagram

Figure 3.1 presents the SmartID Ecosystem's use case diagram with four main actors, Student, Admin, Faculty, and Parent, supported by external systems including RFID Reader, Biometric Device, IoT Access Unit, and Notification System. It maps key interactions, authentication flows, and access control relationships, ensuring the design aligns with secure, biometric-driven campus management objectives.

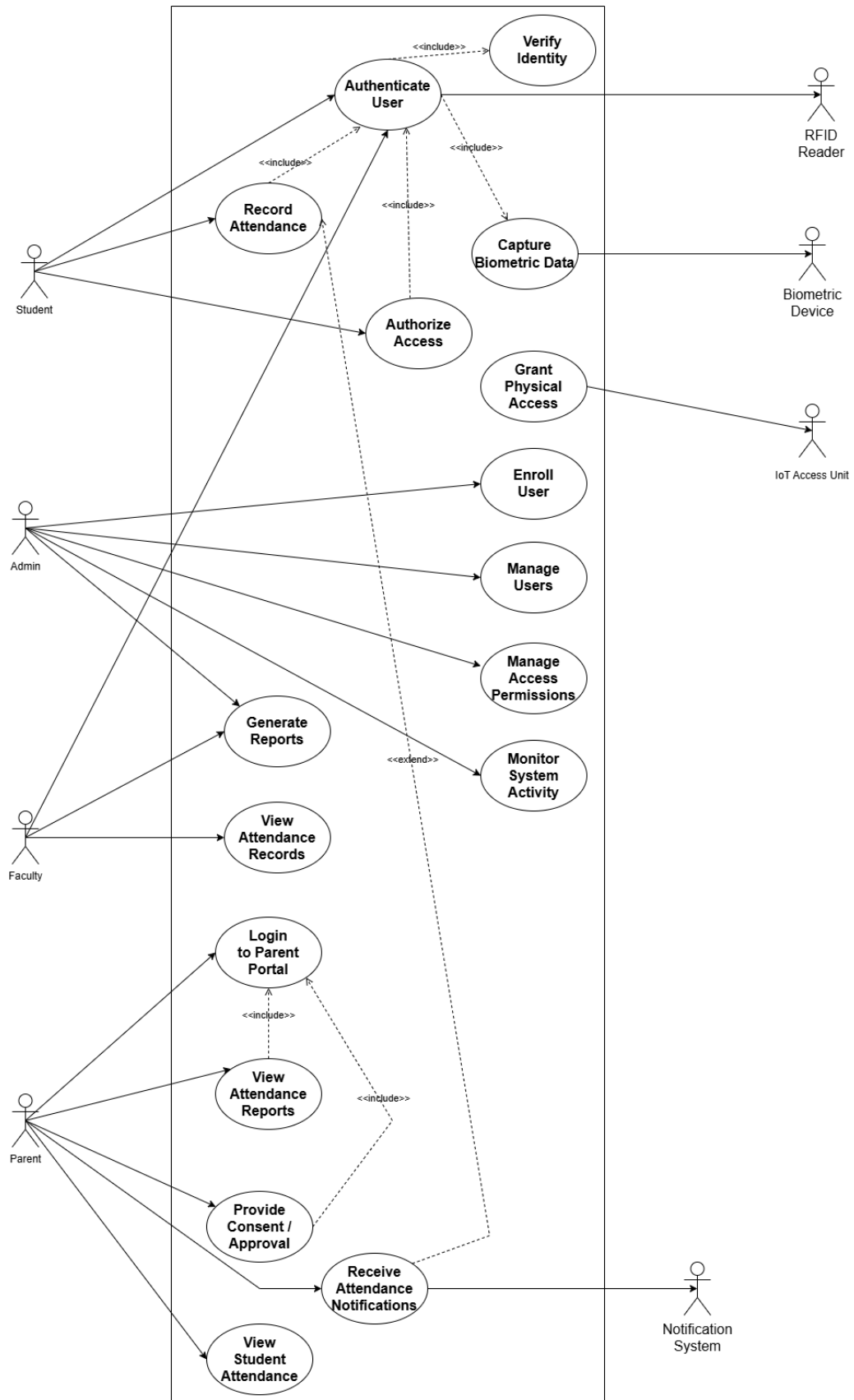


Figure 3.1: Use Case Diagram

3.5 Use Case Descriptions

This section provides detailed descriptions of the SmartID Ecosystem use cases, including the actors involved, triggers, preconditions, postconditions, flows, exceptions, and business rules. The use cases cover key functions such as biometric authentication, attendance management, IoT-based access control, user management, report generation, parent portal access, and real-time notifications. Each use case is presented using a standardized format to ensure clarity, consistency, and support for system design and development.

3.5.1 Authenticate User

This use case defines how students, faculty, and administrators are verified before accessing the SmartID Ecosystem or campus entry points. It includes two sub-processes, Verify Identity and Capture Biometric Data, to ensure secure and accurate authentication. The user must be enrolled with valid biometric data before authentication. The system captures and matches biometric credentials against stored records, verifies identity, and grants access upon successful validation. Alternative flows handle failed biometric matches, while exceptions include device or network failures. Authentication is mandatory before any system access or physical entry is permitted.

Table 3.1: Authenticate user use case

Use Case Attribute	Details
Use Case Name	Authenticate User
Primary Actors	Student, Admin, Faculty
Secondary Actors	Biometric Device, RFID Reader
Description	The system verifies the identity of users through biometric data capture and identity verification before granting system or physical access.
Trigger	A user attempts to log in or access a campus entry point.
Pre-Conditions	User must be enrolled in the system. Valid biometric data must exist in the database. Biometric device must be operational.
Post-Conditions	User identity is successfully verified. Access is granted or denied based on verification result.
Normal Flow	User presents biometric credentials. Biometric device captures data. System matches captured data against stored records. Identity is verified. Access is granted.
Alternative Flow	Biometric match fails → System prompts re-attempt. RFID card presented as fallback → System processes RFID-based verification.
Exceptions	Biometric device malfunction. Network unavailability. Database unreachable.
Business Rules	Authentication must be completed before any system or physical access is granted. Failed attempts beyond the defined limit must trigger a security alert.

3.5.2 Record Attendance

The "Record Attendance" use case defines how student attendance is automatically captured and stored within the SmartID Ecosystem. As shown in Table 3.2, this use case is triggered whenever a student successfully authenticates at a biometric terminal. The pre-condition requires that the student is enrolled, and their class schedule exists in the system. In the normal flow, the student authenticates, the system identifies the corresponding class session based on the time and location, and attendance is automatically recorded. Alternative flows handle cases where the student arrives outside the scheduled time window, while exceptions include device failure or session data unavailability. Business rules enforce that attendance can only be recorded within the defined session time window to prevent manipulation.

Table 3.2: Record Attendance Use Case

Use Case Attribute	Details
Use Case Name	Record Attendance
Primary Actors	Student
Secondary Actors	Biometric Device
Description	The system automatically records student attendance upon successful biometric authentication at a campus terminal.
Trigger	A student successfully authenticates at a biometric terminal.
Pre-Conditions	Student must be enrolled and authenticated. • A valid class session must be scheduled for that time and location.
Post-Conditions	Attendance is recorded and saved in the centralized database. • Attendance record is made available to faculty and admin.
Normal Flow	Student presents biometric credentials. System authenticates the student. System identifies the active class session. Attendance is marked and stored. 5. Confirmation is sent to the student via notification.
Alternative Flow	Student arrives outside the time window → Attendance marked as late or absent based on institutional policy. • Manual override requested by faculty → Admin reviews and updates record.
Exceptions	Biometric device failure. No active session found for the time slot. • Database write error.
Business Rules	Attendance can only be recorded within the defined session time window. • Manual attendance modification must be authorized by an administrator.

3.5.3 Authorize Access

This use case determines whether an authenticated user is permitted to enter a specific campus area and is triggered after successful authentication. The system verifies role-based access permissions for the requested area and, if authorized, sends an approval signal to the IoT Access Unit to grant entry. Alternative flows handle unauthorized access attempts, while exceptions include IoT connectivity failures. Access permissions are strictly role-based and can only be modified by an administrator.

Table 3.3: Authorize Access Use case

Use Case Attribute	Details
Use Case Name	Authorize Access
Primary Actors	Student, Admin, Faculty
Secondary Actors	IoT Access Unit
Description	The system evaluates the authenticated user's role-based permissions and determines whether physical access to a requested campus area should be granted.
Trigger	A user attempts to enter a campus area after successful authentication.
Pre-Conditions	User must be successfully authenticated. • Access permissions for the area must be configured in the system.
Post-Conditions	Access is either granted or denied. • Access event is logged with timestamp and user details.
Normal Flow	User is authenticated. System retrieves user's role and permissions. System checks permissions against the requested area. Authorization signal is sent to the IoT Access Unit. 5. Access is granted or denied accordingly.
Alternative Flow	User lacks required permissions → Access denied and alert generated. • Permission profile not found → System defaults to deny and logs the event.
Exceptions	IoT Access Unit connectivity failure. • Permission data unavailable due to database error.
Business Rules	Access permissions must be role-based and configured by an administrator. • All access attempts, whether successful or denied, must be logged.

3.5.4 Grant Physical Access

The "Grant Physical Access" use case defines the hardware-level execution of campus gate and door control within the SmartID Ecosystem. As shown in Table 3.4, this use case is triggered by an authorization signal received from the Authorize Access use case and is executed through the IoT Access Unit. The pre-condition requires that authorization has been confirmed by the system. In the normal flow, the IoT Access Unit receives the signal, unlocks the gate or door, allows the user to pass, and then re-locks after a defined interval. Alternative flows handle cases where the IoT device does not respond to the signal, while exceptions include power failure or hardware malfunction. Business rules enforce that gates must re-lock automatically after each access event and that all events are logged in real time.

Table 3.4: Physical Access Use case

Use Case Attribute	Details
Use Case Name	Grant Physical Access
Primary Actors	IoT Access Unit
Secondary Actors	Admin
Description	The IoT Access Unit executes the physical unlocking of a gate or door upon receiving an authorization signal from the system.
Trigger	An authorization signal is received from the Authorize Access use case.
Pre-Conditions	Authorization must be confirmed by the system. • IoT Access Unit must be operational and connected.
Post-Conditions	Gate or door is unlocked and user passes through.

	Gate re-locks automatically after the defined interval. • Event is logged with timestamp.
Normal Flow	System sends authorization signal to IoT Access Unit. IoT Access Unit unlocks the gate. User passes through. Gate re-locks after defined interval. 5. Access event is logged.
Alternative Flow	IoT device does not respond → System retries signal and generates alert if unresolved.
Exceptions	Power failure at the access point. Hardware malfunction of IoT Access Unit. • Signal transmission failure.
Business Rules	Gates must re-lock automatically after every access event. • All physical access events must be logged in real time with user identity and timestamp.

3.5.5 Enroll User

This use case defines how new students and staff are registered in the SmartID Ecosystem with their personal information, role details, biometric data, and system credentials. It is triggered when a new user joins the institution and requires verified institutional details before enrollment. The administrator enters user information, captures biometric data, and stores the profile in the centralized database. Alternative flows address incomplete data or biometric capture failures, while exceptions include device unavailability or duplicate records. Enrollment must be completed before a user can authenticate or access any campus service.

Table 3.5: Enrollment use case

Use Case Attribute	Details
Use Case Name	Enroll User
Primary Actors	Admin
Secondary Actors	Biometric Device
Description	The administrator registers new students or staff into the system by capturing their biometric data and creating their campus profile.
Trigger	A new user joins the institution and requires system registration.
Pre-Conditions	User's institutional details must be verified. • Biometric device must be operational.
Post-Conditions	User profile is created and stored in the database. Biometric data is linked to the user profile. • User is ready for authentication.
Normal Flow	Admin initiates enrollment. User details are entered into the system. Biometric data is captured via biometric device. System links biometric data to user profile. 5. Profile is saved and enrollment is confirmed.
Alternative Flow	Biometric capture fails → System prompts re-capture. • Duplicate record detected → System alerts admin and halts enrollment.
Exceptions	Biometric device unavailable. Database write failure. • Incomplete user information provided.
Business Rules	Enrollment must be completed before a user can authenticate or access any campus service. • Each user must have a unique biometric profile in the system.

3.5.6 Manage Users

The "Manage Users" use case allows administrators to add, update, suspend, or remove user accounts within the SmartID Ecosystem. As shown in Table 3.6, this use case is triggered when a user's status or profile requires modification. The pre-condition requires that the user record exists in the system. In the normal flow, the administrator logs in, navigates to the user management section, locates the relevant user profile, and performs the required action such as updating details, changing roles, or deactivating the account. Alternative flows handle unauthorized modification attempts, while exceptions include server errors or connectivity issues. Business rules enforce that only administrators with the appropriate permissions are authorized to modify user accounts.

Table 3.6: User Management Use case

Use Case Attribute	Details
Use Case Name	Manage Users
Primary Actors	Admin
Secondary Actors	Database
Description	Administrators can add, update, suspend, or remove user accounts within the SmartID Ecosystem.
Trigger	A user's status or profile requires modification.
Pre-Conditions	Admin must be logged in with appropriate permissions. • User record must exist in the system.
Post-Conditions	User account is updated, suspended, or removed as required. • Changes are reflected immediately across the system.
Normal Flow	Admin logs in. Navigates to user management. Locates the relevant user profile. Performs the required modification. 5. Changes are saved and confirmed.
Alternative Flow	• Unauthorized admin attempts modification → System denies action and logs attempt.
Exceptions	Server error during update. No internet connectivity. • User record not found.
Business Rules	Only admins with appropriate permissions can modify user accounts. • All account modifications must be logged for auditing purposes.

3.5.7 Manage Access Permissions

The "Manage Access Permissions" use case allows administrators to configure and update role-based access permissions for campus areas within the SmartID Ecosystem. As shown in Table 3.7, this use case is triggered whenever access rules for a specific area or user role need to be created or modified. The pre-condition requires that the campus areas and user roles are already defined in the system. In the normal flow, the administrator logs in, navigates to the access permissions panel, selects the relevant area or role, and configures the permissions accordingly. Alternative flows handle conflicting permission rules, while exceptions include database errors or connectivity failures.

Table 3.7: Permission access Management use case

Use Case Attribute	Details
Use Case Name	Manage Access Permissions
Primary Actors	Admin
Secondary Actors	IoT Access Unit, Database
Description	Administrators configure and update role-based access permissions that determine which users can access specific campus areas.
Trigger	Access rules for a campus area or user role need to be created or modified.
Pre-Conditions	Admin must be logged in. • Campus areas and user roles must be defined in the system.
Post-Conditions	Access permissions are updated and applied immediately. • IoT Access Units reflect the updated permissions.
Normal Flow	Admin logs in. Navigates to access permissions panel. Selects the relevant area or user role. Configures permissions. 5. Changes are saved and applied to IoT Access Units.
Alternative Flow	Conflicting permission rules detected → System alerts admin and requests resolution before saving.
Exceptions	Database error during save. • IoT device not responding to updated permissions.
Business Rules	Permission changes must take effect immediately upon saving. • All permission modifications must be logged with admin identity and timestamp.

3.5.8 Generate Reports

The "Generate Reports" use case allows administrators and faculty to produce attendance and access reports within the SmartID Ecosystem. As shown in Table 3.8, this use case is triggered whenever a stakeholder requires a summary of attendance or access data for a specific period, course, or user group. The pre-condition requires that relevant data exists in the system. In the normal flow, the user logs in, navigates to the reports section, applies the desired filters, and generates the report, which can be viewed on screen or exported. Alternative flows handle cases where no data is available for the selected filters, while exceptions include report generation errors or server timeouts. Business rules enforce that reports are generated based on verified data only and that access to reports is restricted by user role.

Table 3.8: Report Generation use case

Use Case Attribute	Details
Use Case Name	Generate Reports
Primary Actors	Admin, Faculty
Secondary Actors	Database
Description	Administrators and faculty can generate attendance and access reports filtered by date, course, or user group.
Trigger	A stakeholder requires a summary of attendance or access data.
Pre-Conditions	User must be logged in with report access permissions. • Relevant data must exist in the system.
Post-Conditions	Report is generated and displayed or exported successfully.
Normal Flow	User logs in. Navigates to reports section.

	Applies desired filters. System generates report. 5. Report is displayed or exported.
Alternative Flow	No data available for selected filters → System displays empty report with notification.
Exceptions	Report generation error. • Server timeout during data retrieval.
Business Rules	Reports must be generated from verified data only. • Report access is restricted based on user role.

3.5.9 Monitor System Activity

This use case enables administrators to track real-time and historical events within the SmartID Ecosystem. It is triggered by system events or administrator requests and is conditionally activated for events such as failed authentication attempts or unauthorized access. Administrators can access the activity panel to review logs and alerts, provided that system logs are available and up to date. Alternative flows handle situations with no recent activity, while exceptions include missing or corrupted log data. All system events are automatically logged and retained according to institutional policies.

Table 3.9: Monitoring Activities use case

Use Case Attribute	Details
Use Case Name	Monitor System Activity
Primary Actors	Admin
Secondary Actors	Database
Description	Administrators can monitor real-time and historical system events including authentication attempts, access events, and system alerts.
Trigger	A system event occurs or an admin proactively reviews platform activity.
Pre-Conditions	Admin must be logged in. • System logs must be available and up to date.
Post-Conditions	System activity is reviewed and any alerts are addressed.
Normal Flow	Admin logs in. Navigates to system activity panel. Reviews event logs and alerts. 4. Takes action on flagged events if required.
Alternative Flow	No recent activity logged → System displays empty log with last update timestamp.
Exceptions	Corrupt or missing log data. • System monitoring service unavailable.
Business Rules	All system events must be logged automatically without manual intervention. • Logs must be retained for a defined institutional period and protected from unauthorized modification.

3.5.10 View Attendance Records

The "View Attendance Records" use case allows faculty members to access attendance data for their respective courses within the SmartID Ecosystem. As shown in Table 3.10, this use case is triggered whenever a faculty member needs to review class attendance. The pre-condition requires that attendance data exists for the relevant course and time period. In the normal flow, the faculty member logs in, navigates to the attendance section, selects the course and date range,

and views the attendance records. Alternative flows handle cases where no attendance data is available for the selected parameters, while exceptions include data retrieval errors or connectivity issues. Business rules enforce that faculty members can only view attendance records for courses assigned to them.

Table 3.10: Attendance record use case

Use Case Attribute	Details
Use Case Name	View Attendance Records
Primary Actors	Faculty
Secondary Actors	Database
Description	Faculty members can view attendance records for their assigned courses, filtered by date and student.
Trigger	A faculty member needs to review class attendance.
Pre-Conditions	Faculty member must be logged in. • Attendance data must exist for the selected course and period.
Post-Conditions	Attendance records are displayed successfully.
Normal Flow	Faculty logs in. Navigates to attendance section. Selects course and date range. 4. System retrieves and displays attendance records.
Alternative Flow	No attendance data available for selected parameters → System displays notification.
Exceptions	Data retrieval error. • No internet connectivity.
Business Rules	Faculty can only view attendance records for courses assigned to them. • Attendance data displayed must reflect verified biometric records only.

3.5.11 Login to Parent Portal

This use case allows parents to access the SmartID Ecosystem to monitor their child's campus activity and includes viewing attendance reports after successful login. It requires a registered parent account linked to a verified student profile. The parent enters their credentials, is authenticated, and is directed to the dashboard. Alternative flows handle incorrect credentials, while exceptions include server unavailability or account deactivation. Access is permitted only to accounts linked to at least one verified student profile.

Table 3.11: Parental Login use case

Use Case Attribute	Details
Use Case Name	Login to Parent Portal
Primary Actors	Parent
Secondary Actors	Authentication Service, Database
Description	Parents log into the dedicated parent portal to access their child's attendance and campus activity information.
Trigger	A parent attempts to access the parent portal.
Pre-Conditions	Parent must have a registered account. • Account must be linked to a verified student profile.
Post-Conditions	Parent is authenticated and redirected to the parent dashboard.
Normal Flow	Parent navigates to parent portal. Enters login credentials. System authenticates the parent.

	4. Parent is redirected to dashboard with linked student data.
Alternative Flow	• Incorrect credentials entered → System displays error and prompts retry. • Account locked after multiple failed attempts → Parent directed to reset process.
Exceptions	Server unavailability. • Account deactivated by administrator.
Business Rules	• Each parent account must be linked to at least one verified student profile. • Multiple failed login attempts must trigger a temporary account lock.

3.5.12 View Attendance Reports

The "View Attendance Reports" use case allows parents to access detailed attendance summaries for their child within the SmartID Ecosystem. As shown in Table 3.12, this use case is included within the Login to Parent Portal use case and is also connected to the Receive Attendance Notifications use case. The pre-condition requires that the parent is logged in and that attendance data exists for the linked student. In the normal flow, the parent navigates to the attendance reports section, selects the relevant time, and views the attendance summary. Alternative flows handle cases where no attendance data is available, while exceptions include data retrieval errors. Business rules enforce that parents can only access data for their own linked child and that the data displayed reflects verified records.

Table 3.12: Attendance Reports use case

Use Case Attribute	Details
Use Case Name	View Attendance Reports
Primary Actors	Parent
Secondary Actors	Database
Description	Parents can view detailed attendance reports for their linked child, including dates, sessions, and attendance percentages.
Trigger	Parent accesses the attendance reports section after logging in.
Pre-Conditions	Parent must be logged in. • Attendance data must exist for the linked student.
Post-Conditions	• Attendance report is displayed successfully.
Normal Flow	Parent logs in. Navigates to attendance reports section. Selects time or course filter. 4. System retrieves and displays attendance data.
Alternative Flow	No attendance data available → System displays notification with last updated timestamp.
Exceptions	Data retrieval error. • No internet connectivity.
Business Rules	Parents can only view attendance data for their own linked child. • Data must reflect verified biometric attendance records only.

3.5.13 Provide Consent

The "Provide Consent" use case allows parents to authorize specific institutional actions related to their child within the SmartID Ecosystem. As shown in Table 3.13, this use case is triggered whenever the system or institution requires parental acknowledgment or approval for an action such as leave requests or policy acknowledgments. The pre-condition requires that the parent is

logged in and that a pending consent or approval request exists. In the normal flow, the parent receives a notification, logs into the portal, reviews the request details, and submits their approval or rejection. Alternative flows handle cases where the parent does not respond within the defined timeframe, while exceptions include notification delivery failures. Business rules enforce that consent submissions are timestamped and permanently logged for institutional records.

Table 3.13: Consent use case

Use Case Attribute	Details
Use Case Name	Provide Consent / Approval
Primary Actors	Parent
Secondary Actors	Notification System, Database
Description	Parents review and respond to institutional consent or approval requests related to their child's campus activities.
Trigger	A consent or approval request is raised by the institution or system.
Pre-Conditions	Parent must be logged in. • A pending consent or approval request must exist.
Post-Conditions	Consent or approval is submitted and logged. • Relevant department or system is notified of the parent's response.
Normal Flow	Parent receives notification of pending request. Logs into parent portal. Reviews request details. Submits approval or rejection. 5. System logs the response and notifies the institution.
Alternative Flow	Parent does not respond within defined timeframe → System escalates to admin for follow-up.
Exceptions	Notification delivery failure. • Parent portal unavailable.
Business Rules	All consent submissions must be timestamped and permanently logged. • Expired requests must be escalated automatically to the administrator.

3.5.14 Receive Attendance Notifications

The "Receive Attendance Notifications" use case defines how parents and students are automatically alerted about attendance-related events within the SmartID Ecosystem. As shown in Table 3.14, this use case is triggered by the system whenever a significant attendance event occurs, such as an absence, a late arrival, or attendance falling below the institutional threshold. It is connected to the View Attendance Reports use case through an include relationship. The pre-condition requires that the user has an active account with notifications enabled. In the normal flow, the system detects the attendance event, generates the notification, and delivers it to the relevant parent or student through the Notification System. Alternative flows handle cases where notification delivery fails, while exceptions include Notification System downtime. Business rules enforce that notifications are delivered promptly and that all notification events are logged.

Table 3.14: Notification use case

Use Case Attribute	Details
Use Case Name	Receive Attendance Notifications
Primary Actors	Parent
Secondary Actors	Notification System
Description	The system automatically sends attendance-related alerts to parents and students when significant events such as absences or low attendance are detected.
Trigger	A significant attendance event is detected by the system.
Pre-Conditions	User must have an active account with notifications enabled. • Notification System must be operational.
Post-Conditions	Notification is delivered successfully to the parent or student. • Notification event is logged in the system.
Normal Flow	System detects attendance event. Notification is generated. Notification System delivers alert to parent or student. 4. Delivery is confirmed and logged.
Alternative Flow	Notification delivery fails → System retries delivery and logs failure if unresolved.
Exceptions	Notification System downtime. • Invalid or inactive contact details for the user.
Business Rules	Notifications must be delivered promptly upon detection of the triggering event. • All notification events must be logged with delivery status and timestamp.

3.5.15 View Student Attendance

The "View Student Attendance" use case allows parents to access a real-time summary of their child's individual attendance status within the SmartID Ecosystem. As shown in Table 3.15, this use case is triggered whenever a parent wishes to check their child's current attendance standing. The pre-condition requires that the parent is logged in and that attendance records exist for the linked student. In the normal flow, the parent navigates to the student attendance section, and the system displays the current attendance percentage, session-by-session breakdown, and any attendance warnings. Alternative flows handle cases where no records are available, while exceptions include data retrieval errors. Business rules enforce that the data displayed is always sourced from verified biometric records and is updated in real time.

Table 3.15: Attendance parental view use case

Use Case Attribute	Details
Use Case Name	View Student Attendance
Primary Actors	Parent
Secondary Actors	Database
Description	Parents can view a real-time summary of their child's attendance status including overall percentage and session-level breakdown.
Trigger	A parent accesses the student attendance section of the parent portal.
Pre-Conditions	Parent must be logged in. • Attendance records must exist for the linked student.
Post-Conditions	Student attendance summary is displayed successfully.
Normal Flow	Parent logs in.

	Navigates to student attendance section. System retrieves and displays attendance summary. 4. Parent reviews attendance details.
Alternative Flow	No attendance records found → System displays notification with enrollment confirmation details.
Exceptions	Data retrieval error. • No internet connectivity.
Business Rules	Attendance data must always be sourced from verified biometric records. • Data must be updated in real time to reflect the most recent attendance events.

3.6 Functional Requirements

Functional requirements define the specific behaviors and operations that the SmartID Ecosystem must provide to satisfy the needs of its users. These requirements outline the essential capabilities that the system must possess to manage student identity verification, automated attendance recording, IoT-based access control, administrative oversight, and parent portal functionality effectively. The SmartID Ecosystem is designed to offer a secure, reliable, and user-friendly environment that ensures data integrity, institutional security, and efficient campus management across all stakeholder groups.

3.6.1 User Management

The SmartID Ecosystem allows students, faculty, parents, and administrators to access the platform using verified credentials assigned during the enrollment process. Login is secured through role-based authentication that directs each user to their designated interface upon successful verification. Students access their attendance records and campus activity history, faculty access course-level attendance data and leave request management tools, parents access the parent portal for monitoring their child's campus presence, and administrators manage all system functions through a centralized dashboard. Security measures including encrypted credentials and session expiry policies protect sensitive information, and repeated failed login attempts trigger temporary account locks to prevent unauthorized access. [9]

User profiles are created during the enrollment process and can be updated by administrators as required. Student profiles contain personal details, biometric data references, course enrollments, and smart ID card information. Faculty profiles include departmental assignments and course allocations. The system validates all profile changes and maintains a complete modification history for auditing purposes.

Account management ensures that only active and verified users can interact with the system. Administrators review and approve new enrollments, deactivate accounts upon student departure or staff resignation, and manage role assignments across the platform. All account-level actions are logged with administrator identity and timestamp to maintain transparency and institutional accountability.

3.6.2 Biometric Enrollment and Identity Verification

The system provides a structured biometric enrollment process through which administrators register new students and staff by capturing their fingerprint data through integrated biometric devices. Each biometric profile is uniquely linked to the user's campus account and smart ID card, forming the foundation for all authentication and attendance operations across the platform. During authentication, the system captures the user's biometric input, matches it against the stored profile, and returns a verification result within the defined response time. The RFID Reader serves as a secondary verification mechanism for cases where biometric capture is not feasible. All authentication events are logged in real time with user identity, device location, and outcome, ensuring a complete and auditable record of campus identity verification activity [10]. Duplicate biometric entries are detected and rejected during enrollment to maintain the integrity of the identity database. Administrators are notified immediately of any enrollment conflicts or biometric capture failures so that corrective action can be taken without delay.

3.6.3 Attendance Management

The system automatically records student attendance upon successful biometric authentication at a campus terminal, eliminating the need for manual roll calls and reducing the opportunity for proxy attendance. The system identifies the active class session based on the time and physical location of the authentication event and associates the attendance record with the correct course and student profile [9].

Faculty members can view real-time and historical attendance records for their assigned courses through the web interface or mobile application, filtered by date, session, or individual student. The system generates automated attendance warnings when a student's cumulative attendance falls below the institutional minimum threshold and notifies both the student and their linked parent through the Notification System.

Administrators retain the authority to manually override attendance records where required, with all such modifications logged under the administrator's identity and timestamp. This ensures that manual corrections remain traceable and subject to institutional review.

3.6.4 IoT-Based Access Control

The system governs physical entry to campus areas through IoT-enabled gate units that respond to authorization signals issued by the central server. Following successful biometric authentication, the system evaluates the user's role-based access permissions and determines whether entry to the requested area is permitted. If authorization is confirmed, a signal is sent to the IoT Access Unit, which unlocks the gate and re-locks it automatically after a defined interval [8].

All access events, whether granted or denied, are logged in real time with user identity, location, timestamp, and outcome. Administrators can configure and update access permissions for specific campus areas and user roles through the administrative panel, with changes taking effect immediately across all connected IoT devices. Denied access attempts and repeated unauthorized entry efforts trigger immediate alerts to the administrator for investigation and response.

3.6.5 Smart ID Card Management

The system generates a unique smart ID card for each enrolled user upon completion of the biometric enrollment process. Each card is digitally linked to the user's biometric profile and campus role, enabling it to function as a unified identification token across all campus touchpoints. Card details including issuance date, linked profile, and current status are maintained in the centralized database.

Administrators can instantly deactivate a smart ID card in the event of loss, damage, or expiry, preventing unauthorized use of the card after deactivation. Replacement cards can be issued through the administrative panel, with the full issuance and deactivation history retained for institutional reference. The system ensures that deactivated cards are immediately rejected at all authentication and access points across the campus.

3.6.6 Leave Request Management

The system provides a structured leave request workflow that allows students to submit leave applications through the mobile application with supporting justification. Submitted requests are routed directly to the relevant faculty member for review, who can approve or reject the request through their interface. Students receive automated notifications informing them of the outcome of their submitted request.

Approved leave records are reflected in the student's attendance data to ensure that authorized absences are appropriately distinguished from unexcused ones. Parents receive notifications when a leave request is submitted or when a decision is made, ensuring that they remain informed of their child's attendance-related activities. All leave request actions including submission, approval, rejection, and notification delivery are logged for administrative reference.

3.6.7 Parent Portal

The system provides parents with a dedicated portal offering controlled read-only access to their child's attendance records, gate access history, and campus activity data. Parents must authenticate with verified credentials before accessing any data, and each parent account is restricted to information pertaining to their own linked child only [10].

Parents can view session-level attendance breakdowns, overall attendance percentages, and access logs showing campus entry and exit times. The portal also supports parental consent and

approval submission for institutional requests such as leave authorizations or policy acknowledgments. All consent submissions are timestamped and permanently logged for institutional records.

3.6.8 Notifications and Alerts

The system provides a comprehensive notification mechanism to keep all stakeholders informed of important campus events and attendance-related updates. Students receive notifications confirming attendance records, leave request outcomes, and attendance warnings. Parents receive automated alerts when their child is absent, when attendance falls below the institutional threshold, or when a leave request is submitted or decided upon [8].

Administrators receive security alerts when unauthorized access attempts are detected, when authentication failures exceed the defined threshold, or when system anomalies are identified. All notification events are logged with recipient identity, content, delivery timestamp, and delivery status. The system retries notification delivery automatically in the event of an initial failure to ensure that no critical alert goes undelivered.

3.6.9 Report Generation and Analytics

The SmartID Ecosystem provides reporting and analytics tools to support informed decision-making by administrators and faculty. Faculty members can generate course-level attendance reports filtered by date range and student, while administrators can produce system-wide attendance and access control reports covering all users, departments, and campus areas. Generated reports can be viewed on screen or exported in a standard file format for institutional use.

Administrators can also monitor access trends, identify students with consistently low attendance, and review system activity logs through the administrative dashboard. All reporting features rely on verified biometric data to ensure accuracy and institutional reliability. Automated report generation and scheduled alerts further support proactive campus management and administrative oversight [11].

3.6.10 System and Admin Management

Administrators have comprehensive tools to manage users, monitor system activity, configure access permissions, and enforce security policies across the SmartID Ecosystem. They can enroll, update, suspend, or remove user accounts, ensuring that only active and authorized individuals interact with the platform. All administrative actions are logged for auditing and accountability purposes.

The administrative dashboard provides real-time visibility into campus activity including live authentication events, access logs, attendance summaries, and system health indicators. Role-

based access control is strictly enforced across all modules, ensuring that each user can only access the data and functions permitted by their assigned role. Administrators can review system logs to detect anomalies, investigate security incidents, and take corrective action as required.

3.7 Non-Functional Requirements

Non-functional requirements define the quality attributes, performance standards, and operational constraints that the SmartID Ecosystem must meet beyond its core functional behavior. While functional requirements describe what the system does, non-functional requirements specify how the system performs and the standards it must uphold in terms of reliability, security, usability, and efficiency. These requirements are critical for ensuring that the SmartID Ecosystem operates effectively under real-world campus conditions, meets the expectations of all stakeholders, and maintains the integrity and security of sensitive biometric and institutional data. [11]

3.7.1 Performance

The SmartID Ecosystem must be capable of handling large numbers of simultaneous authentication requests without noticeable delays, particularly during peak campus hours when many students and staff authenticate within short time windows. Biometric matching must be completed within 2 seconds under normal operating conditions, and IoT Access Units must respond to authorization signals within 1 second to ensure smooth physical access flow. The system must support a minimum of 500 concurrent users across web and mobile interfaces without response time degradation. Database queries for attendance retrieval, access log review, and report generation must execute efficiently to maintain a consistent and responsive user experience across all interfaces [9].

3.7.2 Scalability

The system must be scalable to accommodate institutional growth, including increases in student enrollment, the addition of new campus buildings, and the integration of additional biometric terminals and IoT Access Units over time. The system architecture must support horizontal scaling so that new hardware and software components can be added without requiring significant restructuring of the existing platform. The database must handle growing volumes of attendance records, access logs, and user profiles without performance impact, ensuring that the SmartID Ecosystem remains responsive and stable as the institution expands [8].

3.7.3 Security

Security is a critical non-functional requirement for the SmartID Ecosystem due to the highly sensitive nature of biometric data, student records, and access logs maintained by the platform. All biometric data must be encrypted at rest and in transit using industry-standard encryption

protocols to protect against unauthorized access and data breaches. Role-based access control must be strictly enforced to ensure that each user can only access the data and functions permitted by their assigned role. The system must support secure authentication mechanisms, enforce automatic session expiry after a defined period of inactivity, and maintain a tamper-proof audit log of all authentication, access, and administrative events. Regular security audits and real-time monitoring must be conducted to detect and mitigate potential vulnerabilities before they can be exploited [10].

3.7.4 Reliability and Availability

The SmartID Ecosystem must provide high reliability and ensure continuous availability throughout campus operational hours, as interruptions to authentication or access control functions can directly disrupt academic activities and campus security. The system must maintain a minimum uptime of 99.5% during operational hours, with robust error-handling mechanisms in place to manage server failures, network disruptions, or hardware malfunctions. Automatic failover procedures must be implemented to maintain access control functionality even in the event of a primary server failure. Real-time system health monitoring must be active at all times to enable proactive identification and resolution of emerging issues [11].

3.7.5 Usability

The SmartID Ecosystem must offer an intuitive and accessible interface suitable for all user groups including administrators, faculty, students, and parents, regardless of their level of technical expertise. The web administrative panel must present a clearly organized dashboard that allows administrators to navigate all system modules without requiring technical training. The mobile application must comply with standard mobile usability guidelines and must be compatible with both Android and iOS platforms. The parent portal must present attendance and access information in a simple and readable format designed for non-technical users. Clear error messages, contextual guidance, and consistent interface design must be implemented across all platform components to reduce user errors and enhance overall satisfaction [9].

3.7.6 Maintainability

The SmartID Ecosystem must be maintainable and adaptable for future enhancements. It should follow a modular architecture that separates frontend, backend, hardware integration, and notification components, enabling updates or modifications without impacting the entire system. Comprehensive documentation, consistent coding standards, and version control practices must be maintained to support efficient maintenance, debugging, and future development.

The system must support rolling updates that can be deployed without requiring full platform downtime, minimizing disruption to campus operations during maintenance windows [8].

3.7.7 Compliance

The system must comply with applicable data protection regulations and institutional privacy policies governing the collection, storage, and use of biometric and personal data. All biometric data handling must adhere to internationally recognized standards for biometric information management to ensure that student and staff privacy rights are protected at every stage of the data lifecycle. Access to sensitive data must be governed by documented authorization policies, and all data processing activities must be auditable to support institutional compliance reviews and regulatory inspections [10].

3.7.8 Backup and Recovery

The SmartID Ecosystem must implement automated and regular data backups to ensure that all student profiles, biometric records, attendance logs, access histories, and system configurations can be restored in the event of hardware failure, accidental deletion, or data corruption. Backups must be stored securely and independently of the primary system to prevent simultaneous loss. Recovery procedures must be tested periodically to verify their effectiveness and to ensure that the system can resume normal operations within a defined and acceptable recovery time following an unexpected failure [11].

3.7.9 Performance Monitoring and Logging

The system must continuously monitor server load, database performance, biometric device connectivity, IoT Access Unit status, and application error rates to enable proactive issue resolution before impacting campus operations. All system events including authentication attempts, access decisions, administrative actions, and notification deliveries must be automatically logged and retained for a defined institutional period. Administrator monitoring dashboards must remain consistently accessible, providing live visibility into the health and activity of the entire SmartID Ecosystem. [8].

Chapter 4

DESIGN AND ARCHITECTURE

4 Design and Architecture

The Design and Architecture chapter of the SmartID Ecosystem explains how the software and hardware are structured, how its components interact, and how data flows throughout the system. This chapter provides a detailed overview of the system's architecture, data representation, process flows, and design models. It highlights the technical decisions made to ensure performance, security, scalability, and maintainability across both the software and IoT layers. By presenting the system design clearly, this chapter demonstrates how the SmartID Ecosystem meets both functional and non-functional requirements while delivering a reliable, automated, and secure attendance and campus access management platform.

4.1 System Architecture

The system architecture defines the overall structure of the SmartID Ecosystem and illustrates how its main components work together to deliver secure identity verification, automated attendance recording, and smart campus access control efficiently. It ensures that each module performs its specific tasks while interacting seamlessly with others to maintain smooth operations and data integrity.

4.1.1 Architectural Style

The SmartID Ecosystem is developed using a modular, multi-tier client-server architecture following the Object-Oriented Design (OOD) methodology. The client layer consists of a ReactJS-based web interface that allows students, faculty, administrators, and parents to interact with the system through role-specific dashboards. The server layer, built using Python and Django, manages business logic, biometric authentication, attendance recording, IoT device communication, and database operations. This separation of concerns enhances maintainability, supports scalability as the number of enrolled users and connected devices grows, and ensures secure, organized handling of sensitive biometric and attendance data.

4.1.2 Components

The SmartID Ecosystem is composed of several interconnected components, each designed to perform a specific function within the overall architecture. These components work collaboratively to ensure smooth system operation, secure data handling, and efficient campus management. By dividing the system into modular parts, SmartID maintains clarity in functionality, supports easier maintenance, and allows individual components to evolve without affecting the entire system.

4.1.2.1 User Interface (UI)

The UI provides web-based access to all user types ,students, faculty, administrators, and parents. It is built using ReactJS and is designed to be intuitive and responsive, ensuring users can navigate features such as enrollment, attendance viewing, access logs, notification management, and consent settings with ease. Each user role is presented with a tailored dashboard that reflects their specific permissions and responsibilities. The UI communicates with the backend exclusively through secure REST API calls over HTTPS and serves as the primary point of interaction between users and the system.

4.1.2.2 API Gateway and Core Backend Module

This component serves as the central entry point for all client requests. It routes incoming API calls to the appropriate internal modules, including Enrollment, Authentication, Attendance, IoT Access Control, Notifications, Analytics, and System Settings. It also enforces authentication, role-based authorization, and input validation, ensuring that all requests are legitimate and properly handled before reaching the core business logic.

4.1.2.3 Multi-Modal Authentication Module

The authentication module manages identity verification for all users through facial recognition, fingerprint scanning, and RFID-based identification. Facial verification is performed using the FaceNet model with MTCNN for face detection, while fingerprint and RFID rely on compatible hardware drivers. The module incorporates liveness detection to prevent spoofing attempts and ensures that only verified users can trigger attendance recording or physical access. Biometric data is stored as encrypted embeddings rather than raw images, preserving user privacy at every stage.

4.1.2.4 Real-Time Attendance Logging Module

This module automatically records student and staff attendance immediately upon successful authentication. It captures the user identity, timestamp, and location for each attendance event and synchronizes the record with the central database in real time. The module supports exception handling workflows, allowing faculty to flag attendance irregularities and administrators to review and finalize changes. All updates are logged for auditing purposes, ensuring accurate and tamper-resistant records.

4.1.2.5 IoT-Based Smart Access Control Module

The IoT access control module integrates with smart locks and entry gates through Raspberry Pi controllers and GPIO-connected relay modules. After receiving an authorization decision from the backend, this module sends an unlock command to the appropriate door for a configured pulse duration, then re-locks automatically. Access decisions, entry timestamps, and outcomes are logged and linked to the user's attendance record, providing a unified view of campus activity for administrators.

4.1.2.6 Admin Dashboard and Reporting Module

This module enables administrators and faculty to view real-time attendance data, access logs, and system activity. It provides visual charts, filtering options by date, course, or student, and data export capabilities to support data-driven decisions. The dashboard retrieves aggregated data from both the attendance database and the predictive analytics module, presenting a comprehensive and up-to-date overview of campus operations.

4.1.2.7 Notification Module

The Notification Module listens for system events such as missed classes, failed authentication attempts, and unauthorized access attempts. It automatically sends alerts to relevant recipients ,students, faculty, parents, or administrators ,through email or in-application notifications. This module integrates closely with the Authentication, Attendance, and IoT Access Control modules, ensuring that stakeholders are informed about critical events in real time.

4.1.2.8 Privacy and Consent Management Module

This module ensures the ethical and secure handling of all sensitive user information. It manages digital consent forms collected during enrollment, enforces AES-256 encryption for stored biometric data, and processes user requests for consent updates or biometric data deletion. When a user revokes consent, the module automatically removes associated biometric templates and disables biometric authentication for that account, ensuring compliance with institutional data privacy policies.

4.1.2.9 Predictive Analytics Module

Using machine learning techniques built on TensorFlow and Keras, this module analyzes historical attendance and access logs to identify absenteeism patterns and unusual access behaviors. It computes risk scores for individual students, flags those with high absenteeism rates, and provides the Admin Dashboard with actionable insights. A scheduled job runs daily analytics to keep predictions current and relevant for administrative decision-making.

4.1.3 Interactions

All components communicate primarily through RESTful APIs, ensuring efficient and standardized data exchange. The UI sends requests to the server layer, which processes data, interacts with the database, and responds with relevant information. For instance, when a student authenticates at a terminal, the request is sent to the Authentication Module, which verifies the biometric input, triggers the Attendance Logging Module to record the event, and simultaneously signals the IoT Access Control Module to unlock the door. All communication is secured using HTTPS protocols and authentication tokens to protect data integrity and user privacy.

4.1.4 System Architecture Diagram

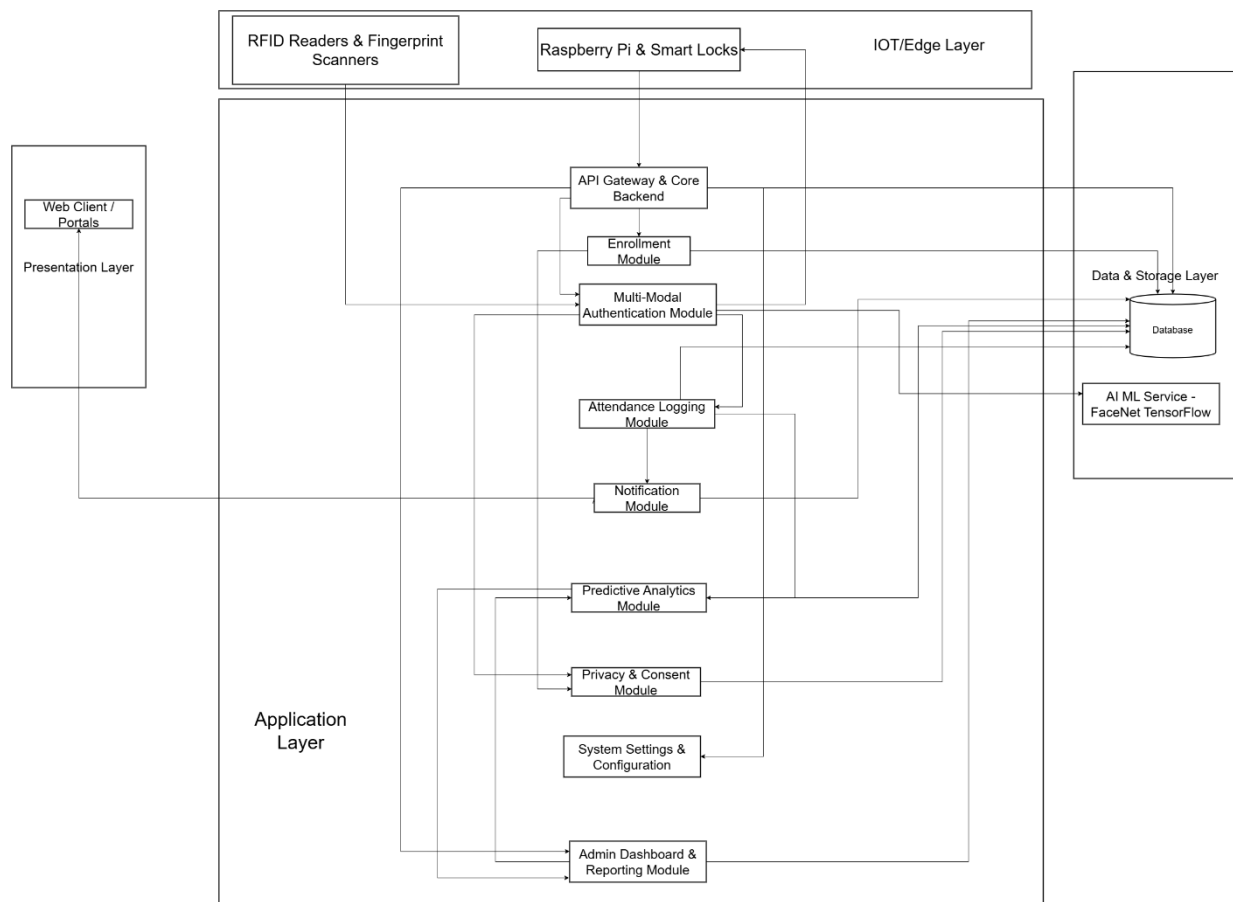


Figure 4.1: Architecture Diagram of SmartID Ecosystem

The system architecture diagram (Figure 4.1) illustrates the overall structure and interaction of the SmartID Ecosystem components. It shows how users including students, faculty, administrators, and parents interact with the system through the web interface. The diagram highlights the flow of information between the user interface, the application server, IoT devices, and the database. When a user performs an action such as enrolling, authenticating, or viewing attendance, the request is sent to the application server. The server processes the request, interacts with the database to retrieve or store data, communicates with IoT hardware where necessary, and returns the appropriate response to the user interface. This structure provides a clear separation of concerns, improves system scalability, and enhances maintainability.

4.2 Data Representation

The Data Representation section explains how information is structured, stored, and managed within the SmartID Ecosystem. It covers the database design approach, entity relationships, and the organization and security of stored data, ensuring that all system components can efficiently access and process the information they need while maintaining accuracy, consistency, and privacy.

4.2.1 Database Design

The SmartID Ecosystem uses a relational database management system (RDBMS), with SQLite used during development and MySQL intended for production deployment. This choice is driven by the structured and interrelated nature of the system's data, including user identities, attendance records, access logs, biometric templates, and course enrollments. A relational model enforces referential integrity between these entities, supports complex queries required by the reporting and analytics modules, and ensures consistent, auditable data management across all system operations.

The database is organized into logical groups: user and role data, biometric and consent records, academic structure, operational runtime records, notifications, analytics results, and system configuration. All sensitive fields, particularly within biometric tables, are encrypted at rest using AES-256 to ensure that raw identity information remains protected.

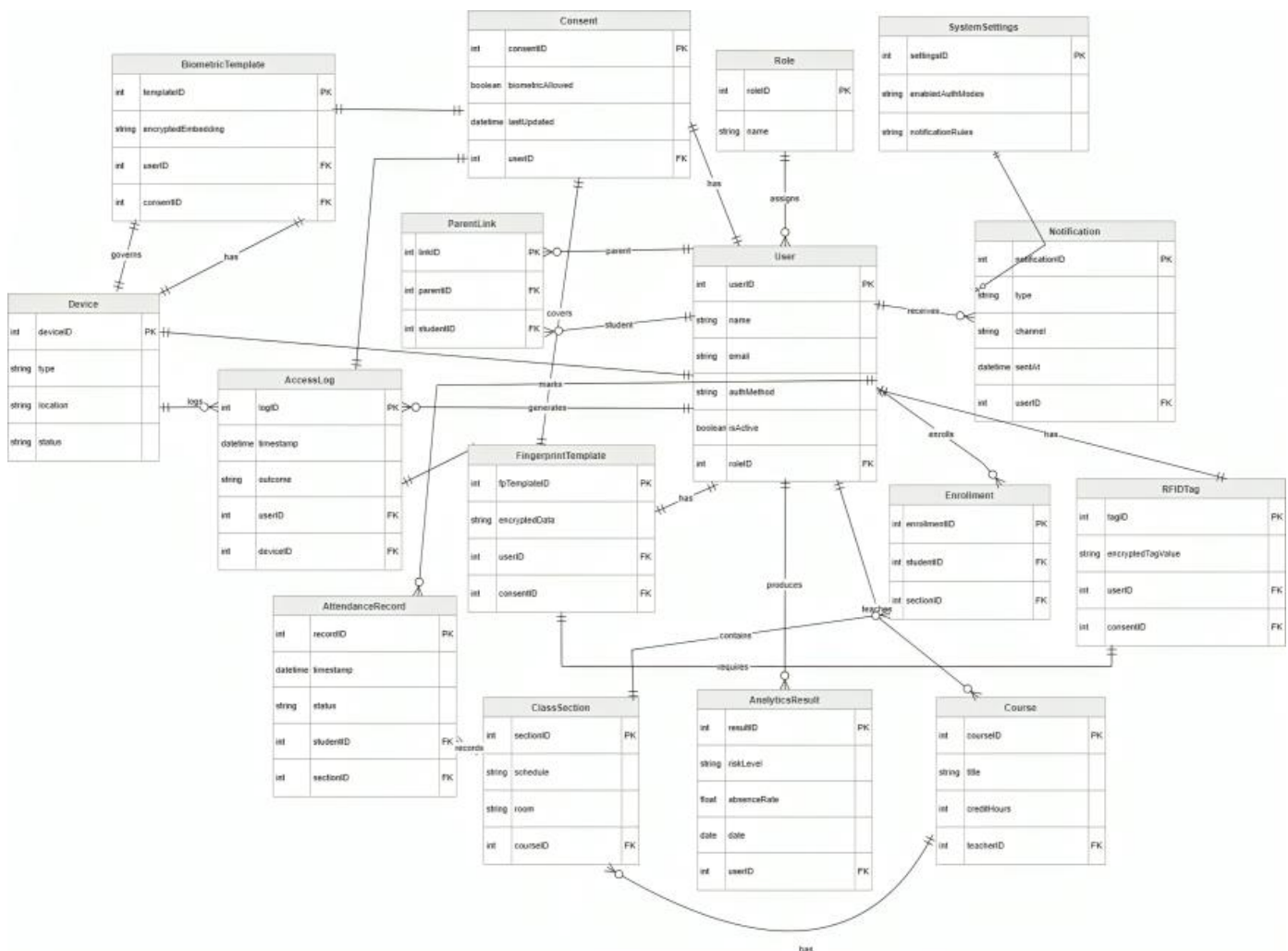


Figure 4.2: ER Diagram of SmartID Ecosystem

An ER diagram (Figure 4.2) illustrates how these entities and their relationships are organized within the SmartID Ecosystem's data model.

4.2.2 Data Flow

The data flow within the SmartID Ecosystem follows a structured and secure pathway, moving information from external actors through core processing modules and into persistent storage. Data originates from multiple sources including students, teachers, administrators, parents, and IoT hardware devices, passing through key processes such as enrollment, authentication, attendance logging, and access control before being stored in structured data stores. The following diagrams illustrate this flow at two levels of abstraction as recommended by Pressman. [12]

4.2.2.1 DFD Level 0

This Level 0 DFD represents the SmartID Ecosystem as a single, high-level process interacting with students, teachers, parents, admins, and IoT devices. The system exchanges user data, authentication requests, attendance information, alerts, reports, and access commands while utilizing core data stores such as user records, attendance logs, notification logs, and analytics data.

4.2.2.2 DFD Level 1

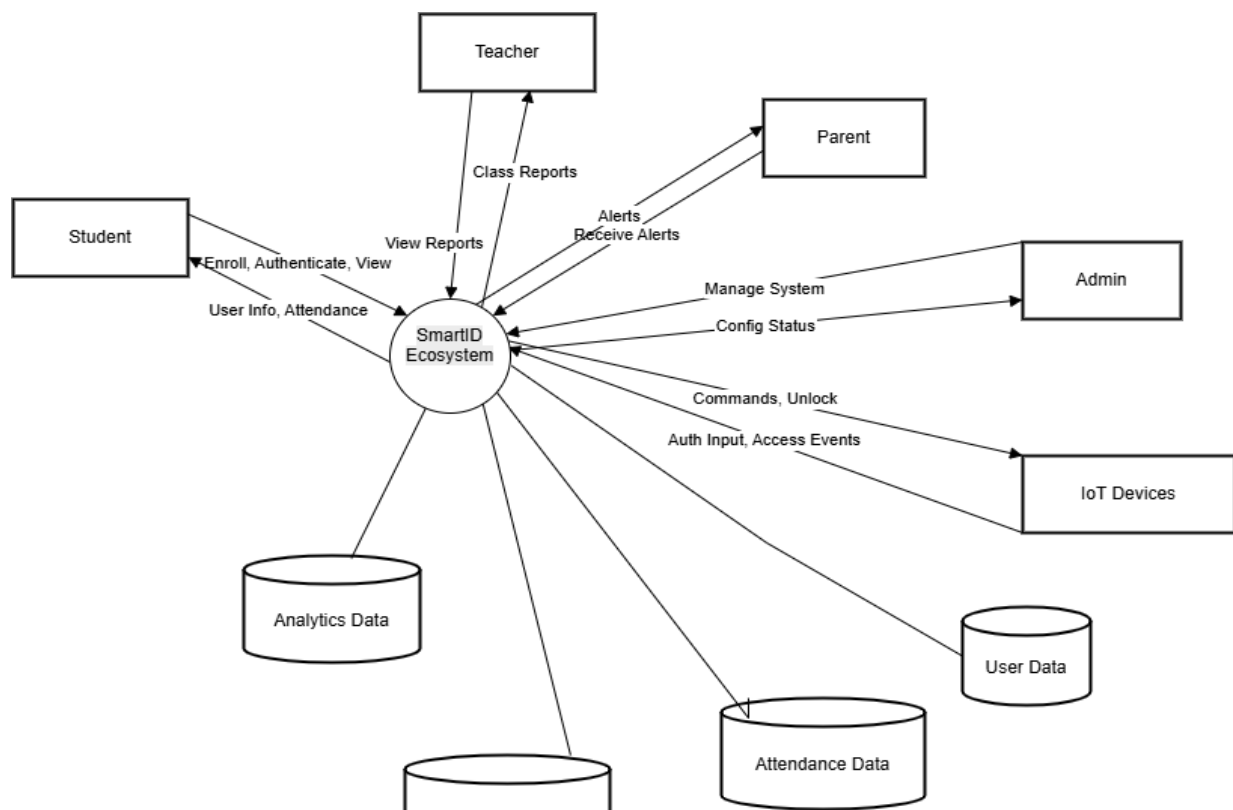


Figure 4.3: DFD Level 0

This Level-1 DFD groups the SmartID Ecosystem into four major processes: user enrollment and consent, authentication and attendance, smart access control, and monitoring and communication. It shows how students, teachers, parents, admins, and IoT devices interact with these processes, and how data flows through three main stores: user and consent data, attendance

records, and system logs/alerts. The structure is intentionally minimal to keep flows readable and easy for evaluators to understand.

4.3 Process Flow/Representation

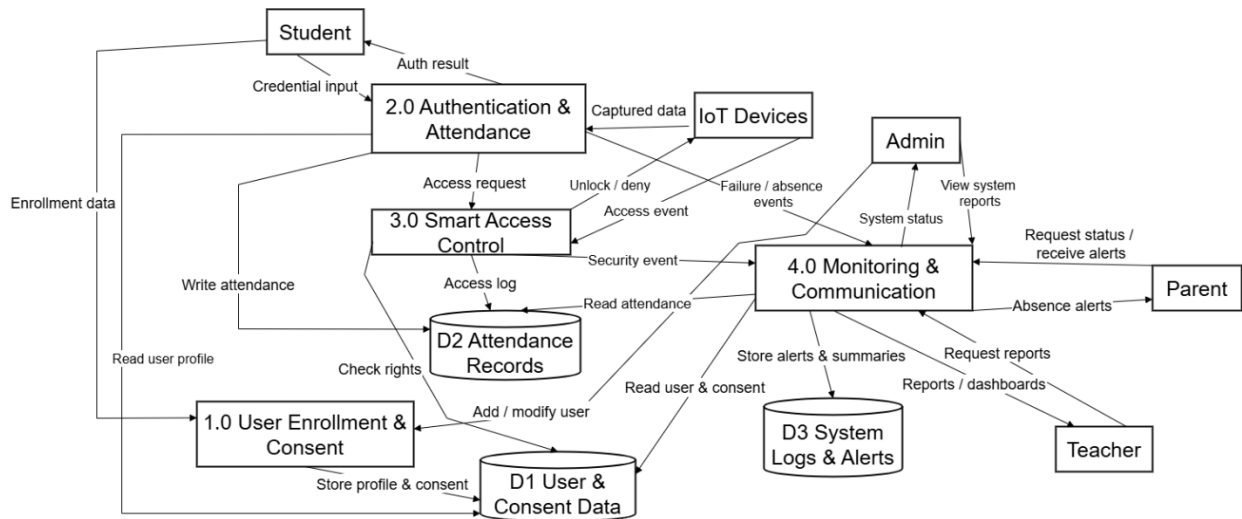


Figure 4.4: DFD Level 1

The activity flow diagram illustrates the complete operational workflow of the SmartID Ecosystem, beginning from user enrollment and progressing through daily authentication, access control, and administrative monitoring. The process starts with User Enrollment, where a new user enters personal information, selects their role (Student, Parent, or Professor), captures biometric data, and accepts a digital consent form. All data is then securely stored in the system database.

Following enrollment, the system enters the Daily Authentication cycle. Each time a user attempts to authenticate, the system evaluates whether authentication is successful. If authentication fails, the system logs the attempt, sends a failure alert, and notifies the parent. The system then checks whether the maximum number of attempts has been reached, if not, the user may retry; if yes, the session is terminated. If authentication succeeds, attendance is automatically marked and the system evaluates whether physical access should be granted. If access is permitted, the Raspberry Pi unlocks the door and the event is logged. If access is denied, an alert is generated and the denial is recorded for security purposes.

After authentication and access control, the system updates the dashboard, performs attendance analytics, and generates notifications. Parents receive attendance updates, while professors review reports and flag exceptions. The process then reaches the End Session node, completing the authentication cycle while the system remains available for new requests.

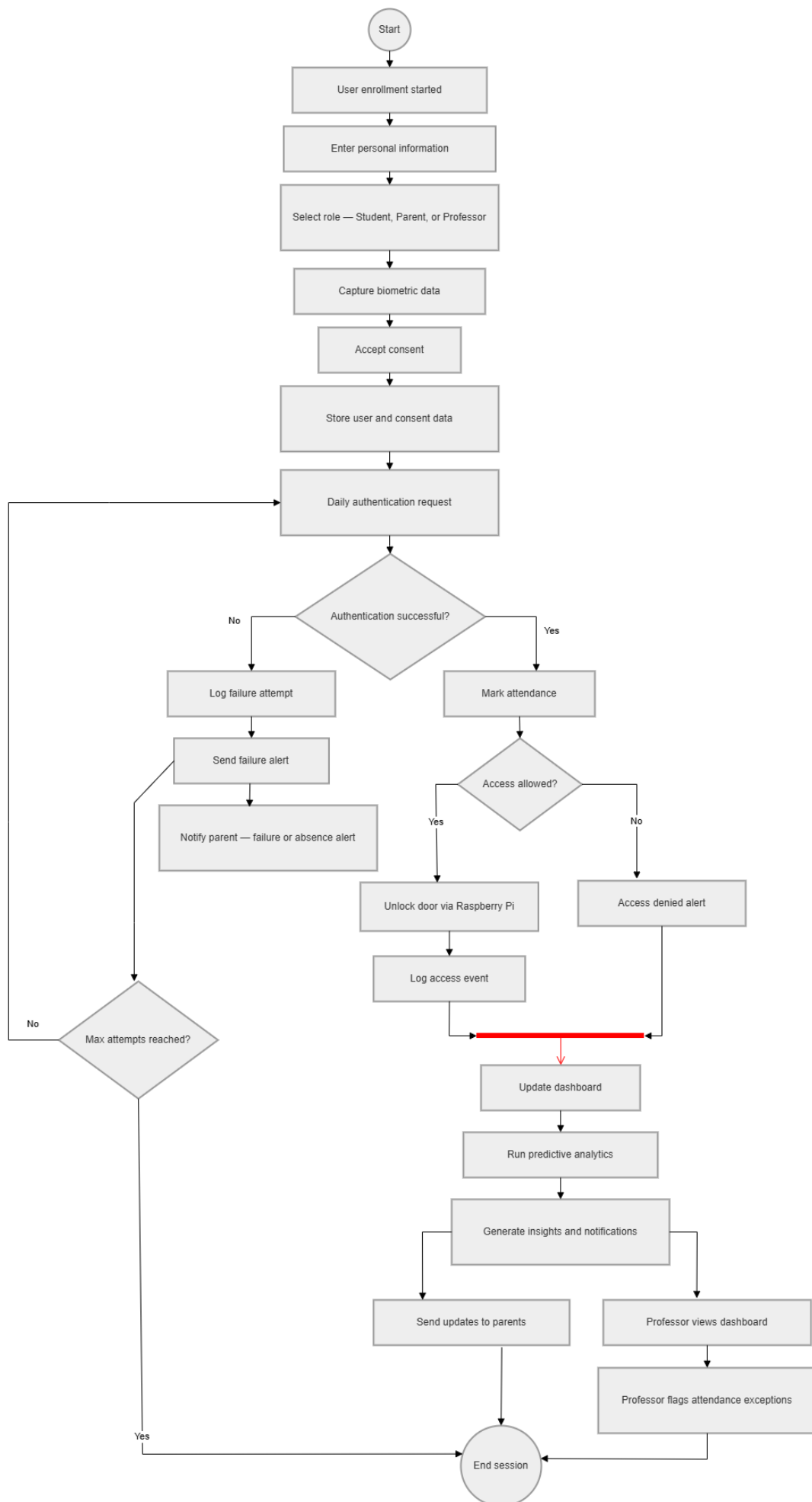


Figure 4.5: Activity Flow Diagram

4.4 Design Models

4.4.1 Class Diagram

This class diagram represents the core structure of the SmartID Ecosystem using proper UML relationships. The User class is the superclass of Student, Teacher, and Admin, shown through inheritance with hollow arrowheads. Key subsystems such as Authentication, AccessControl, Notification, Analytics, and SystemSettings are connected to User through associations, while essential parts like AttendanceRecord and Consent use composition, indicating they cannot exist without a User. Overall, the diagram clearly shows how major components collaborate to provide authentication, attendance, access control, privacy, and analytics functionality.

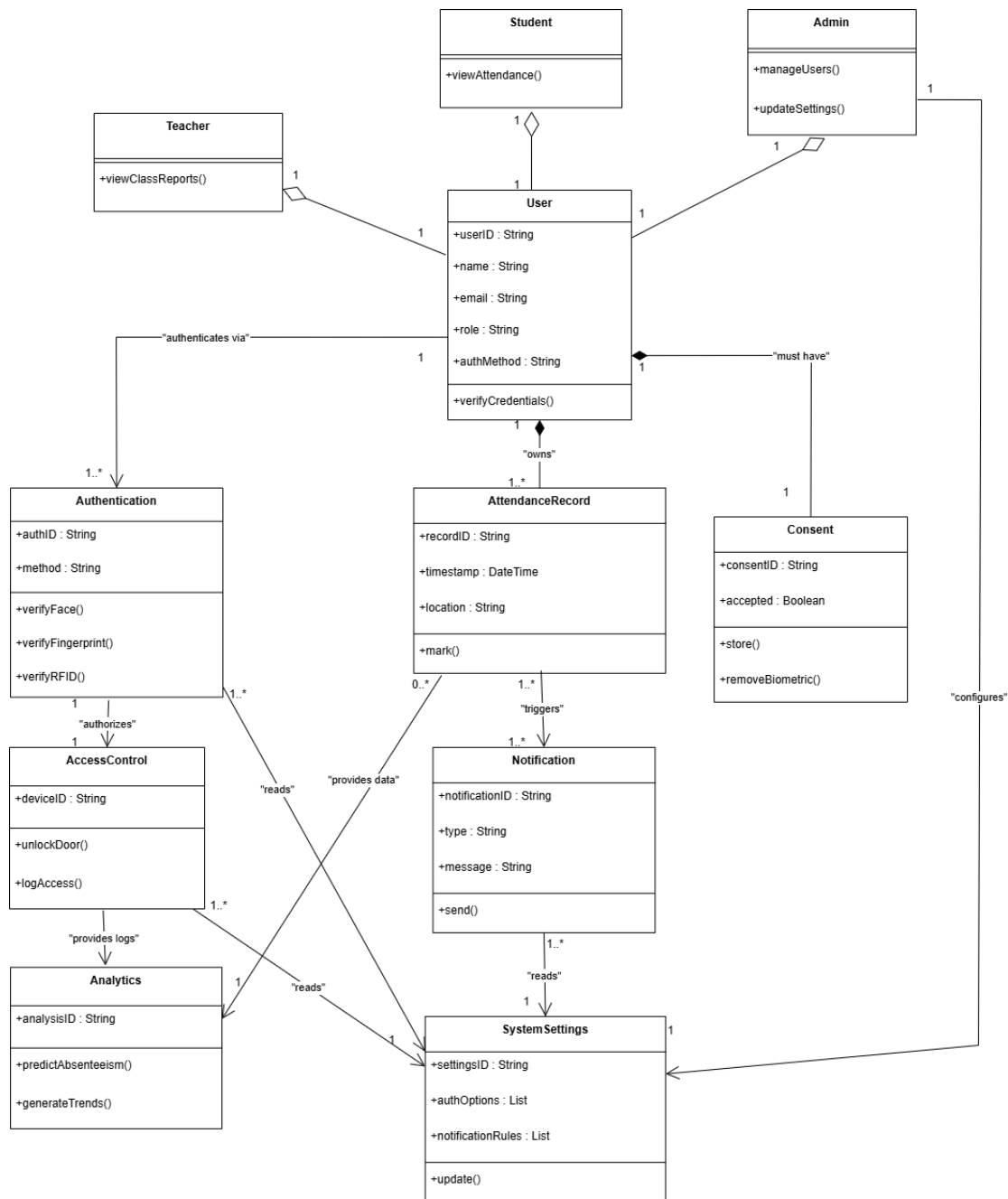


Figure 4.6: Class diagram of SmartId Ecosystem

4.4.2 Sequence Diagrams

4.4.2.1 User Enrollment Module

This sequence diagram shows how a user submits their details for enrollment, after which the system validates the input, checks consent, and stores the user profile in the database. Depending on validation results, the user either sees error messages or receives successful registration confirmation. The Enrollment Service controls and coordinates the entire process.

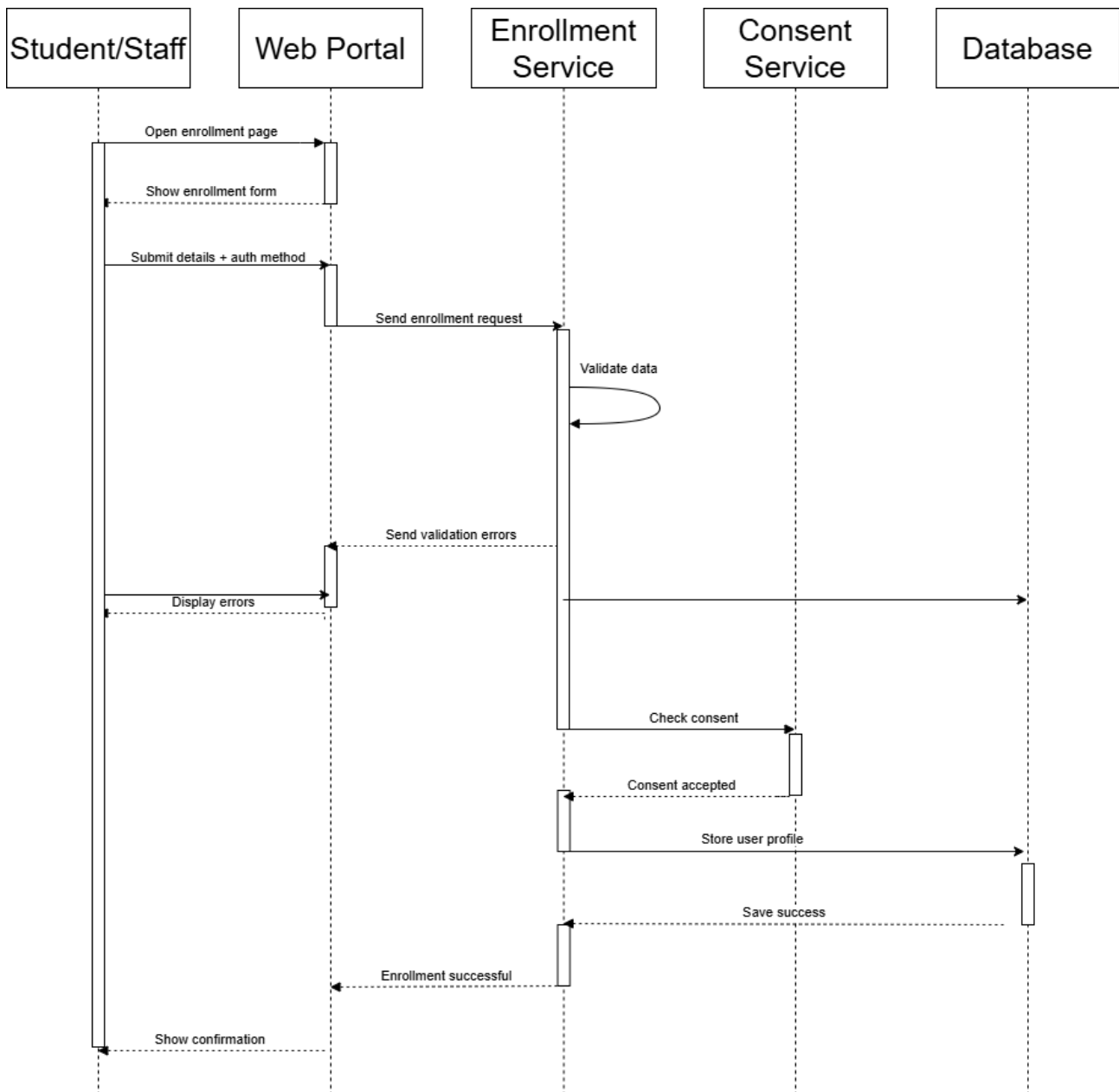


Figure 4.7: Sequence Diagram of User Enrollment Module

4.4.2.2 Multi-Modal Authentication Module

This sequence diagram models the Multi-Modal Authentication Module, where a user presents face, fingerprint, or RFID at a terminal. The Authentication Service retrieves configuration and stored user data, verifies the credential, and then informs the terminal of success or failure. On

failure, it may also call the Notification Service to log the event or send alerts to the user.

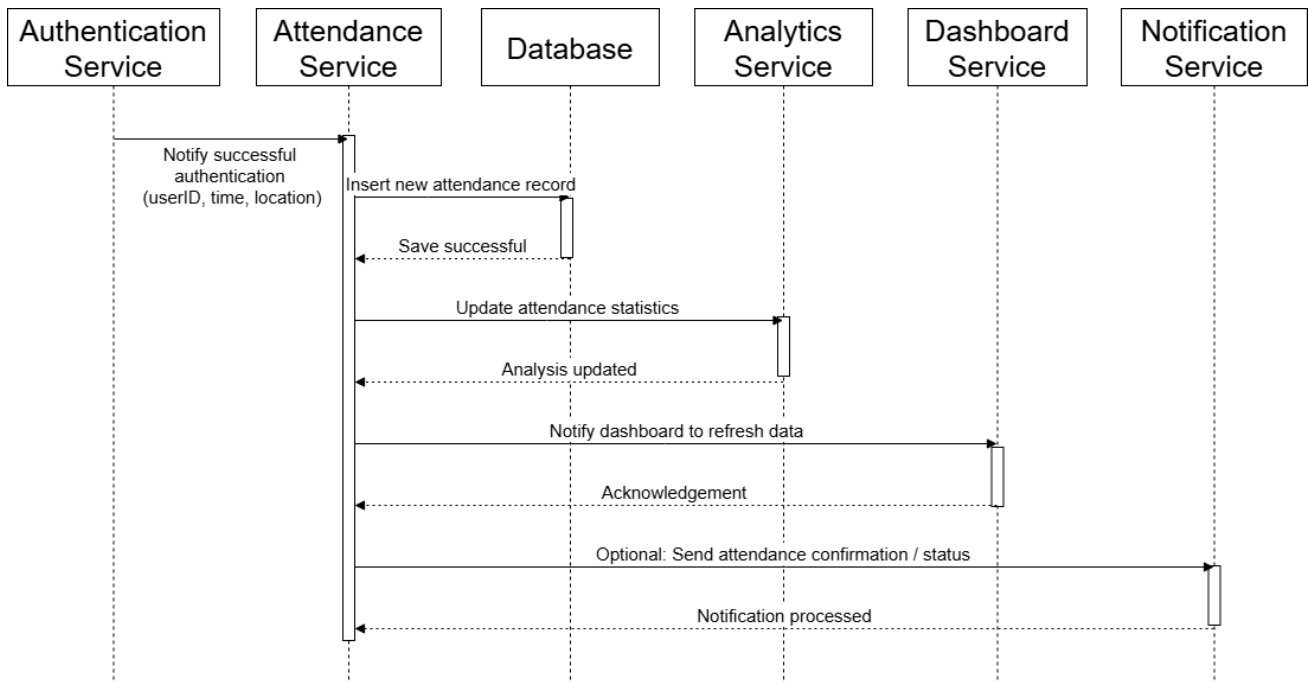


Figure 4.8: Sequence Diagram of Multi-Modal Authentication

4.4.2.3 Real-Time Attendance Logging

This sequence diagram represents the Real-Time Attendance Logging Module, which starts after the Authentication Service reports a successful login. The Attendance Service stores a new attendance record in the database, updates analytics, and informs the dashboard so that reports and charts remain up to date. Optionally, it can also trigger the Notification Service to send confirmations or alerts related to attendance status.

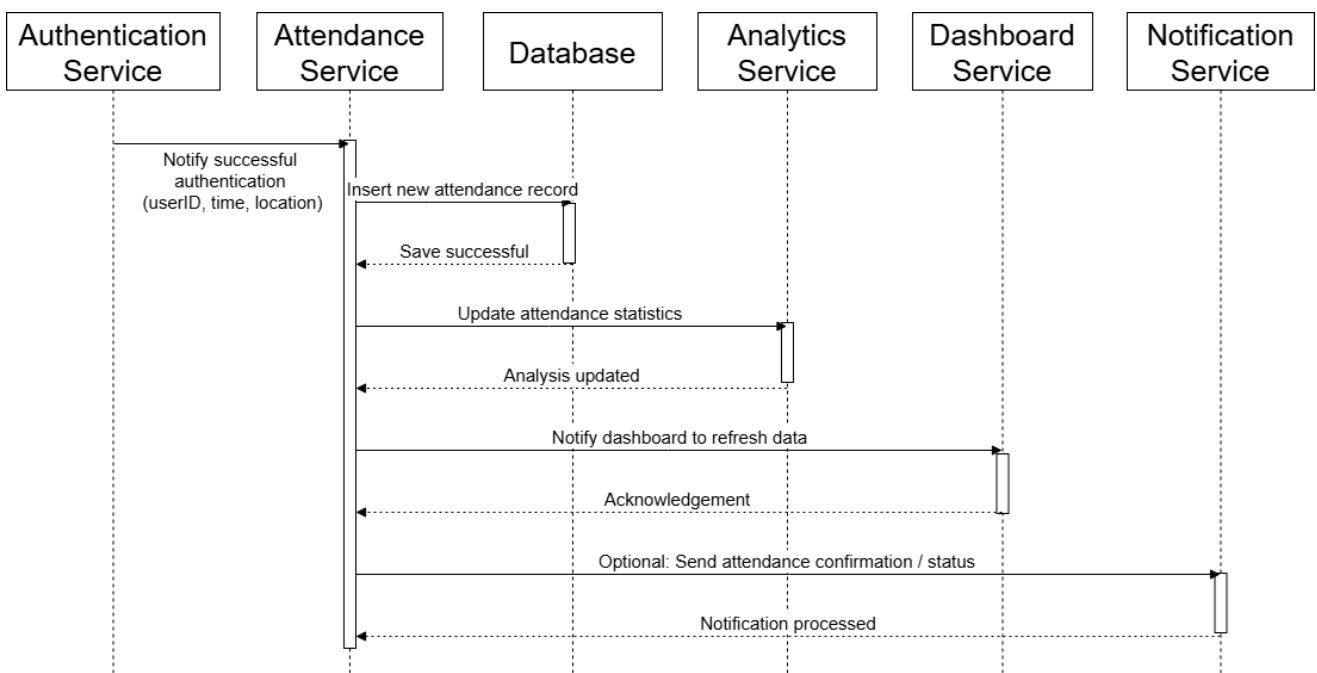


Figure 4.9: Sequence Diagram of Real Time Attendance

4.4.2.4 IoT-Based Smart Access Control Module

This sequence diagram explains the Smart Access Control Module. After authentication, the system checks the user's access permissions in the database. If permission is granted, the Raspberry Pi unlocks the door and logs the event. If permission is denied, no unlock command is issued; instead, the system reports "Access Denied" and sends a security alert through the Notification Service.

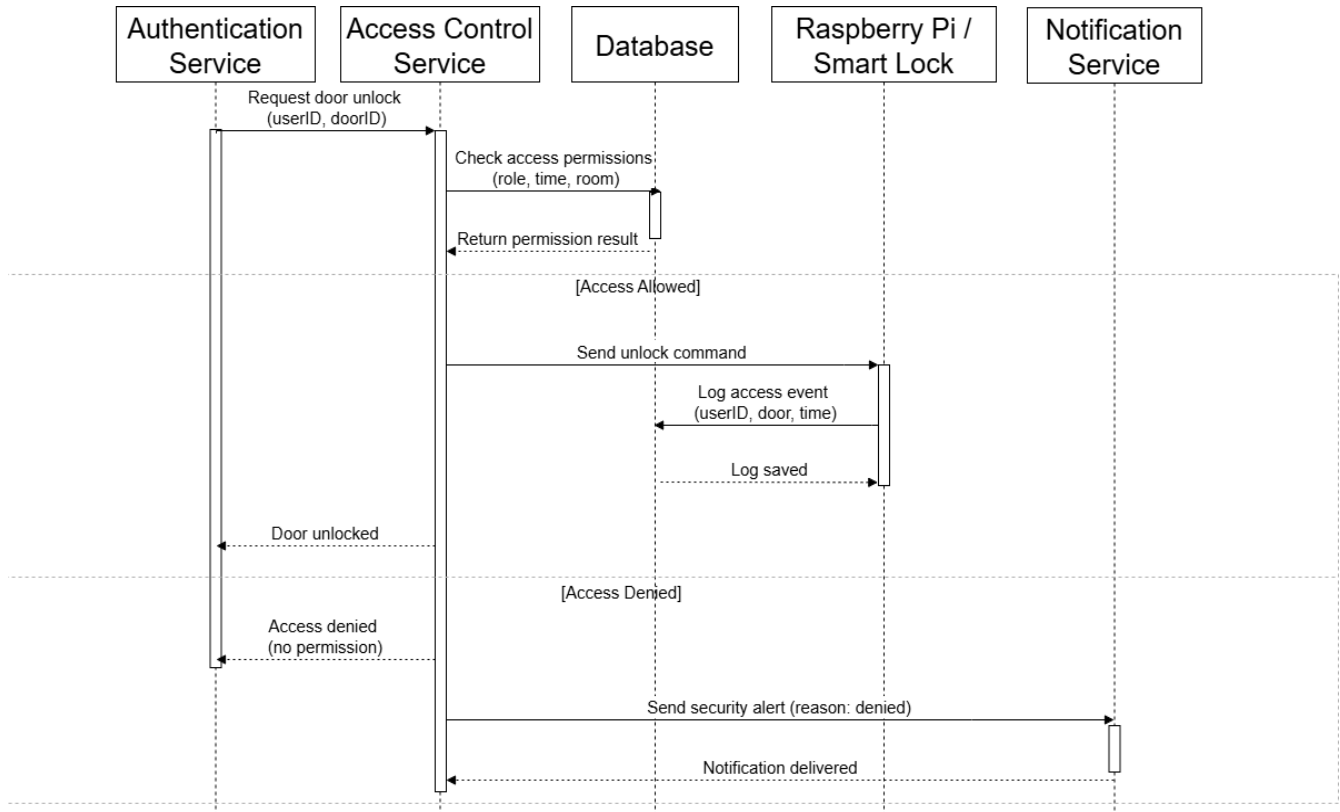


Figure 4.10: Sequence Diagram of IOT Based Smart Access Control

4.4.2.5 Admin Dashboard and Reporting Module

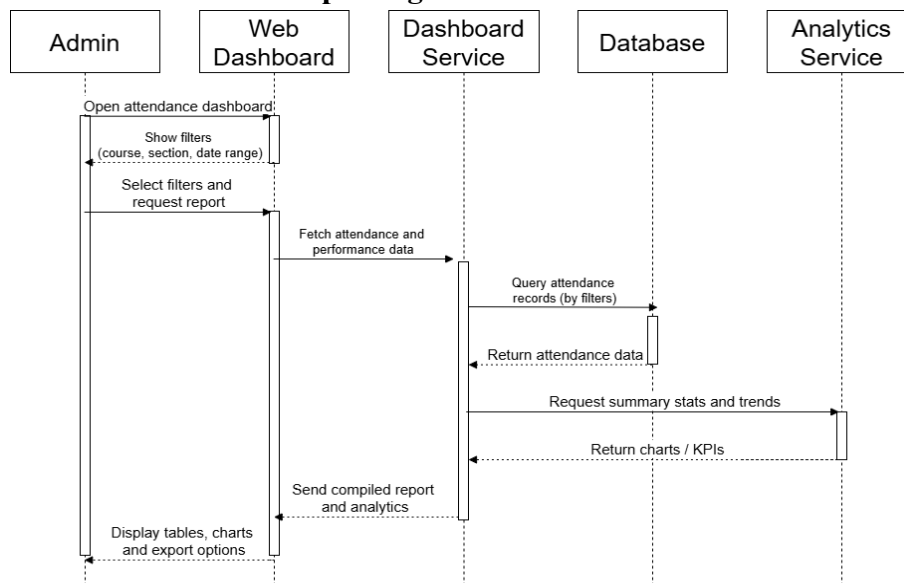


Figure 4.11: Sequence Diagram of Admin Dashboard

This sequence diagram represents the Admin Dashboard & Reporting Module. An admin or teacher opens the dashboard, applies filters, and the Web UI forwards this request to the Dashboard Service. The service retrieves attendance data from the database, combines it with analytics results (trends, KPIs), and returns a compiled report that the UI displays as tables and charts with export options for the user.

4.4.2.6 Student & Staff Notification Module

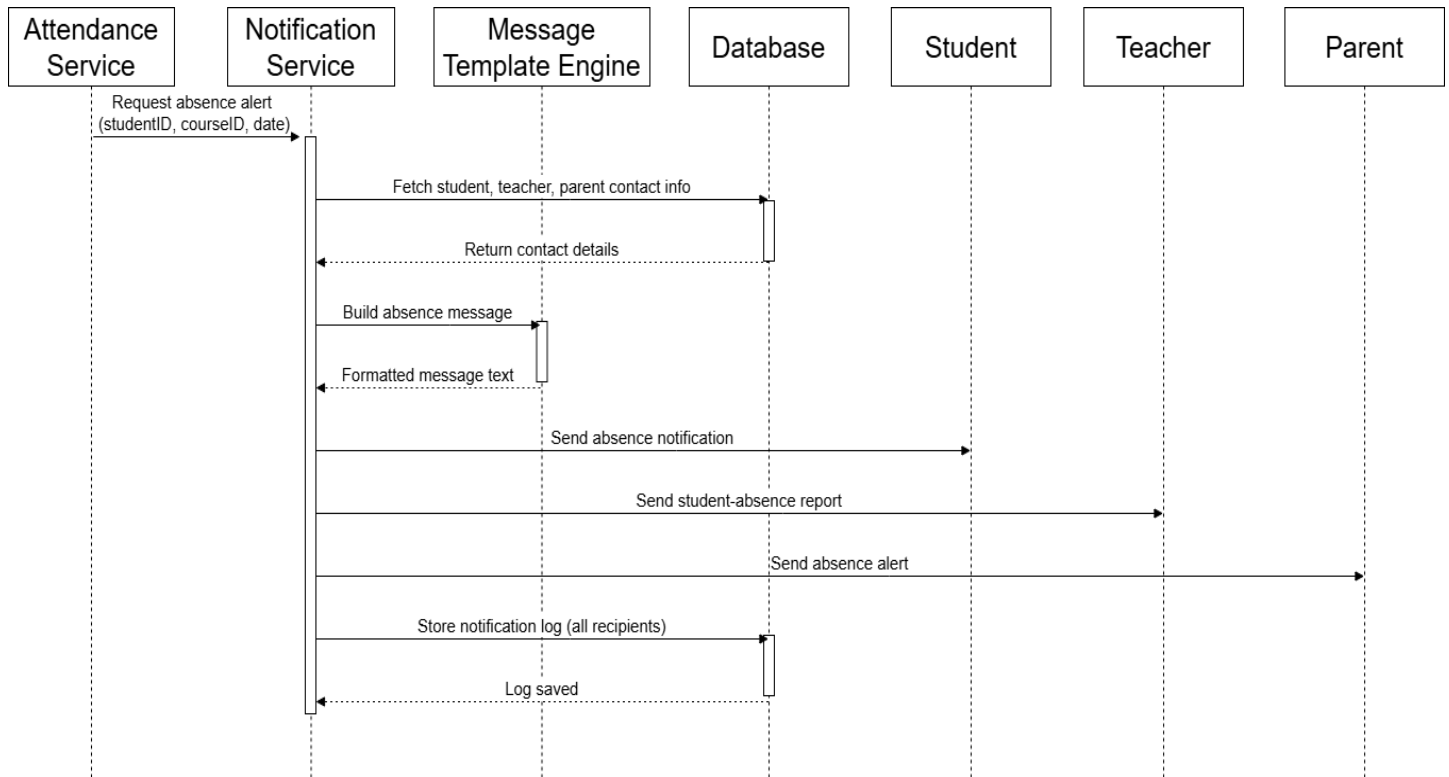


Figure 4.12: Sequence Diagram of Notification Module

In case of a missed lecture, the Attendance Service triggers the Notification Service, which fetches contact details for the student, teacher, and parent, builds an absence message, and sends it to all three recipients. Finally, the system stores the notification history in the database for auditing and tracking.

4.4.2.7 Privacy, Consent & Biometric Data Management

This sequence diagram describes the Privacy & Consent Management Module, where a user views and updates their consent settings or requests deletion of biometric data. The Consent Service retrieves existing records, updates consent status, and if needed, deletes biometric data from the database and disables biometric authentication for that user. The user receives a confirmation once the privacy changes are successfully applied.

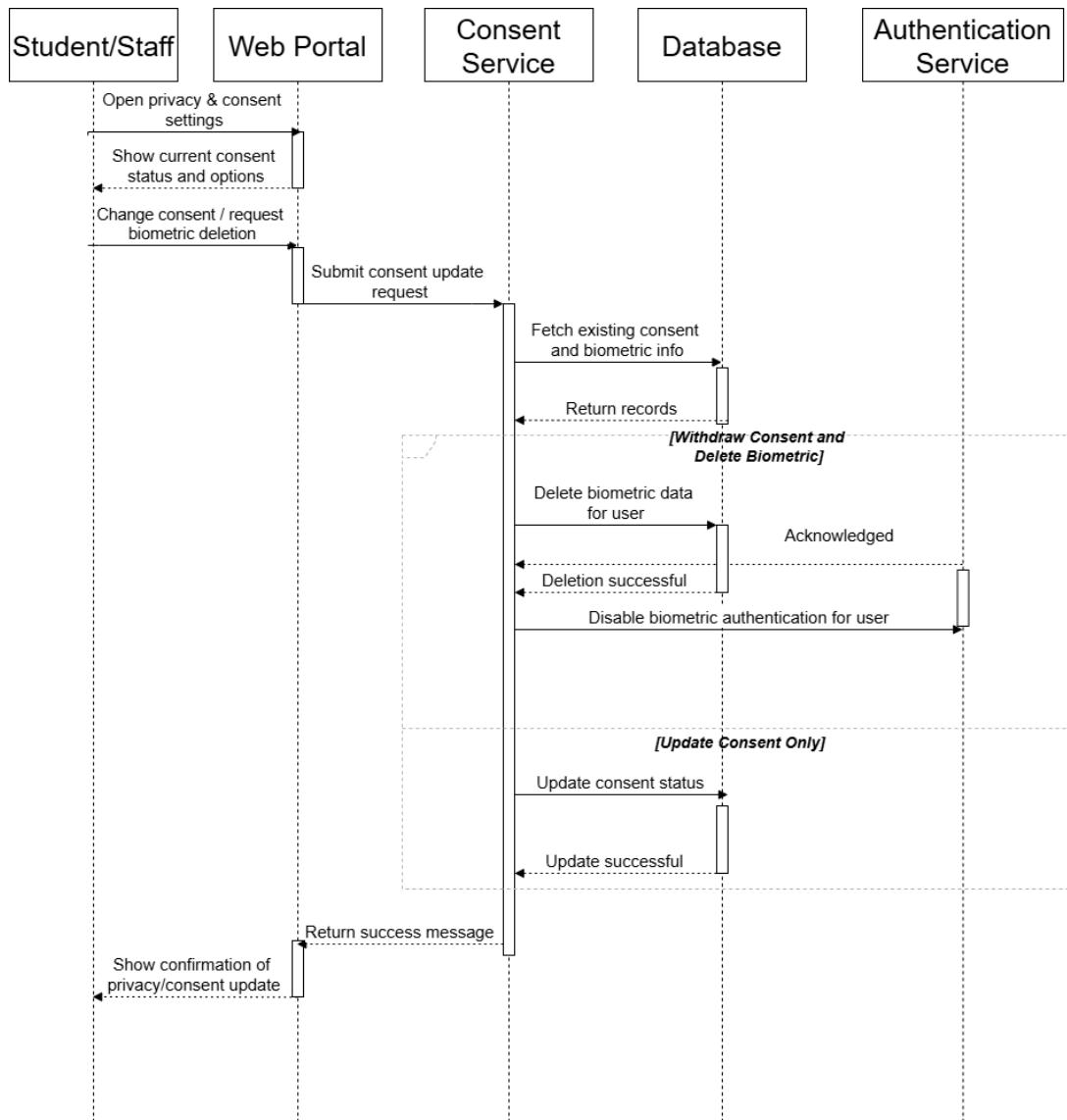


Figure 4.13: Sequence Diagram of Consent Management

4.4.2.8 Predictive Analytics Module

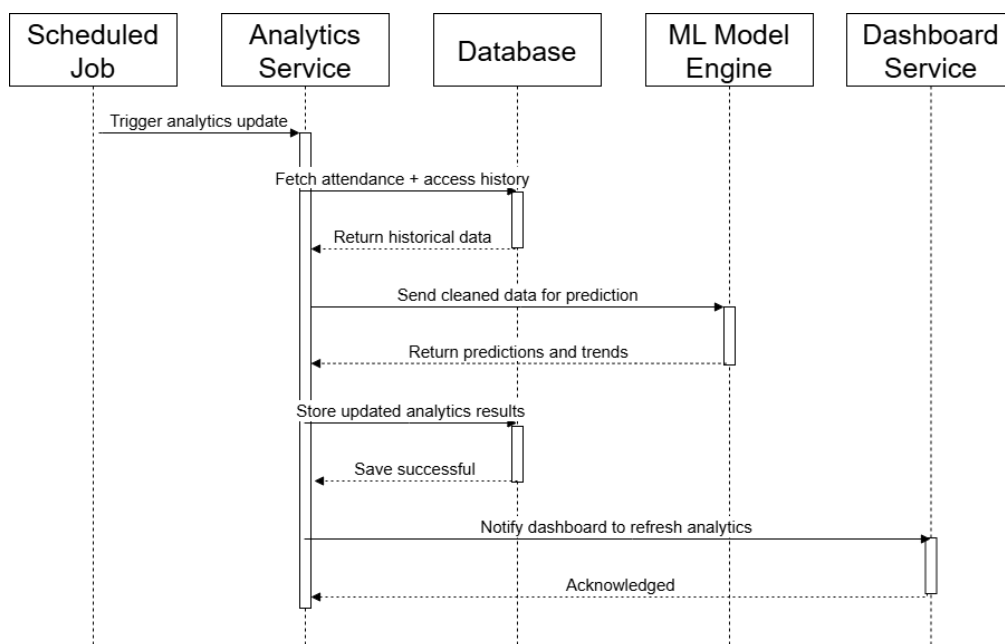


Figure 4.14: Sequence Diagram of Predictive Analytics

This sequence diagram shows how the Predictive Analytics Module automatically processes attendance and access data. A scheduled job triggers the Analytics Service, which retrieves historical data, sends it to the ML engine for predictions, stores updated results, and notifies the dashboard to refresh analytics. This enables early detection of absenteeism patterns or unusual behavior.

4.4.2.9 System Settings and Configuration Module

This sequence diagram models the System Settings & Configuration Module, where an admin manages global settings such as authentication methods, thresholds, and notification or access rules. The System Settings Service validates and stores changes, updates dependent services, and returns either validation errors or a success confirmation to the admin..

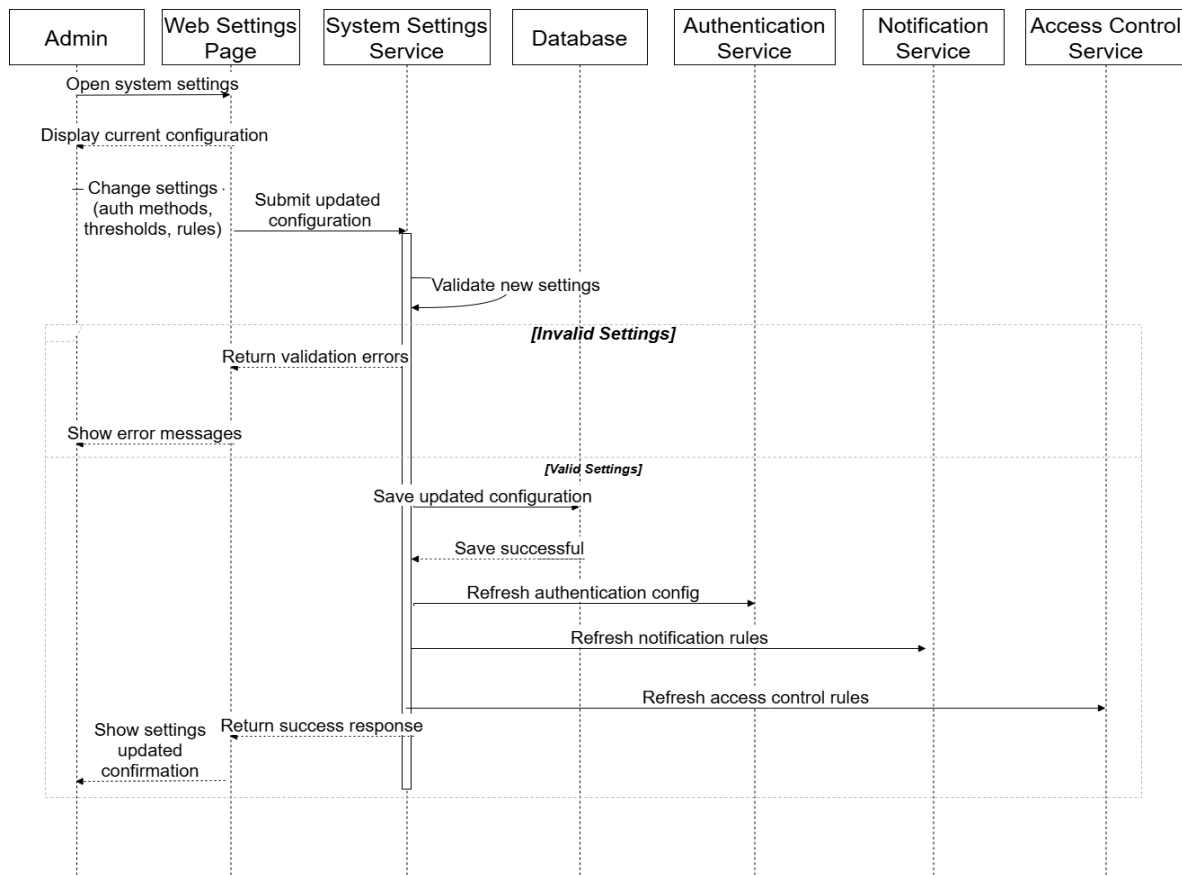


Figure 4.15: system Settings and configuration Module

4.4.3 State Transition diagrams

4.4.3.1 User Authentication Access

This diagram models the lifecycle of a user session in the SmartID Ecosystem, starting from Idle when the system is ready, through WaitingForInput when a user arrives, to Verifying once a credential (face, fingerprint, or RFID) is submitted. A successful match leads to either AccessGranted (door unlocked, attendance recorded) or AccessDenied (alert sent), while repeated failures trigger a Locked state requiring manual reset. All paths ultimately converge at

SessionEnded.

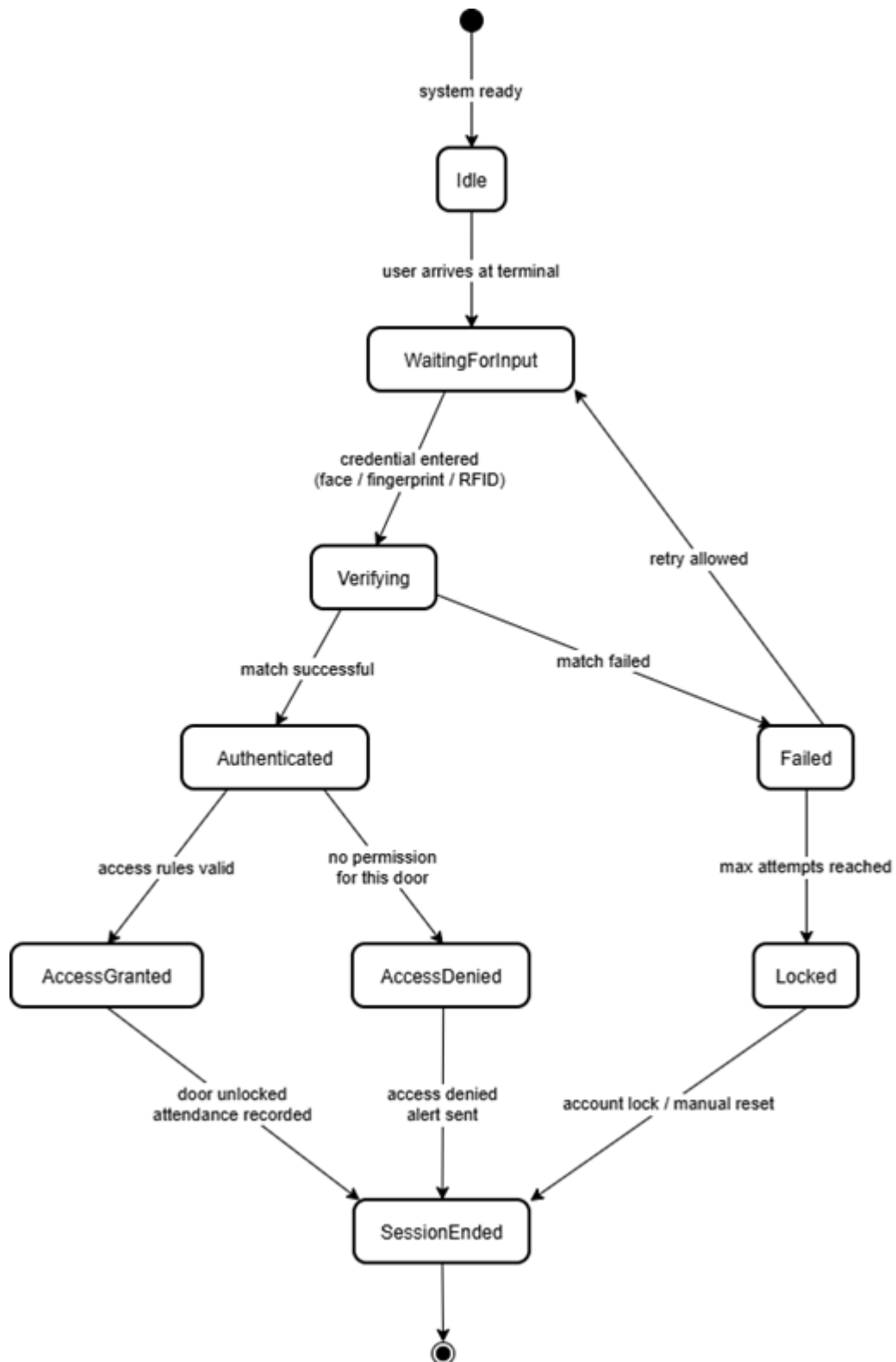


Figure 4.16: State Transition Diagram of User Authentication

4.4.3.2 Attendance Record Lifecycle

This state diagram represents how an attendance record progresses through the SmartID Ecosystem. Starting from the initial “NotCreated” state, it moves to pending marking after authentication, then transitions to “Marked” once saved. From there, it may be synced, trigger notifications, or be analyzed by the analytics engine. Error conditions loop back for retry to ensure reliability before reaching the final analyzed state.

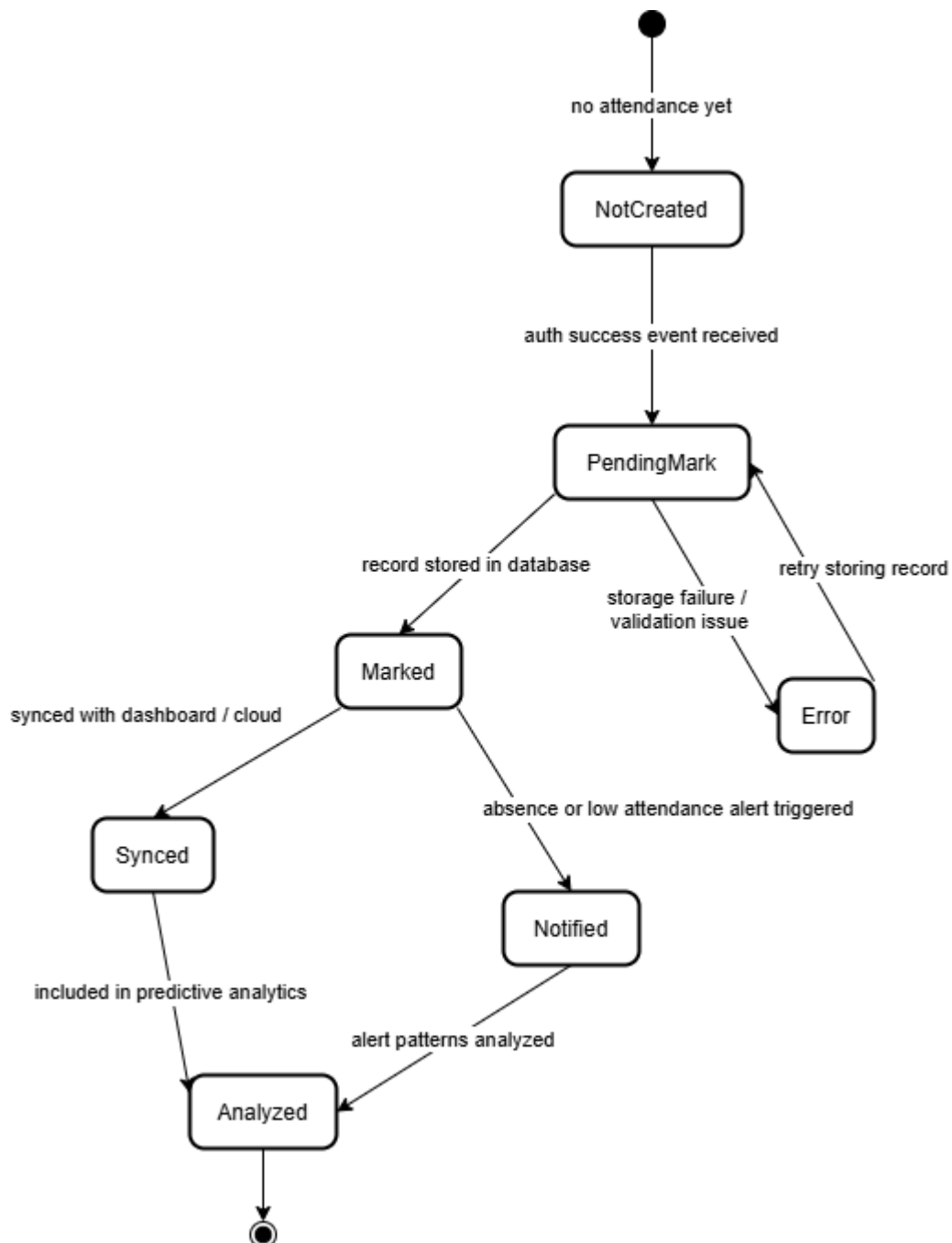


Figure 4.17: State Transition Diagram of Attendance Record Lifecycle

4.4.3.3 Smart Access Lifecycle

This state diagram models the lifecycle of a door-access request in the SmartID Ecosystem. An unlock request triggers a permission check, resulting in approval or rejection. Approved requests send a command to the Raspberry Pi, which may succeed or fail. All outcomes (granted, failed, or denied) are logged for auditing before reaching the final state.

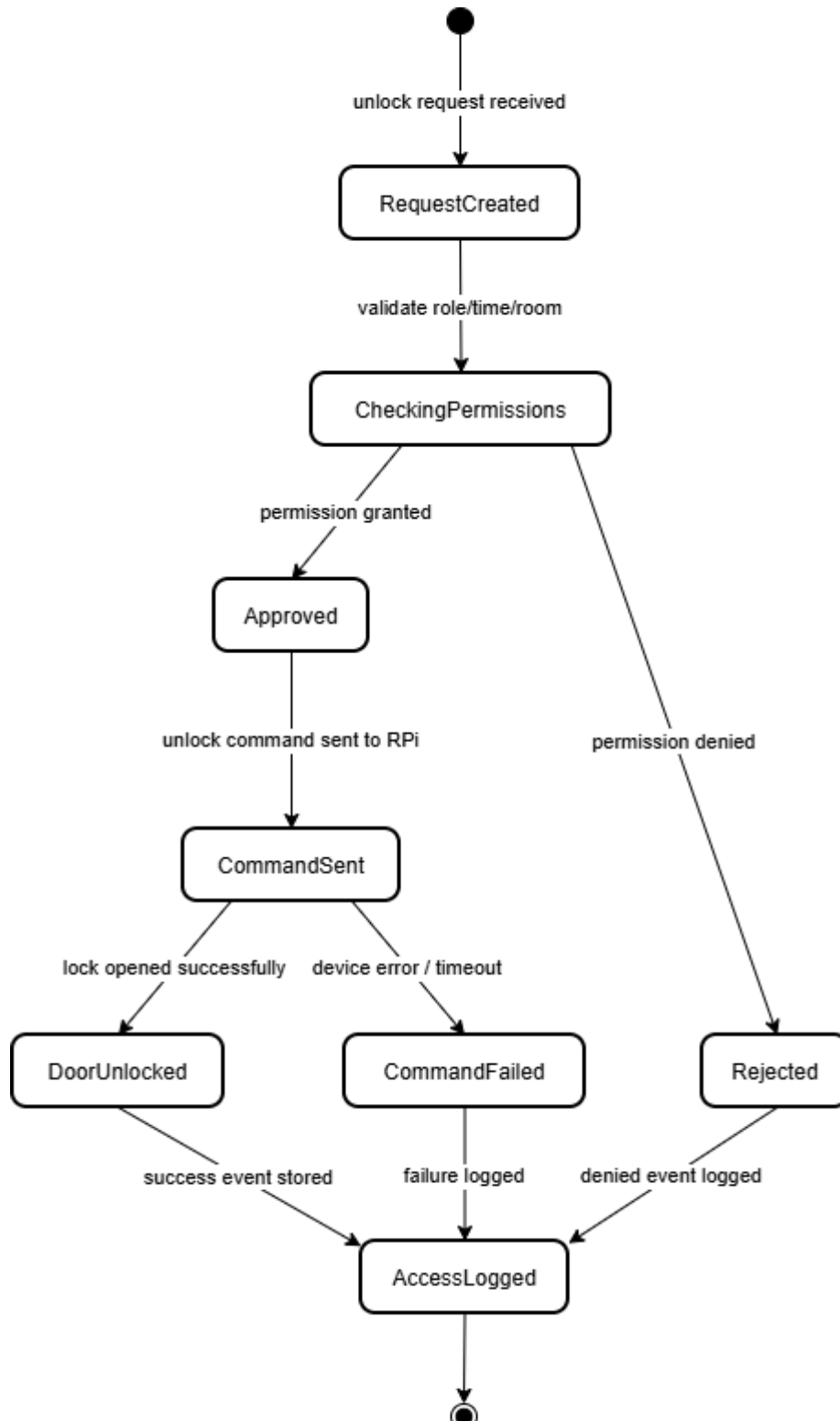


Figure 4.18: State Transition Diagram of Smart access

4.4.3.4 Notification Lifecycle

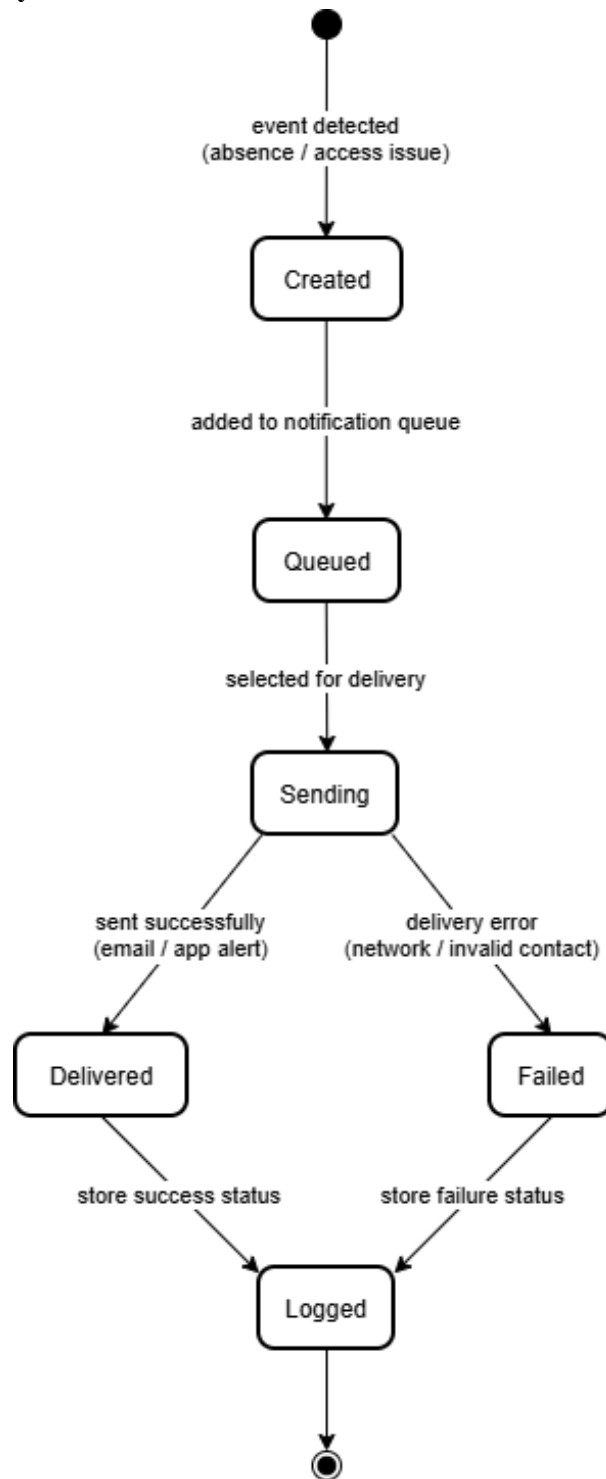


Figure 4.19: State Transition Diagram of Notification

This state diagram represents how a notification flows through the SmartID Ecosystem. When an event such as an absence or access problem occurs, a notification is created, queued, and then moved into the sending state. Depending on the outcome, it transitions to delivered or failed and finally reaches a logged state where the delivery result is stored for auditing.

4.4.3.5 Consent & Biometric Data Lifecycle

This state diagram models how user consent and biometric data evolve in the SmartID Ecosystem. It shows transitions from no consent to consent granted, biometric capture, active usage, and updates to consent preferences. If consent is withdrawn, biometric data is securely deleted, returning the system to a no-consent state to maintain user privacy and compliance.

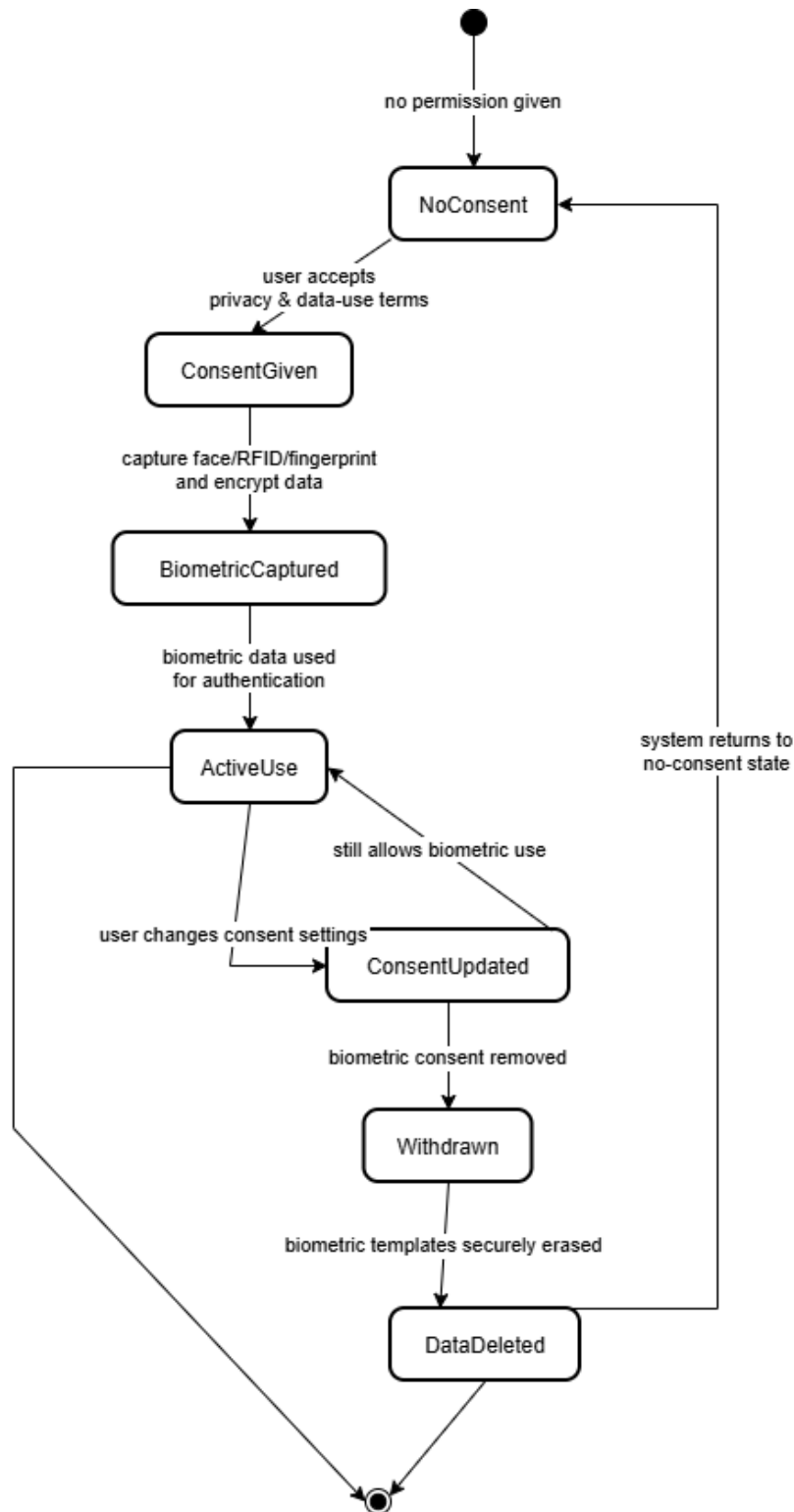


Figure 4.20: State Transition Diagram of Consent Management

4.4.4 Timing Diagram

This timing diagram shows the temporal relationship between the key signals involved in SmartID Ecosystem’s hardware flow. After the credential is captured at the terminal, the authentication processing signal goes high while the Raspberry Pi verifies the user. Once authentication succeeds, the relay output is activated, causing the door lock signal to go high and physically unlock the door. A notification event is then triggered slightly later, indicating that the system has logged the action and sent alerts to relevant users.

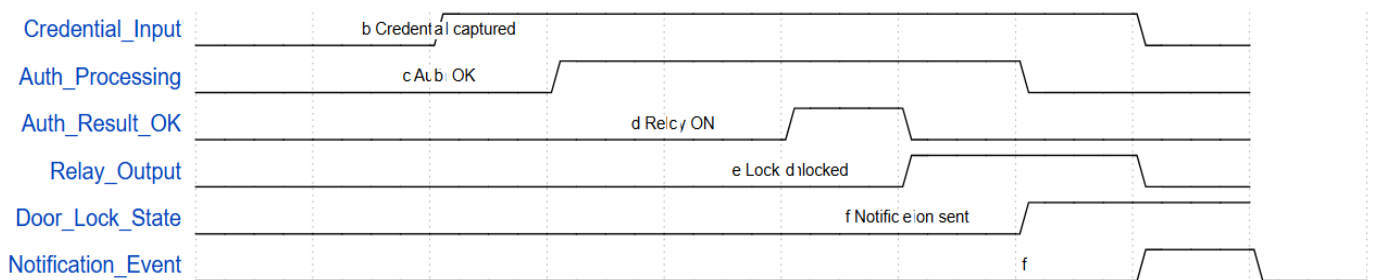


Figure 4.21: Timing Diagram of SmartID Ecosystem

4.4.5 Schematic Diagram

The schematic diagram illustrates the complete hardware configuration of the SmartID Ecosystem. A Raspberry Pi 4 acts as the central controller and interfaces with three authentication modules: the RC522 RFID reader via the SPI interface, a UART-based fingerprint sensor through TX/RX pins, and the Raspberry Pi Camera Module through the CSI port for facial recognition. A 5V-controlled relay module is connected to one of the Pi’s GPIO pins and is used to activate a 12V electronic door strike lock powered by an external battery pack. All components share a common ground to ensure stable signal reference. This hardware setup enables multi-modal authentication, secure access control, and seamless integration between sensors, actuators, and the backend system.

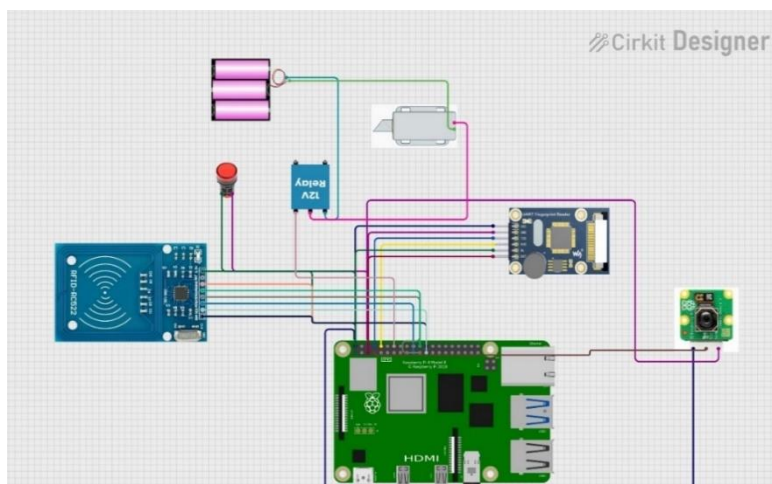


Figure 4.22: Schematic Diagram of SmartID Ecosystem

Chapter 5

IMPLEMENTATION

5 Implementation

This chapter explains how the SmartID Ecosystem was implemented at the technical level. It does not include source code but provides descriptions of the algorithms, external APIs, and user interface decisions that were used to build the system. The goal of this chapter is to show how the system's design was transformed into a functional software solution that ensures accurate attendance recording, secure multi-modal authentication, IoT-enabled access control, and real-time campus monitoring while maintaining privacy, inclusivity, and efficiency.

5.1 Algorithm

This section describes the main algorithms used in the SmartID Ecosystem. Each algorithm represents the internal logic behind a specific module and explains how different functions operate. These descriptions are presented in natural language and pseudocode to clearly communicate the workflow behind major modules such as authentication, attendance logging, access control, notification delivery, and predictive analytics.

5.1.1 Multi-Modal Authentication Algorithm

The multi-modal authentication algorithm is the core security mechanism of the SmartID Ecosystem. It supports three verification modes ,facial recognition, fingerprint scanning, and RFID card identification ,ensuring that users can authenticate even if one method is unavailable due to environmental or personal constraints. Liveness detection is incorporated into facial recognition to prevent spoofing attempts. The algorithm determines the user's enrolled method and routes the verification accordingly.

Steps:

1. User presents face, RFID card, or fingerprint at the SmartID terminal.
2. The system reads the current session configuration and identifies the selected authentication method.
3. If the method is Face Recognition:
 - a. Capture a live image frame from the camera module.
 - b. Apply liveness detection to verify that the input is from a real person.
 - c. Extract a 128-dimensional facial embedding using the FaceNet model.
 - d. Compare the embedding against stored encrypted embeddings in the database.
 - e. If the similarity score exceeds the configured threshold, proceed to Step 6.
4. If the method is Fingerprint:
 - a. Capture fingerprint data from the connected sensor.
 - b. Match the captured template against the stored fingerprint template for the user.
 - c. If a match is confirmed, proceed to Step 6.

5. If the method is RFID:
 - a. Read the tag ID from the RFID reader.
 - b. Look up the tag ID in the RFIDTag table linked to an active user account.
 - c. If a match is found and the user is active, proceed to Step 6.
6. If authentication is successful:
 - a. Log the authentication event with user ID, timestamp, method, and location.
 - b. Trigger the Attendance Logging Module to record presence.
 - c. Trigger the IoT Access Control Module to evaluate door permissions.
7. If authentication fails:
 - a. Increment the failed attempt counter for the session.
 - b. If failed attempts exceed the configured maximum, temporarily lock the terminal session and send an alert.
 - c. Display a failure message and prompt the user to retry.

5.1.2 Automated Attendance Logging Algorithm

This algorithm is responsible for automatically recording student and staff attendance immediately after a successful authentication event. It eliminates the need for manual attendance entry by faculty, ensures tamper-proof records, and handles edge cases such as duplicate entries and late arrivals. At the end of each scheduled session, a background job marks absent status for all enrolled users who did not authenticate.

Steps:

1. The Authentication Module sends a success signal containing the user ID, timestamp, and location to the Attendance Manager.
2. The system checks whether an attendance record already exists for the given user, course, section, and date.
3. If a record already exists:
 - a. Update the timestamp if the new entry is earlier (first-entry rule).
 - b. Return the existing record ID without creating a duplicate.
4. If no record exists:
 - a. Create a new AttendanceRecord with status set to PRESENT and the current timestamp.
 - b. Save the record to the relational database.
 - c. Sync the record to the cloud-based attendance store in real time.
5. After the scheduled session window closes, the system executes a batch absence job:
 - a. Retrieve all enrolled students for every active course and section.
 - b. For each student without a PRESENT record on the current date, create an ABSENT entry.

6. If a student's absence count crosses the configured threshold:

Trigger the Notification Module to send absence alerts to the student, teacher, and parent.

7. Update the Admin Dashboard with the latest attendance statistics and course-level summaries.

5.1.3 IoT-Based Smart Access Control Algorithm

The smart access control algorithm manages physical entry to classrooms, laboratories, and restricted campus areas by linking digital authentication results to hardware-level door commands. Raspberry Pi devices serve as edge controllers that receive authorization signals from the backend and activate relay-controlled electronic door strikes. All access decisions are governed by role-based and time-based rules, and every event is logged for security auditing.

Steps:

1. A successful authentication event triggers the Access Control Service with the authenticated user ID and door device ID.
2. The system fetches the user's role and the access rules stored in SystemSettings.
3. Evaluate time-based conditions:
 - a. Check if the current time falls within the allowed access window for the user's role and the specified door.
 - b. If the time is outside the allowed window, set the decision to DENIED.
4. Evaluate role-based permissions:
 - a. Check whether the user's assigned role has permission to access the requested door or area.
 - b. If permission is not granted, set the decision to DENIED.
5. Log the access attempt in the AccessLog table with user ID, door ID, decision, and timestamp regardless of outcome.
6. If the decision is GRANTED:
 - a. Send an unlock command to the corresponding Raspberry Pi via HTTP or MQTT.
 - b. The Pi activates the GPIO pin connected to the relay for a configured pulse duration (e.g., 3 seconds).
 - c. The relay energizes the 12V electronic door strike, physically unlocking the door.
 - d. After the pulse duration, the GPIO pin returns LOW, re-locking the door.
7. If the decision is DENIED:
 - a. No unlock command is issued.
 - b. The Notification Service sends an access-denied alert to the responsible administrator.

5.1.4 Parental Notification Algorithm

The parental notification algorithm automates the communication of attendance-related events to parents and guardians, addressing a critical gap identified in existing systems. Whenever a student is marked absent, fails repeated authentication attempts, or triggers an anomaly flag from the analytics engine, the system composes and dispatches targeted notifications through email or in-application channels. This feature promotes accountability and maintains transparent communication between the institution and families.

Steps:

1. The Notification Service receives a trigger event from one of three sources: the Attendance Manager (absence), the Authentication Module (repeated login failure), or the Analytics Service (high risk flag).
2. The service identifies the event type: Absence Alert, Access Denied Alert, or Risk Alert.
3. Fetch the contact details for all relevant recipients ,student, linked parent or guardian, and the assigned teacher ,from the database.
4. Compose the notification message using the appropriate template: Include the student's name, course, date, type of event, and any relevant statistics such as cumulative absence count.
5. For each recipient, determine the enabled delivery channels (Email or In-App Notification) by reading the user's notification preferences.
6. Dispatch the message through each enabled channel:
 - a. For Email: invoke the Email Service with recipient address and composed message body.
 - b. For In-App: push the notification to the user's application notification queue.
7. Record a Notification entry in the database for each dispatch attempt, capturing the recipient, channel, content summary, timestamp, and delivery status (SENT or FAILED).
8. If delivery fails, log the failure in the SystemLog table for administrative review and schedule a retry if the retry policy is enabled.

5.1.5 Predictive Analytics Algorithm

The predictive analytics algorithm processes historical attendance and access log data to forecast absenteeism risk and detect unusual behavioral patterns. It runs as a scheduled background job and produces risk scores stored in the AnalyticsResult table, which are then visualized on the Admin Dashboard. Early identification of at-risk students allows administrators and faculty to intervene before academic performance is seriously affected.

Steps:

1. The Analytics Service is triggered by a scheduled job (e.g., daily at midnight).
2. Retrieve all AttendanceRecord and AccessLog entries for the preceding N days for every

enrolled student.

3. Aggregate the data per student:
 - a. Compute totalLectures, presents, absents, lateCount, and deniedAccessCount.
 - b. Calculate the absence rate: $\text{absenceRate} = \text{absents} / \text{totalLectures}$.
4. Apply the risk classification model:
 - a. If absenceRate exceeds the HIGH threshold (configurable, e.g., 25%), assign riskLevel = HIGH.
 - b. If absenceRate exceeds the MEDIUM threshold (e.g., 15%) but is below HIGH, assign riskLevel = MEDIUM.
 - c. Otherwise, assign riskLevel = LOW.
5. Store a new AnalyticsResult record for each student containing the date, risk level, and computed metrics.
6. For students classified as HIGH risk: Trigger the Notification Service to send a risk alert to the student, parent, and academic advisor.
7. Refresh the Admin Dashboard analytics view so that updated charts and KPIs are immediately available to administrators and faculty.

5.1.6 User Enrollment Algorithm

The user enrollment algorithm governs the registration of new students and staff into the SmartID Ecosystem. It ensures that every user provides the required information, consents to the use of their biometric data, and successfully registers at least one authentication method before the account becomes active. Raw biometric images and fingerprint scans are never stored; only encrypted embeddings or templates are retained, in compliance with the system's privacy-first design.

Steps:

1. The System Administrator opens the Enrollment Portal and selects the user role (Student or Teacher).
2. The admin enters the user's personal information: name, institutional ID, email address, department, and contact details.
3. The system validates all input fields for completeness and uniqueness of the email or ID.
4. If validation fails, display specific error messages and return to the form without saving.
5. Present the digital consent form to the user or administrator:
 - a. The user must explicitly accept biometric data collection terms before proceeding.
 - b. Consent status is stored with a timestamp in the Consent table.
6. The user selects one or more preferred authentication methods: Facial Recognition,

Fingerprint, or RFID.

7. For each selected method, initiate the biometric capture process:
 - a. Face: Capture multiple image frames, extract 128-D FaceNet embeddings, and store them encrypted in the BiometricTemplate table.
 - b. Fingerprint: Scan the fingerprint, generate a template, and store it encrypted in the FingerprintTemplate table.
 - c. RFID: Read the card tag ID and store it in the RFIDTag table linked to the user record.
8. Create the User record in the database, assign the appropriate Role, and set account status to Active.
9. Send an enrollment confirmation notification to the user's registered email address.

5.1.7 Privacy and Consent Management Algorithm

The privacy and consent management algorithm enforces the ethical handling of sensitive biometric data throughout the system lifecycle. Users retain control over their data at all times and may revoke consent or request deletion of their biometric information through the portal. Upon revocation, the system automatically removes all associated biometric records and disables the affected authentication methods, ensuring compliance with institutional privacy policies.

Steps:

1. The user accesses the Privacy and Consent Management section of the web portal.
2. The system retrieves and displays the user's current consent preferences and enrolled biometric methods.
3. The user modifies their preferences ,for example, revoking permission for facial recognition data.
4. The system compares the updated preferences against the previously stored flags.
5. If biometricAllowed has changed from TRUE to FALSE for any method:
 - a. Delete or anonymize the corresponding records from BiometricTemplate, FingerprintTemplate, or RFIDTag tables.
 - b. Log the deletion event in the SystemLog table for audit purposes.
 - c. Notify the Authentication Service to disable the affected method for this user.
6. Update the Consent record in the database with the new preference flags and the current timestamp.
7. Display a confirmation message to the user confirming that their privacy settings have been applied.

5.2 External APIs

The table below summarizes the key APIs used in the SmartID Ecosystem, their purpose, and where they are integrated within the system. These include machine learning APIs such as FaceNet and MTCNN for biometric facial recognition, the RPi.GPIO library for IoT-based smart door control, the MFRC522 library for RFID card reading, the Flask REST API for backend communication, the SMTP Email API for parental and user notifications, the SQLite API for data storage, and TensorFlow/Keras for machine learning inference. Each API is carefully integrated to ensure security, reliability, and seamless interaction between the frontend, backend, and hardware components.

Table 5.1: External API's used in our SmartID Ecosystem

Name of API	Description of API	Purpose of Usage	Function Name in which Used
FaceNet (via TensorFlow/Keras)	A deep learning model that converts facial images into 128-dimensional numerical embeddings used to uniquely represent and compare individual faces.	Used to extract facial embeddings during enrollment and to verify a user's identity during authentication by comparing live embeddings against stored encrypted embeddings using cosine similarity.	AuthenticationService.verifyFace(), EnrollmentService.registerUser()
MTCNN (Multi-Task Cascaded Convolutional Networks)	A face detection library that locates, crops, and aligns the face region within a camera frame before passing it to FaceNet for embedding extraction.	Used as the preprocessing stage of facial recognition to isolate the face region from camera input, improving recognition accuracy under varying lighting and camera angles.	AuthenticationService.verifyFace(), EnrollmentService.registerUser()
TensorFlow / Keras API	An open-source machine learning runtime that provides APIs for loading pre-trained models, executing inference, and performing tensor-level computations within the Python backend.	Used to load and run the FaceNet model for facial embedding generation and to power the predictive analytics module that computes student absenteeism risk scores.	AuthenticationService.verifyFace(), AnalyticsService.predictRisk(), AnalyticsService.runDailyAnalytics()
RPi.GPIO (Raspberry Pi GPIO Library)	A Python library that provides an API to configure and control the General Purpose Input Output pins on a Raspberry Pi, allowing software to send timed	Used to send pulse signals to the relay module connected to the electronic door strike, physically unlocking classroom and lab doors when a	AccessControlService.sendUnlockCommand()

	electrical signals to connected hardware such as relay modules and door strikes.	user's access request is authorized by the backend.	
MFRC522 (RC522 RFID Reader Library)	A Python library that abstracts the SPI communication between the Raspberry Pi and the RC522 RFID reader hardware, providing functions to detect card presence and read unique tag IDs from RFID proximity cards.	Used to read the unique tag ID from a student's or staff member's RFID card at the authentication terminal and forward it to the backend Authentication Service for identity verification.	AuthenticationService.verifyRFID(), EnrollmentService.registerUser()
Flask REST API (Flask Framework)	A lightweight Python web framework that exposes RESTful HTTP endpoints, handles request routing, input validation, and enforces role-based access control for all backend service modules.	Used to expose all system functionalities ,enrollment, authentication, attendance, access control, notifications, analytics, and system settings ,as secure HTTPS API endpoints consumable by the React frontend and IoT devices.	EnrollmentService.registerUser(), AuthenticationService.startSession(), AttendanceManager.markAttendance(), AccessControlService.checkAndUnlock(), NotificationService.sendAbsenceAlert(), SystemSettingsService.updateSettings()
SMTP Email API (via smtplib / Flask-Mail)	Python's smtplib library or Flask-Mail extension provides an API for composing and dispatching email messages over the SMTP protocol to external email addresses with support for server authentication and message formatting.	Used to deliver absence alerts, access-denied warnings, enrollment confirmations, and high-risk notifications to students, parents, teachers, and administrators through their registered email addresses.	NotificationService.sendAbsenceAlert(), NotificationService.sendAccessDeniedAlert(), EnrollmentService.registerUser()
SQLite (via Python sqlite3 Module)	An embedded relational database engine accessed through Python's built-in sqlite3 module, providing a SQL API for creating, reading, updating, and deleting records without requiring a separate database server process.	Used as the primary data store for all system entities including users, roles, consent records, biometric templates, attendance records, access logs, notifications, analytics results, and system configuration settings.	AttendanceManager.markAttendance(), AttendanceManager.handleAbsence(), AccessControlService.logAccessEvent(), ConsentService.updateConsent(), ConsentService.deleteBiometricData(), AnalyticsService.runDailyAnalytics()

5.3 User Interface

The user interface of the SmartID Ecosystem is designed to provide a secure, intuitive, and efficient experience for all user types including students, faculty members, system administrators, and parents. It focuses on clarity, role-based access, and real-time feedback, ensuring that users can complete tasks such as enrolling in the system, authenticating their identity, monitoring attendance, and managing system settings with minimal effort. Consistent navigation elements, clearly labeled action buttons, and color-coded status indicators help users understand system responses at every step. The design follows modern web principles with a clean layout structure, ensuring smooth interaction across different screen sizes. Visual dashboards, structured data tables, and contextual alerts reduce cognitive load and support data-driven decision-making for administrators and faculty. Overall, the SmartID interface is built to minimize errors, promote transparency, and provide every user type with exactly the information and controls relevant to their role.

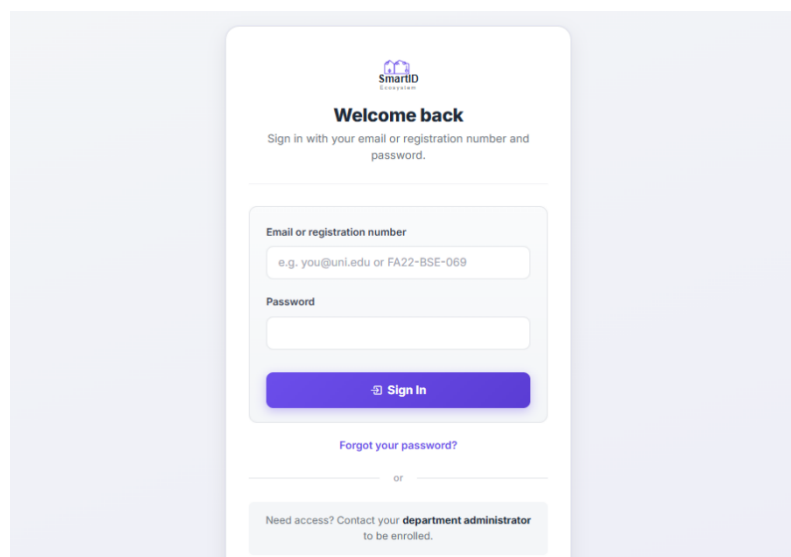


Figure 5.1: Login page

5.1 shows the Login Page of the SmartID Ecosystem, which serves as the secure entry point for all system users including administrators, faculty, students, and parents. The screen presents an email and password input form along with a role-specific access restriction notice, clearly communicating that this interface is restricted to authorized users only. Options such as "Remember Me," "Forgot Password," and alternative sign-in methods through Google and GitHub are provided to improve convenience while maintaining security. The clean, minimal layout ensures that users can authenticate quickly without confusion, setting the right tone for the rest of the system experience.

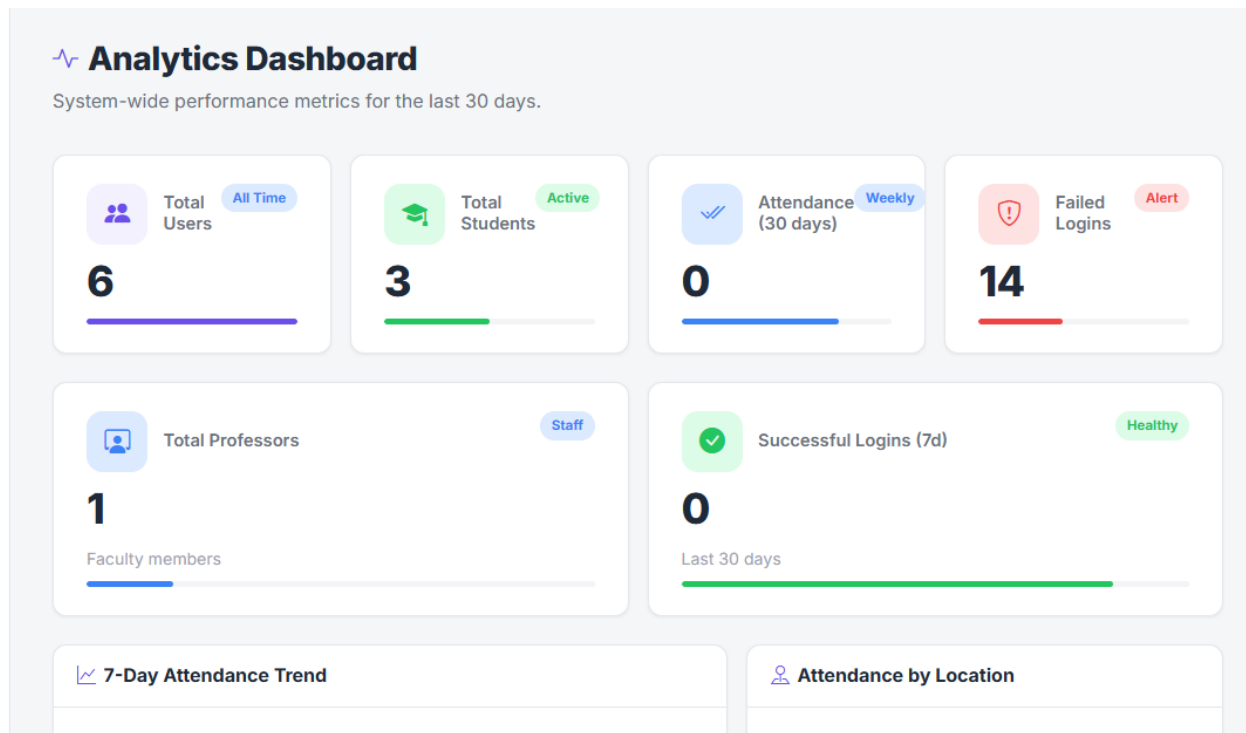


Figure 5.2: Admin Dashboard

Figure 5.2 displays the Admin Dashboard of the SmartID Ecosystem, which provides system administrators with a centralized view of all key performance indicators and system operations. The dashboard presents real-time KPI cards showing the total number of enrolled students, active users, registered courses, and enabled authentication methods, giving administrators an instant overview of the system's current state. A navigation sidebar allows quick access to user management, student records, enrollment, authentication settings, and today's attendance logs. The recent enrollments table at the bottom of the dashboard enables administrators to monitor new registrations and take immediate action through inline edit, view, and remove controls.

User Enrollment
Enroll students or teachers with complete profile and biometric data

Select Role

Select User Role
Choose whether you're enrolling a student or teacher

Student
Enroll a new student

Teacher
Enroll a new teacher

Continue →

Figure 5.3: User Enrollment Page

Figure 5.3 shows the User Enrollment Screen, which follows a structured three-step wizard pattern guiding administrators through role selection, personal information entry, and biometric capture. In the first step, the administrator selects whether the new user is a student or a teacher, ensuring that the subsequent form fields and authentication options are tailored to the selected role. This staged approach prevents input errors by presenting only the fields relevant to each step, allowing the administrator to review and confirm data before advancing to the biometric registration phase. The progress indicator at the top of the screen helps users track where they are always in the enrollment process.

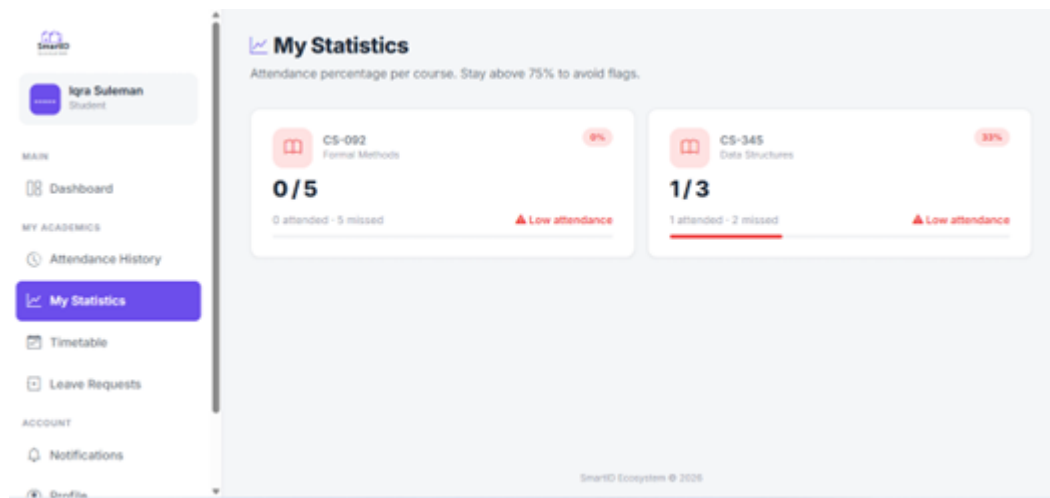


Figure 5.4: Statistics of Attendance

Figure 5.4 shows a student-facing attendance analytics module within an academic management system. It provides a per-course breakdown of attendance performance, displaying each enrolled course as a card that shows the total classes attended versus total classes held, the number of sessions missed, and the overall attendance percentage. A progress indicator and a warning flag highlight courses where attendance falls below the required threshold (75%), helping students proactively identify and address attendance issues before they result in academic consequences. Figure 5.5 shows a student-facing absence management module within an academic management system. It allows students to formally submit leave or absence requests for specific courses by providing the relevant course, date, and a brief reason for the absence. Once submitted, requests are logged and tracked in a personal request history table, where students can monitor the status of each submission whether it is pending, approved, or rejected, along with details on which faculty member reviewed it.

Figure 5.5: Leave Request

Figure 5.6 shows a faculty-facing dashboard module within an academic management system. It provides instructors with a high-level visual overview of attendance trends across all the courses they teach. The dashboard surfaces key summary metrics at a glance ,such as the total number of active courses and the count of pending leave requests awaiting review. It also presents two data visualizations: a 7-day attendance trend line chart that tracks day-by-day attendance patterns over the past week, and a donut chart that breaks down average attendance distribution across individual courses. Together, these components help faculty quickly identify attendance irregularities, monitor student engagement over time, and make informed decisions about follow-up actions ,all from a single consolidated view.



Figure 5.6: Teaching Analytics

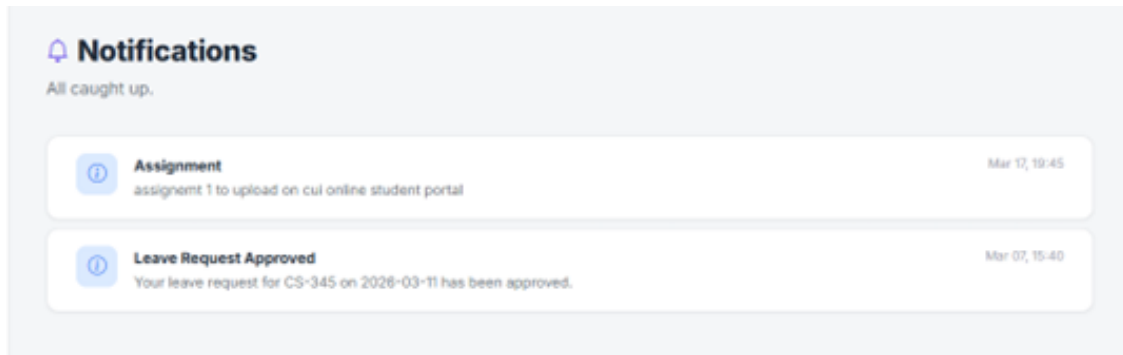


Figure 5.7: Notifications

Figure 5.7 displays the Student and Staff Notification Screen, which presents a log of all system-generated alerts sent to students, staff, and parents. Each notification entry shows the recipient, notification type such as absence alert or access-denied warning, the delivery channel used, and the timestamp of dispatch. This screen allows administrators to verify that notifications have been successfully delivered and to identify any failed dispatch events that may require follow-up. The structured layout ensures that all communication activity is transparent and auditable, supporting the system's accountability and parental engagement objectives.

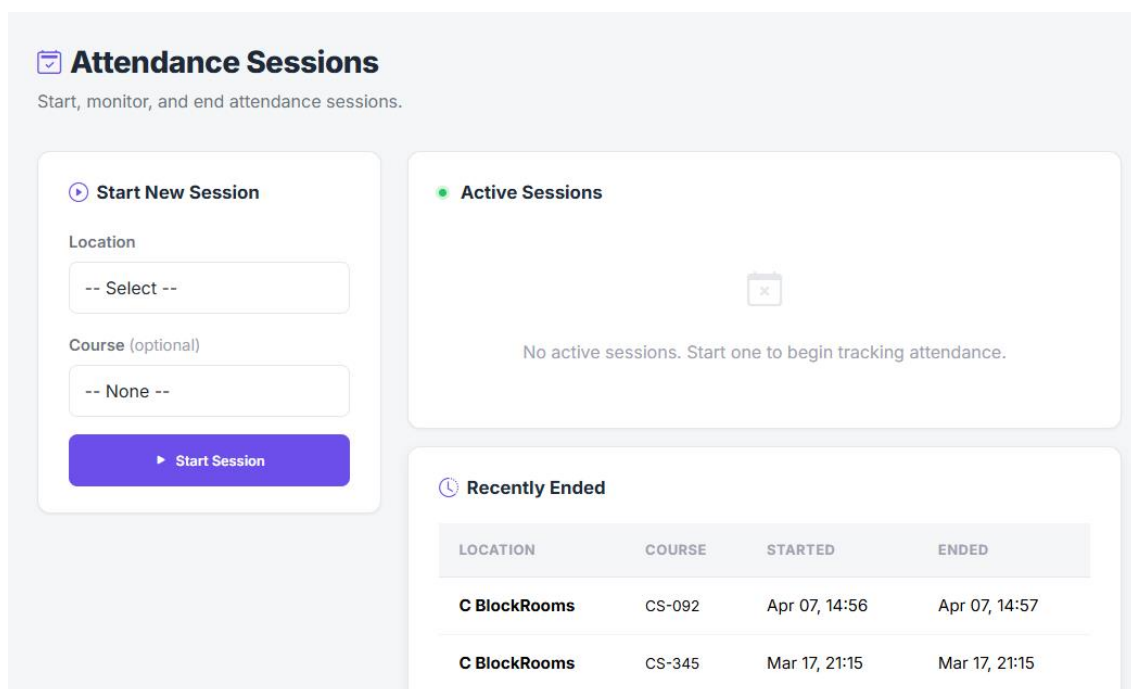


Figure 5.8: Attendance Session

Figure 5.8 shows a faculty-facing session management module within an academic management system. It enables instructors to initiate, monitor, and close attendance tracking sessions for their classes. To start a session, the instructor selects a physical location and optionally associates it with a specific course, then launches the session with a single action. Once active, live sessions are displayed in real time so the instructor can monitor ongoing attendance capture. The module also maintains a log of recently ended sessions, showing key details such as location, associated course, and the exact start and end timestamps. This gives faculty a clear operational record of

when and where attendance was collected, supporting both accountability and administrative follow-up.

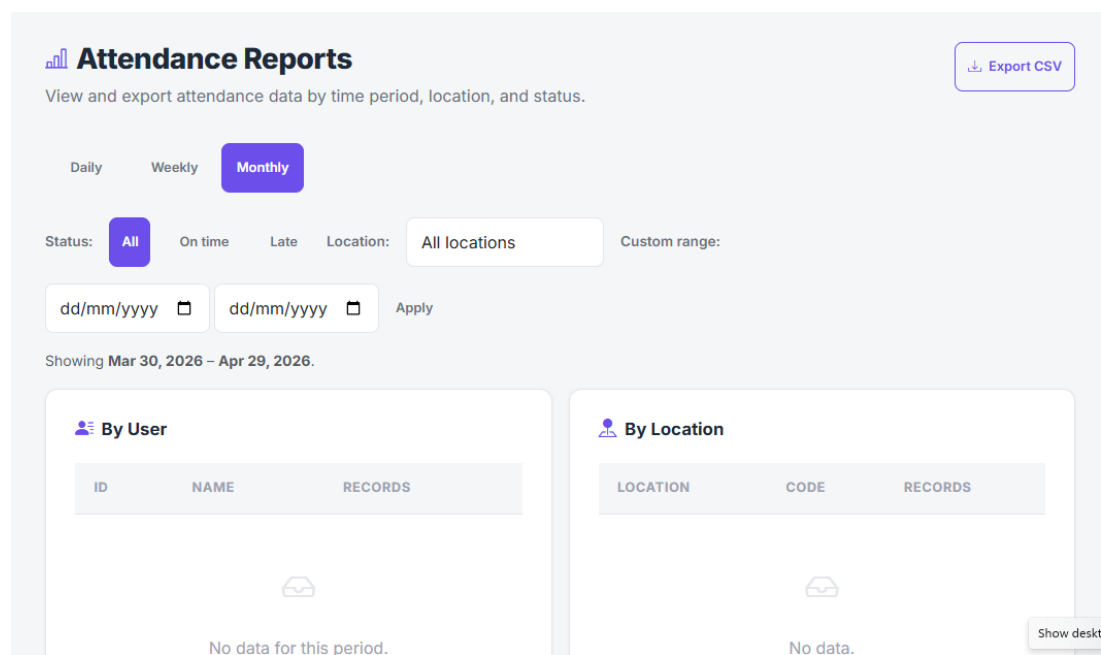


Figure 5.9: Attendance reports

Figure 5.9 shows a administrative reporting module within an academic management system. It allows authorized users to query, analyze, and export attendance data across configurable dimensions. Users can filter records by time granularity ,daily, weekly, or monthly ,as well as by attendance status (all, on time, or late), physical location, and a custom date range. The results are presented in two parallel breakdowns: one aggregated by individual user, showing each person's attendance record count, and another aggregated by location, showing how many attendance records were captured at each physical venue. The module also includes a one-click CSV export option, enabling administrators to extract filtered datasets for offline analysis, reporting, or compliance purposes. This makes it a central tool for institutional oversight of attendance patterns across users and locations.

Figure 5.10 shows a guardian-facing portal within an academic management system. It provides parents or guardians with a centralized hub to monitor their children's academic attendance and stay informed about their progress. The dashboard greets the logged-in parent by name and offers three primary navigation actions: viewing linked children's attendance and statistics, accessing notifications, and managing their own account profile. Below the action cards, each linked child is displayed as a summary card showing their name, student ID, email, and key attendance metrics ,including the number of enrolled courses, total sessions attended, and any pending leave requests. Quick-access buttons on each child's card allow the parent to directly navigate to that child's statistics, attendance history, timetable, and leave records. This module empowers parents

to stay actively involved in their child's academic attendance without requiring direct access to the student's account.

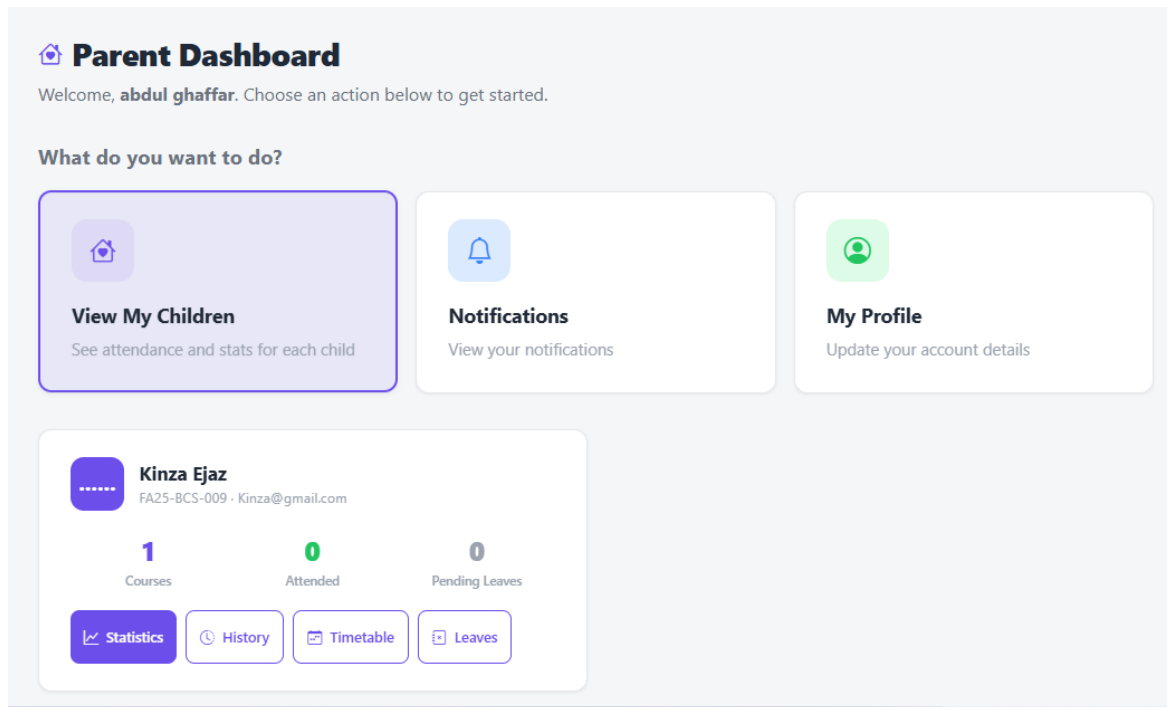


Figure 5.10: Parent Dashboard

Chapter 6

TESTING AND EVALUATION

6 Testing and Evaluation

This chapter demonstrates that the SmartID Ecosystem functions correctly, meets the specified requirements, and provides a reliable and secure experience for all users including students, faculty members, administrators, and parents. Testing has been performed both manually and through automated tools to verify that each module works as intended and that the system operates smoothly as a whole. Manual testing helps identify errors that may not be visible during normal use, while automated testing improves efficiency and repeatability. During testing, several issues were discovered, documented, and either resolved or acknowledged as known limitations arising from hardware constraints and environmental conditions.

6.1 Manual Testing

Manual testing is a process where a tester interacts directly with the SmartID Ecosystem to verify that all functionalities are working correctly. Unlike automated testing, manual testing relies on human observation to identify errors, usability issues, and unexpected behavior. It is a critical step in ensuring that the system meets its functional and non-functional requirements and provides a smooth experience for all user types. Manual testing in the SmartID Ecosystem is carried out in multiple stages including system testing, unit testing, functional testing, and integration testing before deployment.

6.1.1 System testing

Once the system has been successfully developed, testing has to be performed to ensure that the system is working as intended. This is also to check that the system meets the requirements stated earlier. Besides that, system testing will help in finding the errors that may be hidden from the user. There are few types of testing which includes the unit testing, functional testing and integration testing. The testing must be completed before it is being deployed for the user to use. In the SmartID Ecosystem, system testing validated the complete end-to-end flow from biometric authentication through to attendance recording, IoT door control, notification dispatch, and analytics generation. Several defects were uncovered during this stage, most notably face recognition failures under poor lighting, a timing conflict between the authentication service and the IoT relay under unstable network conditions, and an unhandled edge case in the predictive analytics module for students enrolled mid-semester.

6.1.2 Unit Testing

Unit Testing 1: User Enrollment with Valid Data

Testing Objective: To ensure that a new user can be successfully enrolled in the SmartID Ecosystem with complete and valid personal information and biometric consent.

Table 6.1: User Enrollment Unit Testing

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Submit enrollment form with complete valid data and accept consent	Name: Ali Hassan, ID: FA22-BSE-001, Email: ali@cuisahiwal.edu.pk , Auth Method: Face	User profile created successfully and enrollment confirmation email sent.	Pass
2.	Submit enrollment form with a duplicate email address already in the system	Email: ali@cuisahiwal.edu.pk (already registered)	Error message displayed: "Email already in use", no duplicate record created	Fail ,system created a duplicate entry without displaying an error; a uniqueness constraint was subsequently added to the email field and retested successfully

Unit Testing 2: Multi-Modal Authentication - Face Recognition

Testing Objective: To ensure that the facial recognition authentication method correctly verifies enrolled users under different environmental conditions.

Table 6.2: Facial Recognition unit testing

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Authenticate enrolled student using face recognition in adequate lighting	Live camera frame of enrolled student, well-lit environment	Authentication successful within 2 seconds, attendance trigger fired	Pass
2.	Authenticate enrolled student using face recognition in a dimly lit room	Live camera frame of enrolled student, low-light environment	Authentication successful	Fail, MTCNN could not detect the face region reliably; system prompted fallback to alternative method; acknowledged as hardware and environmental constraint

Unit Testing 3: Multi-Modal Authentication - RFID card

Testing Objective: To ensure that RFID card authentication correctly accepts registered cards and rejects unregistered ones.

Table 6.3: RFID unit testing

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Tap a registered RFID card at the SmartID terminal	RFID Tag ID: 4A3F2B1C (registered user)	Authentication successful, session created	Pass
2.	Tap an unregistered RFID card at the SmartID terminal	RFID Tag ID: 00000000 (not in database)	Authentication failed, failure event logged	Pass

Unit Testing 4: Real-Time Attendance Logging

Testing Objective: To ensure that attendance is automatically recorded after successful authentication and that duplicate records are not created on rapid re-scans.

Table 6.4: Real Time Attendance Unit Testing

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Authenticate student and verify attendance record is created automatically	UserID: FA22-BSE-001, CourseID: CS401, Timestamp: 08:00 AM	AttendanceRecord created with status PRESENT within 1 second	Pass
2.	Scan the same student a second time within the same session	UserID: FA22-BSE-001, second scan at 08:03 AM, same course	Existing record updated, no new duplicate record created	Fail: system created a second record; fixed after adding a duplicate-check query before insert; retested successfully

Unit Testing 5: IoT-Based Smart Access Control

Testing Objective: To ensure that the Raspberry Pi relay correctly unlocks the door for authorized users and denies access to unauthorized roles.

Table 6.5: IOT Based Access Unit Testing

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Authorized student attempts to access a classroom during permitted hours	UserID: FA22-BSE-001, DoorID: LAB-01, Time: 09:00 AM	GPIO pin set HIGH for 3 seconds, door unlocked, access event logged	Pass
2.	Student attempts to access a staff-only restricted area	UserID: FA22-BSE-001 (Student role), DoorID: STAFF-ROOM-01	Access denied, no unlock command issued, denial logged	Pass

Unit Testing 6: Student and Staff Notification

Testing Objective: To ensure that absence alert notifications are correctly dispatched to students, parents, and teachers when a student is marked absent.

Table 6.6: Noification Unit Testing

No.	Test case/ Test script	Attribute and value	Expected result	Result
1.	Trigger absence alert after student is marked absent in batch job	StudentID: FA22-BSE-001, CourseID: CS401, Date: 2025-11-10	Email alert sent to student and parent within 5 minutes	Fail ,email delivery failed when SMTP server was temporarily unreachable; a retry mechanism was not implemented at this stage; acknowledged as a known limitation to be addressed in future work
2.	Trigger in-app notification for failed authentication attempt	UserID: FA22-BSE-001, 3 consecutive failed face scans	In-web notification delivered to user and admin alert triggered	Pass

Unit Testing 7: Privacy and Consent Management

Testing Objective: To ensure that a user can revoke biometric consent and that their biometric data is deleted and the related authentication method is disabled.

Table 6.7: Privacy & consent Unit Testing

No.	Test case/ Test script	Attribute and value	Expected result	Result
1.	User revokes face recognition consent through the privacy portal	UserID: FA22-BSE-001, Action: Revoke Face Data	BiometricTemplate deleted from database, face authentication disabled for this user	Pass
2.	User attempts to authenticate with face after revoking consent	UserID: FA22-BSE-001, Method: Face	System rejects face authentication and prompts user to use alternative method	Pass

6.1.3 Functional Testing

The functional testing will take place after the unit testing. In this functional testing, the functionality of each module is tested. This is to ensure that the system produced meets the specifications and requirements. Functional testing in the SmartID Ecosystem verifies end-to-end workflows from the user's perspective, including role-based login, attendance viewing, report generation, access control enforcement, and notification delivery. Several functional defects were identified during this stage and resolved before final evaluation.

Functional Testing 1: Role-Based Login

Objective: To ensure that after login, the correct dashboard with the appropriate navigation options is loaded for each user role.

Table 6.8: Role Based Login Functional Testing

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Login as Admin with valid credentials	Username: admin@cuisahiwal.edu.pk , Password: Admin@123	Admin dashboard loaded with full system controls, user management, and configuration options	Pass
2.	Login as Faculty with valid credentials	Username: faculty@cuisahiwal.edu.pk , Password: Faculty@123	Faculty dashboard loaded showing class attendance records and report options	Pass
3	Login as Student with valid credentials	Username: ali@cuisahiwal.edu.pk , Password: Student@123	Student dashboard loaded showing personal attendance summary and notifications	Pass
4	Login with incorrect password	Username: ali@cuiwah.edu.pk , Password: WrongPass	Error message displayed: "Invalid credentials. Please try again."	Pass

Functional Testing 2: Account Lockout on Repeated Login Failures

Objective: To ensure that a user account is temporarily locked after exceeding the maximum number of failed login attempts and that an admin alert is triggered.

Table 6.9: Repeated Login Attempts Functional Testing

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Enter incorrect password five times consecutively	Username: ali@cuiwah.edu.pk , 5 consecutive wrong passwords	Account temporarily locked, admin alert triggered	Fail ,account was not locked after 5 failures; lockout threshold had not been configured in initial deployment; threshold was subsequently set in SystemSettings and retested successfully
2.	Attempt login after lockout period expires	Username: ali@cuiwah.edu.pk , after 10-minute lockout period	Login form accessible again, successful authentication accepted	Pass

Functional Testing 3: Attendance Report Generation

Objective: To ensure that the admin dashboard correctly generates and exports course-wise attendance reports for a selected date range.

Table 6.10: Attendance Report Generation Functional Testing

No.	Test case/ Test script	Attribute and value	Expected result	Result
1.	Generate attendance report for a specific course and date range	CourseID: CS401, DateRange: Nov 1–30, 2025	Report generated with present and absent breakdown per student, displayed correctly	Pass
2.	Export attendance report to CSV	CourseID: CS401, DateRange: Nov 1–30, 2025, Action: Export CSV	CSV file downloaded successfully	Fail ,export failed on first attempt due to a file path configuration error on the server; corrected and retested successfully

Functional Testing 4: Predictive Analytics Risk Classification

Objective: To ensure that the analytics module correctly classifies students into risk categories based on their absence rate and triggers alerts for HIGH-risk cases.

Table 6.11: Analysis risk Functional Testing

No.	Test case/ Test script	Attribute and value	Expected result	Result
1.	Run analytics for a student with 7 absences out of 20 lectures	StudentID: FA22-BSE-001, Absences: 7/20 (35% absence rate)	Student classified as HIGH risk, alert triggered to student, parent, and advisor	Fail : student was classified as MEDIUM due to an incorrect threshold value in SystemSettings; threshold corrected from 30% to 25% and retested successfully
2.	Run analytics for a student with 2 absences out of 20 lectures	StudentID: FA22-BSE-002, Absences: 2/20 (10% absence rate)	Student classified as LOW risk, no alert triggered	Pass

Functional Testing 5: IoT Access Denial Outside Permitted Hours

Objective: To ensure that the system correctly denies physical access when a user attempts entry outside the configured time window for their role.

Table 6.12: IOT Access Denial Functional Testing

No.	Test case/ Test script	Attribute and value	Expected result	Result
1.	Student attempts to access lab after permitted hours	UserID: FA22-BSE-001, DoorID: LAB-01, Time: 11:45 PM	Access denied, no unlock command issued, denial event logged	Pass
2.	Faculty member accesses lab within permitted hours	UserID: Faculty-001, DoorID: LAB-01, Time: 10:00 AM	Access granted, door unlocked, event logged	Pass

6.1.4 Integration Testing**Table 6.13: Integration Testing**

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Authenticate student and verify attendance is automatically recorded	UserID: FA22-BSE-001, Method: Face, CourseID: CS401, Time: 08:00 AM	Authentication succeeds and AttendanceRecord created with PRESENT status within 10 second	Pass
2.	Authenticate student and trigger IoT door unlock simultaneously	UserID: FA22-BSE-001, DoorID: LAB-01, Method: RFID, Time: 09:00 AM	Authentication succeeds, permission verified, GPIO relay activated, door unlocked	Fail on first test : unlock command arrived 6 seconds late under unstable Wi-Fi, exceeding the 3-second pulse window; timeout tolerance increased in the access control service and retested successfully
3.	Batch absence job marks absent student and triggers parental notification	StudentID: FA22-BSE-001, CourseID: CS401, Date: 2025-11-10, no scan recorded	ABSENT record created, email alert dispatched to student and parent	Pass (after SMTP retry mechanism was added following failure identified in Unit Testing 6)
4	Three consecutive failed authentication attempts trigger admin security alert	UserID: FA22-BSE-999 (unregistered), 3 failed face scans	Session locked, security alert sent once to admin	Fail : admin alert was sent twice due to a duplicate event trigger in the notification queue; resolved by adding an event deduplication check; retested successfully
5	IoT access event logged and reflected on admin dashboard	UserID: FA22-BSE-001, DoorID: LAB-01, GRANTED event at 09:00 AM	Access event logged in AccessLog table and visible on admin dashboard within 5 seconds	Pass

6.2 Automated Testing:

Tools used:

Table 6.14: Automated Testing

Tool Name	Tool Description	Applied on [list of related test cases / FR / NFR]	Results
Pytest	An open-source Python testing framework used to write and run automated unit and integration tests for backend modules	Authentication service methods (verifyFace, verifyRFID, verifyFingerprint), Attendance Manager (markAttendance, handleAbsence, duplicate prevention), Consent Service (updateConsent, deleteBiometricData), Analytics risk classification-FR-1, FR-2, FR-7	Majority of tests passed; risk classification threshold test initially failed due to incorrect SystemSettings value corrected and retested successfully
Postman	An API testing tool used to send HTTP requests to REST endpoints and verify response codes, payload structure, and error handling	Enrollment API, Authentication API, Attendance API, Access Control API, Notification API, System Settings API-FR-1, FR-2, FR-3, FR-5, FR-6, FR-8	All endpoints returned correct responses; the Notification API initially returned HTTP 200 even when SMTP delivery failed internally, fixed by implementing proper error propagation and retested
Selenium	A browser automation tool used to test the React frontend by simulating user interactions such as navigation, form submission, and button clicks	Admin login flow, student enrollment wizard, attendance report filtering, dashboard KPI rendering, notification log display, CSV export button - USE-1, USE-2, USE-3, USE-5	Navigation and rendering tests passed; CSV export button failed to trigger a download on first run due to a browser security policy conflict with response headers, resolved and retested successfully

Chapter 7

CONCLUSION

7 Conclusion and Future Work

7.1 Conclusion

The SmartID Ecosystem has been successfully designed and implemented as a comprehensive, hardware and web-based campus identity, attendance, and access control platform. Using a multi-tier architecture built on ReactJS for the frontend, Python and Flask for the backend, TensorFlow and FaceNet for machine learning-based facial recognition, and Raspberry Pi OS for IoT-enabled smart access control, the system delivers core functionalities including multi-modal biometric authentication, real-time attendance logging, IoT-based door access management, predictive analytics, parental notifications, and privacy-conscious biometric data handling. The modular design ensures that each component operates independently while communicating seamlessly through well-defined REST APIs, providing a scalable and maintainable solution for modern academic institutions.

Through careful system design, implementation, and rigorous testing including unit testing, functional testing, integration testing, and automated testing, the system has proven to be reliable, secure, and user-friendly. The SmartID Ecosystem meets all specified functional and non-functional requirements and provides a seamless experience for students, faculty members, administrators, and parents. The integration of external libraries and APIs such as FaceNet, MTCNN, RPi.GPIO, MFRC522, Flask, and the SMTP Email API further enhances the system's authentication accuracy, hardware responsiveness, and communication reliability. Overall, the project demonstrates how modern software engineering practices combined with IoT and artificial intelligence can be leveraged to eliminate manual attendance processes, improve campus security, promote institutional transparency, and support data-driven decision-making in educational environments.

7.2 Future Work

While the current version of the SmartID Ecosystem successfully covers core attendance management and campus access control functionalities, there are several opportunities for future enhancement that could further increase its impact, scalability, and intelligence.

7.2.1 Mobile Application Integration

In future versions, the system can be extended to include dedicated mobile applications for students, faculty members, and parents, allowing them to receive real-time attendance notifications, view reports, and manage consent settings directly from their smartphones. A mobile application would significantly improve accessibility and engagement, especially for parents who need timely alerts about their child's attendance status without requiring access to a web browser.

7.2.2 Multi-Campus Deployment

The SmartID Ecosystem can be extended to support multiple campuses or departments within a university network, allowing centralized administration of user accounts, attendance records, and access policies across geographically distributed locations. This would enable institutions to maintain synchronized records, generate cross-campus analytics, and enforce consistent access control policies from a single administrative panel.

7.2.3 QR Code and Voice Authentication

Future updates could introduce additional authentication modes such as QR code scanning and voice recognition, further increasing inclusivity and flexibility for users who may have difficulties using face recognition or fingerprint scanning. These additional modalities would expand the system's coverage in diverse physical environments and strengthen its multi-modal identity verification framework without requiring significant changes to the existing architecture.

7.2.4 Learning Management System Integration

The SmartID Ecosystem could be integrated with existing Learning Management Systems such as Moodle or the COMSATS university portal, allowing attendance records to be automatically reflected in the academic performance tracking tools used by faculty and administration. This integration would eliminate data silos, reduce administrative duplication, and provide students with a unified view of their academic engagement and attendance standing.

7.2.5 Advanced Predictive Analytics and AI Insights

Future improvements could expand the current predictive analytics module to incorporate more sophisticated machine learning models that analyze not only attendance patterns but also access behavior, time-of-arrival trends, and course-specific engagement metrics. These advanced models would enable the system to generate more accurate early-warning indicators for at-risk students and provide administrators with deeper institutional insights to support academic counseling and resource planning decisions.

7.2.6 Enhanced Security and Privacy Features

Future versions of the system can enhance security through end-to-end encrypted communication, blockchain-based audit logging, and AI-driven anomaly detection for identifying suspicious authentication activities. These improvements would strengthen compliance with data protection standards and increase trust in the SmartID Ecosystem as a long-term campus security solution.

With these enhancements, the SmartID Ecosystem could evolve into a comprehensive smart campus management platform, delivering advanced, adaptive services for universities, administrators, faculty, students, and parents.

References

- [1] Jibble, "What Is a Biometric Attendance System?," [Online]. Available: https://www.jibble.io/article/what-are-biometric-attendance-systems?utm_source=chatgpt.com.
- [2] Tipsoi, "Best Face Recognition Attendance Systems for 2025 (Top 8 List)," [Online]. Available: https://tipsoi.pro/face-recognition-attendance-system/?utm_source=chatgpt.com.
- [3] Decibel, "Employee Attendance Software in Pakistan: Top Picks for 2025," [Online]. Available: https://decibel360cloud.com/employee-attendance-software-in-pakistan-for-2025/?utm_source=chatgpt.com.
- [4] P. HRMS, "Attendance Management System for Modern Workforce," [Online]. Available: https://www.pockethrms.com/attendance-management-system/?utm_source=chatgpt.com.
- [5] Arxiv, "AttenFace: A Real Time Attendance System using Face Recognition," [Online]. Available: https://arxiv.org/abs/2211.07582?utm_source=chatgpt.com.
- [6] Tarz, "Top 10 Biometric Attendance Machine You Should Buy in Pakistan," [Online]. Available: https://tarz.pk/top-10-biometric-attendance-machine-buy-in-pakistan/?utm_source=chatgpt.com.
- [7] ZKTeco, "Time Attendance," [Online]. Available: https://www.zkteco.com.pk/TimeAttendancef4p?utm_source=chatgpt.com.
- [8] K. K. & P. S. M. Patel, "Internet of Things-IOT: Definition, Characteristics, Architecture, Enabling Technologies, Application and Future Challenges.," *International Journal of Engineering Science and Computing*, 2016.
- [9] A. K. R. A. & P. S. Jain, "An Introduction to Biometric Recognition," in *IEEE Transactions on Circuits and Systems for Video Technology*.
- [10] N. S. a. A. K. P. M. Puan, "A new unique and fast fingerprint matching algorithm using score based adaptive partitioning," in *IEEE Transactions on Information Forensics and Security*, vol. 3, 2011.
- [11] R. Want, "An introduction to RFID technology," in *IEEE Pervasive Computing*, vol. 1, 2006.
- [12] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed., New York: McGraw-Hill, 2014.
- [13] we360ai, "The Ultimate Guide to Automated Attendance Systems: How They Work and Why They Matter," [Online]. Available: https://we360.ai/blog/the-ultimate-guide-to-automated-attendance-systems-how-they-work-and-why-they-matter?utm_source=chatgpt.com.
- [14] Reddit, "Seeking Open Source Alternatives to Cams Biometrics for Access Control & Attendance Management," [Online]. Available: https://www.reddit.com/r/opensource/comments/18ggmme?utm_source=chatgpt.com.
- [15] R. Want, "An introduction to RFID Technology," in *IEEE Pervasive Computing*.

